

计算物理学

Matt Li

2026 年 6 月 18 日

目录

1	计算物理基础	6
1.1	数据存储	6
1.1.1	单精度浮点数	6
1.1.2	双精度浮点数	7
1.2	计算误差	7
1.2.1	舍入误差	7
1.2.2	截断误差	8
2	线性代数问题	8
2.1	范数	8
2.1.1	向量的范数	8
2.1.2	矩阵的范数	9
2.2	Gauss 消元法	9
2.3	增广矩阵法	10
2.4	继续消元法	11
2.5	列主元消元法	12
2.6	LU 分解法	12
2.6.1	LU 分解介绍	12
2.6.2	高斯消元与 LU 分解的关系	14
2.6.3	列主元 LU 分解	16
2.6.4	LU 分解求解逆矩阵与行列式	18
2.6.5	三对角矩阵的 LU 分解	19
2.7	LDL^T 分解	20

2.8	Cholesky 分解	20
2.9	Jacobi 迭代	21
2.10	最速下降法	22
2.11	共轭梯度法	23
2.12	QR 分解法	26
2.13	SVD 分解	27
2.14	Sherman-Morrison 公式	28
3	插值方法	29
3.1	拉格朗日插值	29
3.2	Hermite 插值	30
3.3	三次样条插值	31
3.4	Akima 插值	33
3.5	有理函数插值	35
3.6	Pade 近似	35
4	数值微分和积分	36
4.1	一阶微分公式	36
4.2	二阶微分公式	38
4.3	Simpson 公式	38
4.4	牛顿-科特斯公式	39
4.5	复合积分公式	40
4.6	余项外推	40
4.7	龙贝格算法	41
4.8	高斯积分	43
4.9	带权函数的高斯积分	47
5	非线性方程	48
5.1	二分法	49
5.2	不动点法	49
5.3	牛顿法	50
5.3.1	牛顿法求解单重根问题	50
5.3.2	牛顿法求解多重根问题	51
5.4	割线法 (Secant Method)	51
5.5	Aitken 加速算法	52
5.6	Steffensen 方法	52

5.7	秦九韶算法	53
5.8	Brent 方法	53
5.9	牛顿法求解非线性方程组	54
5.10	准牛顿法 (Broyden 方法)	56
5.11	改进的 Broyden 方法	58
6	优化问题	58
6.1	Fletcher-Reeves 算法	58
6.2	线性共轭梯度法	59
6.3	拟牛顿法优化	60
6.4	Broyden 算法	60
6.5	BFGS 算法	61
6.6	线性规划问题—单纯形法	62
6.7	下山单纯形法 (Nelder-Mead 方法)	65
6.8	模拟退火法	66
6.9	遗传算法	67
7	本征值问题	68
7.1	Jacobi 方法	68
7.2	Householder 变换	70
7.3	QR 分解	73
7.4	带位移的 QR 分解	75
7.5	广义特征值问题	76
7.6	SVD 分解	76
7.7	幂法 (Power Method)	79
7.8	反幂法	82
7.9	Wielandt 收缩法	82
7.10	Krylov 子空间方法	84
7.11	Lanczos 迭代	85
7.12	Arnoldi 迭代	87
7.13	DMRG 算法	87
8	蒙特卡洛算法 (Monte Carlo Method)	88
8.1	统计概率基础	88
8.2	随机投点法	89
8.3	期望值估计法	90

8.4	一维随机行走	92
8.5	马尔科夫过程 (Markov Process)	92
8.5.1	时间演化	93
8.5.2	扩散方程	94
8.6	Metropolis 算法	97
8.6.1	平衡态统计物理应用	98
8.7	MCMC 积分	99
8.8	哈密顿蒙特卡洛	100
8.9	量子蒙特卡洛 (QMC)	100
8.10	扩散蒙特卡洛 (DMC)	102
9	微分方程	102
9.1	欧拉法 (Euler's Method)	102
9.2	高阶 Taylor 展开法	103
9.3	Runge-Kutta 方法	104
9.3.1	中点法	105
9.3.2	改进的欧拉法	105
9.3.3	三阶 Runge-Kutta 方法	106
9.3.4	四阶 Runge-Kutta 方法	106
9.3.5	Runge-Kutta-Fehlberg 方法	106
9.4	多步法——Adams 预测-校正方法	107
9.4.1	Adams-Moulton 三阶隐式方法	107
9.4.2	Adams-Bashforth 三阶显式方法	107
9.4.3	Adams-Bashforth 四阶显式方法	107
9.5	扩展的四阶 Runge-Kutta 方法	107
9.6	高阶常微分方程	108
9.7	刚性微分方程	110
9.7.1	隐式梯形法	111
9.7.2	Gear 方法	111
9.8	显性与隐性欧拉法的稳定性分析	112
9.9	线性打靶法	113
9.10	非线性打靶法	114
9.11	Numerov 方法	115
9.12	Green 函数法	117
9.13	有限差分法	119

10 偏微分方程	120
10.1 有限差分法	120
10.2 Fourier 变换—Fourier 微分算符	123
10.3 基矢展开法	125
10.3.1 轴对称量子系统	125
10.3.2 三维直角坐标系下的基矢展开法	127
10.4 离散变量表示 (DVR)	127
10.5 B-spline 展开法	130
10.6 小波分析	131
10.7 有限元法 (FEM)	131
11 动力学方程	134
11.1 一维扩散方程	134
11.1.1 有限差分法	135
11.1.2 Crank-Nicolson 格式	135
11.2 波动方程	137
11.2.1 一维波动方程的有限差分法	137
11.2.2 二维波动方程的有限差分法	137
11.2.3 变系数波动方程的有限差分法	138
11.3 量子力学中的时间演化	139
11.3.1 时间演化算符	139
11.3.2 有限差分法	139
12 神经网络	140
12.1 bp 神经网络	140
12.2 贝叶斯神经网络	142
12.3 正则化	144
12.4 奥卡姆剃刀	145
13 量子计算	145
13.1 MPI	145
13.2 OpenMP	147
13.3 量子计算	147

1 计算物理基础

1.1 数据存储

数据以位、字节存储。位 (bit): 0, 1; 字节 (byte): 8bit=1byte

整型 (int): 表示整数; 浮点 (float): 表示实数。

数值精度大小是由限制的, 浮点数分为单精度浮点数与双精度浮点数。

1.1.1 单精度浮点数

占用 4 字节 (32 位), 能够表示 7 位有效数字。32 位分为 1 个符号位, 8 个指数位和 23 和小数位。

$$\begin{array}{c} \text{S} \underbrace{\text{EEE} \cdots \text{EEE}}_{8 \text{ 个 E}} \underbrace{\text{FFF} \cdots \text{FFF}}_{23 \text{ 个 F}} \end{array}$$

(i) S 为符号位: 0 为正, 1 为负;

(ii) E 为指数位, 表示 2 的幂次。存在 127 的偏移量, 即

$$\text{实际指数} = \text{存储指数} - 127$$

除去全 0 (00000000) 和全 1 (11111111) (有其他用途), 指数范围为 1 ~ 254 (00000001 ~ 11111110), 于是实际指数范围为 -126 ~ 127, 能够表示

$$2^{-126} \sim 2^{127}$$

的数字, 大概是 $10^{-38} \sim 10^{38}$ 的范围。

(iii) F 为小数位, FFF...FFF 表示一个小于 1 的小数 (记作 F), 实际的小数值为 1.F。

于是, 32 位形成的单精度浮点数为

$$(-1)^s \times 1.F \times 2^{E-127}$$

一些特殊情况:

E=0, F≠0, 值为

$$(-1)^s \times 2^{-126} \times 0.F$$

E=255, F=0, 值为

$$(-1)^s \infty$$

E=255, F≠0, 值为 NaN (运算结果无意义)

例: 将 -5.75 转化为单精度浮点数。

需要先将 -5.75 转化为二进制数:

$$-5.75 = -101.11 = -1.0111 \times 2^2$$

于是: 符号位 S=1, 指数位 E=10000001, 小数位 F=0111000...000

1.1.2 双精度浮点数

占用 8 字节 (64 位), 能够表示 16 个有效数字。分为 1 个符号位, 11 个指数位和 52 个小数位。

$$\begin{array}{c} \text{S} \underbrace{\text{EEE} \dots \text{EEE}}_{11 \text{ 个 E}} \underbrace{\text{FFF} \dots \text{FFF}}_{52 \text{ 个 F}} \end{array}$$

- (i) S 为符号位: 0 为正, 1 为负;
 - (ii) E 为指数位, 偏移量为 1023;
 - (iii) F 为小数位, 表示一个小于 1 的小数 (记作 F), 实际小数为 1.F。
- 最终得到的值为

$$(-1)^s \times 1.F \times 2^{E-1023}$$

大概能表示 $4.9 \times 10^{-324} \sim 1.7 \times 10^{308}$ 的范围。

1.2 计算误差

1.2.1 舍入误差

由于中间结果舍入引起的误差。

例: 求解一元二次方程

$$x^2 + 62.10x + 1 = 0$$

精确解为:

$$x_1 = -0.01610723, x_2 = -62.08390$$

数值计算中, 用 4 位有效数字的浮点数来表示, 已知求根公式

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

已知

$$b = 62.10, a = 1.000, c = 1.000$$

$$b^2 = 3856, 4ac = 4.000$$

$$b^2 - 4ac = 62.06$$

$$x_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a} = -0.02000$$

当 $b > 0, |ac| \ll b^2$ 时, 相对误差是较大的, 应该避免两个相近的数字相减, 应该使用公式:

$$x_1 = \frac{2c}{-b - \sqrt{b^2 - 4ac}}$$

1.2.2 截断误差

如 Taylor 展开的计算中

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots + \frac{x^n}{n!} + R_n(x)$$

其中拉格朗日余项

$$R_n(x) = \frac{e^\xi}{(n+1)!} x^{n+1}, \xi = (0, x)$$

2 线性代数问题

该部分旨在求解线性方程组：

$$a_{00}x_0 + a_{01}x_1 + a_{02}x_2 + \cdots + a_{0,n-1}x_{n-1} = b_0$$

$$a_{10}x_0 + a_{11}x_1 + a_{12}x_2 + \cdots + a_{1,n-1}x_{n-1} = b_1$$

$$a_{20}x_0 + a_{21}x_1 + a_{22}x_2 + \cdots + a_{2,n-1}x_{n-1} = b_2$$

...

$$a_{m-1,0}x_0 + a_{m-1,1}x_1 + a_{m-1,2}x_2 + \cdots + a_{m-1,n-1}x_{n-1} = b_{m-1}$$

或者形成矩阵形式：

$$\mathbf{A} \cdot \vec{x} = \vec{b}$$

2.1 范数

2.1.1 向量的范数

向量的范数可以用来测量向量的长度，定义：

(i) 1-范数

$$||\vec{x}||_1 = |x_0| + |x_1| + \cdots + |x_{n-1}|$$

即曼哈顿距离。

(ii) 2-范数

$$||\vec{x}||_2 = \sqrt{|x_0|^2 + |x_1|^2 + \cdots + |x_{n-1}|^2}$$

即欧几里得距离。

(iii) ∞ -范数

$$||\vec{x}||_\infty = \max_{0 \leq i \leq n-1} |x_i|$$

p -范数

$$\|\vec{x}\|_p = (|x_0|^p + |x_1|^p + \cdots + |x_{n-1}|^p)^{\frac{1}{p}}$$

柯西-施瓦茨不等式:

$$|\vec{x}^T \vec{y}| \leq \|\vec{x}\|_2 \|\vec{y}\|_2$$

三角不等式:

$$\|\vec{x} + \vec{y}\| \leq \|\vec{x}\| + \|\vec{y}\|$$

2.1.2 矩阵的范数

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=0}^{m-1} \sum_{j=0}^{n-1} |a_{ij}|^2}$$

$$\|\mathbf{A}\|_\infty = \max_{0 \leq i \leq m-1} \sum_{j=0}^{n-1} |a_{ij}|$$

$$\|\mathbf{A}\vec{z}\| \leq \|\mathbf{A}\| \cdot \|\vec{z}\|$$

可以定义条件数:

$$K(\mathbf{A}) = \|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\|$$

条件数反映了线性方程组的稳定性, 条件数越大, 线性方程组越不稳定。如果条件数很大时, 就要进行预处理, 使得

$$K(\mathbf{A}) \approx 1$$

2.2 Gauss 消元法

Gauss 消元法是最经典的求解线性方程组的方法, 考虑线性方程组:

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} w_1 \\ w_2 \\ w_3 \\ w_4 \end{pmatrix}$$

逐列消元, 先消去第一列:

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ 0 & a_{22} - \frac{a_{21}a_{12}}{a_{11}} & a_{23} - \frac{a_{21}a_{13}}{a_{11}} & a_{24} - \frac{a_{21}a_{14}}{a_{11}} \\ 0 & a_{32} - \frac{a_{31}a_{12}}{a_{11}} & a_{33} - \frac{a_{31}a_{13}}{a_{11}} & a_{34} - \frac{a_{31}a_{14}}{a_{11}} \\ 0 & a_{42} - \frac{a_{41}a_{12}}{a_{11}} & a_{43} - \frac{a_{41}a_{13}}{a_{11}} & a_{44} - \frac{a_{41}a_{14}}{a_{11}} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} y_1 \\ w_2^{(2)} \\ w_3^{(2)} \\ w_4^{(2)} \end{pmatrix}$$

可以把方程组写成如下形式：

$$\begin{pmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ 0 & a_{22}^{(2)} & a_{23}^{(2)} & a_{24}^{(2)} \\ 0 & a_{32}^{(2)} & a_{33}^{(2)} & a_{34}^{(2)} \\ 0 & a_{42}^{(2)} & a_{43}^{(2)} & a_{44}^{(2)} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} y_1 \\ w_2^{(2)} \\ w_3^{(2)} \\ w_4^{(2)} \end{pmatrix}$$

则新、旧系数的关系为：

$$\begin{aligned} b_{1k} &= a_{1k}, k = 1, 2, 3, 4 \\ a_{ik}^{(2)} &= a_{ik} - \frac{a_{i1}a_{1k}}{a_{11}}, i, k = 2, 3, 4 \\ y_1 &= w_1 \\ w_i^{(2)} &= w_i - \frac{a_{i1}w_1}{a_{11}}, i = 2, 3, 4 \end{aligned}$$

再以此类推，向前消元，得到系数的关系：

$$\begin{aligned} a_{ik}^{(m+1)} &= a_{ik}^{(m)} - \frac{a_{im}^{(m)}a_{mk}^{(m)}}{a_{mm}^{(m)}}, i, k = m+1, m+2, \dots, n \\ w_i^{(m+1)} &= w_i^{(m)} - \frac{a_{im}^{(m)}w_m^{(m)}}{a_{mm}^{(m)}}, i = m+1, m+2, \dots, n \end{aligned}$$

最终变成上三角矩阵：

$$\begin{pmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ 0 & b_{22} & b_{23} & b_{24} \\ 0 & 0 & b_{33} & b_{34} \\ 0 & 0 & 0 & b_{44} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{pmatrix}$$

向后回代得到最终的解：

$$x_m = \frac{1}{b_{mm}} \left(y_m - \sum_{k=m+1}^n b_{mk} x_k \right), m = n-1, n-2, \dots, 1$$

2.3 增广矩阵法

增广矩阵法是 Gauss 消元法的变种，直接在原矩阵的基础上进行消元，得到增广矩阵：

$$\mathbf{A} = [A, y] = \begin{pmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} & y_1 \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} & y_2 \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} & y_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \cdots & a_{nn} & y_n \end{pmatrix}$$

第一次消元，把第一列元素消为零：

$$E'_j = E_j - \frac{a_{j1}}{a_{11}} E_1, j = 2, 3, \dots, n$$

以此类推，第 m 次消元，把第 m 列元素消为零：

$$E'_j = E_j - \frac{a_{jm}^{(m)}}{a_{mm}^{(m)}} E_m, j = m+1, m+2, \dots, n$$

最后得到上三角矩阵：

$$\begin{pmatrix} b_{11} & b_{12} & b_{13} & \cdots & b_{1n} & w_1 \\ 0 & b_{22} & b_{23} & \cdots & b_{2n} & w_2 \\ 0 & 0 & b_{33} & \cdots & b_{3n} & w_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & b_{nn} & w_n \end{pmatrix}$$

向后回代得到方程的解：

$$x_m = \frac{1}{b_{mm}} \left(w_m - \sum_{k=m+1}^n b_{mk} x_k \right), m = n-1, n-2, \dots, 1$$

2.4 继续消元法

继续消元法是增广矩阵法的变种，在得到上三角矩阵后继续消元，得到对角矩阵：

$$\begin{pmatrix} a_{11}^{(1)} & 0 & 0 & \cdots & 0 & w_1^{(1)} \\ 0 & a_{22}^{(2)} & 0 & \cdots & 0 & w_2^{(2)} \\ 0 & 0 & a_{33}^{(3)} & \cdots & 0 & w_3^{(3)} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & a_{nn}^{(n)} & w_n^{(n)} \end{pmatrix}$$

向后回代得到方程的解：

$$x_m = \frac{w_m^{(m)}}{a_{mm}^{(m)}}, m = n-1, n-2, \dots, 1$$

这种方法对于求解相似的方程组十分有效：

$$\begin{cases} \mathbf{A}\vec{x} = \vec{b}_1 \\ \mathbf{A}\vec{x} = \vec{b}_2 \end{cases}$$

因为继续消元法实际上已经保存了系数矩阵消元结果，不需要再重复消元。

2.5 列主元消元法

列主元消元法是 Gauss 消元法的改进，避免了当 $a_{mm}^{(m)}$ 较小时引入较大的误差。每次消元前，先在第 m 列中寻找绝对值最大的元素，将其所在行（假设为 p 行）与第 m 行交换，然后再进行消元。即，如果找到

$$|a_{pm}^{(m)}| = \max_{m \leq i \leq n} |a_{im}^{(m)}|$$

则交换 m, p 行：

$$E_m \leftrightarrow E_p$$

例：求解线性方程组

$$\begin{cases} 0.003x_1 + 59.14x_2 = 59.17 \\ 5.291x_1 - 6.13x_2 = 46.78 \end{cases}$$

如果不选主元，消元的乘数：

$$m = \frac{5.291}{0.003} = 1763.67$$

乘数非常大，此时舍入误差会被放大，得到的结果不准确：

$$x_1 = -10.00, x_2 = 1.001$$

如果选主元，第一列的绝对值最大元素为 5.291，交换两行：

$$\begin{cases} 5.291x_1 - 6.13x_2 = 46.78 \\ 0.003x_1 + 59.14x_2 = 59.17 \end{cases}$$

消元的乘数：

$$m = \frac{0.003}{5.291} = 0.000567$$

乘数很小，得到的结果较为准确：

$$x_1 = 10.00, x_2 = 1.000$$

2.6 LU 分解法

2.6.1 LU 分解介绍

LU 分解法是将系数矩阵 $\mathbf{A}$ 分解为一个下三角矩阵 $\mathbf{L}$ 和一个上三角矩阵 $\mathbf{U}$ 的乘积，即

$$\mathbf{A} = \mathbf{L} \cdot \mathbf{U}$$

$$\begin{pmatrix} \alpha_{11} & 0 & 0 & \cdots & 0 \\ \alpha_{21} & \alpha_{22} & 0 & \cdots & 0 \\ \alpha_{31} & \alpha_{32} & \alpha_{33} & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \alpha_{n1} & \alpha_{n2} & \alpha_{n3} & \cdots & \alpha_{nn} \end{pmatrix} \cdot \begin{pmatrix} \beta_{11} & \beta_{12} & \beta_{13} & \cdots & \beta_{1n} \\ 0 & \beta_{22} & \beta_{23} & \cdots & \beta_{2n} \\ 0 & 0 & \beta_{33} & \cdots & \beta_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & \beta_{nn} \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \cdots & a_{nn} \end{pmatrix}$$

然后将方程组 $\mathbf{A}\vec{x} = \vec{b}$ 转化为两个方程组：

$$\mathbf{L}\vec{y} = \vec{b}$$

$$\mathbf{U}\vec{x} = \vec{y}$$

对于第一个方程 $\mathbf{L}\vec{y} = \vec{b}$ ，向前回代得到方程的解：

$$y_1 = \frac{b_1}{\alpha_{11}}$$

$$y_m = \frac{1}{\alpha_{mm}} \left(b_m - \sum_{k=1}^{m-1} \alpha_{mk} y_k \right), m = 2, 3, \dots, n$$

对于第二个方程 $\mathbf{U}\vec{x} = \vec{y}$ ，向后回代得到方程的解：

$$x_n = \frac{y_n}{\beta_{nn}}$$

$$x_m = \frac{1}{\beta_{mm}} \left(y_m - \sum_{k=m+1}^n \beta_{mk} x_k \right), m = n-1, n-2, \dots, 1$$

下面来考虑如何将系数矩阵 $\mathbf{A}$ 拆分成 $\mathbf{L}$ 和 $\mathbf{U}$ 的乘积。为方便，将 $\mathbf{L}$ 的对角元素都设为 1，这样是合理的，因为我们可以把 $\mathbf{U}$ 的对角元乘一个相应的系数。即

$$\mathbf{L} = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ \alpha_{21} & 1 & 0 & \cdots & 0 \\ \alpha_{31} & \alpha_{32} & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \alpha_{n1} & \alpha_{n2} & \alpha_{n3} & \cdots & 1 \end{pmatrix}$$

$$\mathbf{U} = \begin{pmatrix} \beta_{11} & \beta_{12} & \beta_{13} & \cdots & \beta_{1n} \\ 0 & \beta_{22} & \beta_{23} & \cdots & \beta_{2n} \\ 0 & 0 & \beta_{33} & \cdots & \beta_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & \beta_{nn} \end{pmatrix}$$

可以得到矩阵元的关系：

$$a_{ij} = \begin{cases} \alpha_{i1}\beta_{1j} + \alpha_{i2}\beta_{2j} + \cdots + \beta_{ij}, i < j \\ \alpha_{i1}\beta_{1j} + \alpha_{i2}\beta_{2j} + \cdots + \alpha_{ij}\beta_{jj}, i > j \\ \alpha_{i1}\beta_{1j} + \alpha_{i2}\beta_{2j} + \cdots + \beta_{jj}, i = j \end{cases}$$

求解 $\mathbf{L}$ 和 $\mathbf{U}$ 的算法叫作 Doolittle 算法，按照行优先的顺序计算 $\mathbf{L}$ 和 $\mathbf{U}$ 的元素。对于第 1 行，先计算 $\mathbf{U}$ 的第一行元素，再求解 $\mathbf{L}$ 的第一列：

$$\beta_{1j} = a_{1j}, j = 1, 2, \cdots, n$$

因为 $\mathbf{L}$ 的第一行元素为 $(1, 0, 0, \cdots, 0)$ 。

$$\alpha_{i1}\beta_{11} = a_{i1} \Rightarrow \alpha_{i1} = \frac{a_{i1}}{\beta_{11}}, i = 1, 2, \cdots, n$$

再从 2 到 $n-1$ ，依次进行递推。先求 $\mathbf{U}$ 的第 i 个对角元元素。

$$a_{ii} = \sum_{k=1}^{i-1} \alpha_{ik}\beta_{ki} + \beta_{ii} \Rightarrow \beta_{ii} = a_{ii} - \sum_{k=1}^{i-1} \alpha_{ik}\beta_{ki}$$

再求 $\mathbf{U}$ 的 i 行非对角元元素：

$$a_{ij} = \sum_{k=1}^{i-1} \alpha_{ik}\beta_{kj} + \beta_{ij} \Rightarrow \beta_{ij} = a_{ij} - \sum_{k=1}^{i-1} \alpha_{ik}\beta_{kj}, j = i+1, i+2, \cdots, n$$

至此， $\mathbf{U}$ 的第 i 行元素已经全部求出。下面来求 $\mathbf{L}$ 的第 j 列元素。

$$a_{ij} = \sum_{k=1}^{j-1} \alpha_{ik}\beta_{kj} + \alpha_{ij}\beta_{jj} \Rightarrow \alpha_{ij} = \frac{1}{\beta_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} \alpha_{ik}\beta_{kj} \right), i = j+1, j+2, \cdots, n$$

至此， $\mathbf{L}$ 的第 j 列元素已经全部求出。最后求 $\mathbf{U}$ 的第 n 个对角元元素。

$$a_{nn} = \sum_{k=1}^{n-1} \alpha_{nk}\beta_{kn} + \beta_{nn} \Rightarrow \beta_{nn} = a_{nn} - \sum_{k=1}^{n-1} \alpha_{nk}\beta_{kn}$$

所有元素均已求出，完成了 $\mathbf{LU}$ 分解。

2.6.2 高斯消元与 $\mathbf{LU}$ 分解的关系

对于高斯消元，每一步消元的过程相当于对系数矩阵 $\mathbf{A}$ 左乘一个矩阵，最后变为上三角矩阵 $\mathbf{U}$ ：

$$\mathbf{U} = \mathbf{G}_{n-1}\mathbf{G}_{n-2}\cdots\mathbf{G}_1\mathbf{A}$$

而 **LU** 分解就相当于高斯消元的逆记录：

$$\mathbf{A} = \mathbf{G}_1^{-1} \mathbf{G}_2^{-1} \cdots \mathbf{G}_{n-1}^{-1} \mathbf{U} = \mathbf{L} \mathbf{U}$$

实际上，可以把高斯消元写作矩阵形式，第一次消元：

$$\mathbf{E}'_j = \mathbf{E}_j - \frac{a_{j1}^{(1)}}{a_{11}^{(1)}} \mathbf{E}_1$$

定义

$$m_{j1} = \frac{a_{j1}^{(1)}}{a_{11}^{(1)}}$$

于是高斯消元相当于左乘矩阵：

$$\mathbf{M}^{(1)} = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ -m_{21} & 1 & 0 & \cdots & 0 \\ -m_{31} & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ -m_{n1} & 0 & 0 & \cdots & 1 \end{pmatrix}$$

相当于

$$\mathbf{A}^{(2)} \vec{x} = \mathbf{M}^{(1)} \mathbf{A} \vec{x} = \mathbf{M}^{(1)} \vec{b}^{(1)} = \vec{b}^{(2)}$$

第二次消元，同理：

$$m_{j2} = \frac{a_{j2}^{(2)}}{a_{22}^{(2)}}$$

对应矩阵 $\mathbf{M}^{(2)}$ ：

$$\mathbf{M}^{(2)} = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ 0 & -m_{32} & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & -m_{n2} & 0 & \cdots & 1 \end{pmatrix}$$

$$\mathbf{A}^{(3)} \vec{x} = \mathbf{M}^{(2)} \mathbf{A}^{(2)} \vec{x} = \mathbf{M}^{(2)} \mathbf{M}^{(1)} \mathbf{A} \vec{x} = \mathbf{M}^{(2)} \vec{b}^{(2)} = \vec{b}^{(3)}$$

以此类推：

$$\mathbf{M}^{(k)} = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 & \cdots & 0 \\ 0 & 0 & 0 & \cdots & -m_{k+1,k} & \cdots & 0 \\ \vdots & \vdots & \vdots & \cdots & \cdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & -m_{nk} & \cdots & 1 \end{pmatrix}$$

$$\mathbf{A}^{(k+1)} \vec{x} = \mathbf{M}^{(k)} \mathbf{A}^{(k)} \vec{x} = \mathbf{M}^{(k)} \mathbf{M}^{(k-1)} \cdots \mathbf{M}^{(1)} \mathbf{A} \vec{x} = \mathbf{M}^{(k)} \vec{b}^{(k)} = \vec{b}^{(k+1)}$$

得到上三角矩阵：

$$\mathbf{A}^{(n)} = \mathbf{M}^{(n-1)} \mathbf{M}^{(n-2)} \cdots \mathbf{M}^{(1)} \mathbf{A} = \mathbf{U}$$

$$\Rightarrow \mathbf{A} = \mathbf{M}^{(1)-1} \mathbf{M}^{(2)-1} \cdots \mathbf{M}^{(n-1)-1} \mathbf{U} = \mathbf{L} \mathbf{U}$$

$$\mathbf{L} = \mathbf{M}^{(1)-1} \mathbf{M}^{(2)-1} \cdots \mathbf{M}^{(n-1)-1} = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ m_{21} & 1 & 0 & \cdots & 0 \\ m_{31} & m_{32} & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ m_{n1} & m_{n2} & m_{n3} & \cdots & 1 \end{pmatrix}$$

可以看到， $\mathbf{U}$ 就是消元后的上三角矩阵， $\mathbf{L}$ 记录着消元乘数。

2.6.3 列主元 LU 分解

列主元 $\mathbf{LU}$ 分解是 $\mathbf{LU}$ 分解的改进，避免了当 $a_{mm}^{(m)}$ 较小时引入较大的误差。其思路和列主元高斯消元法类似，可以利用置换矩阵 $\mathbf{P}$ 来记录行交换。若原始行的顺序为 $1, 2, 3, \dots, n$ ，交换后行的顺序为 $k_1, k_2, k_3, \dots, k_n$ ，则

$$\mathbf{P}_{ij} = \begin{cases} 1, j = k_i \\ 0, j \neq k_i \end{cases}$$

i 行 j 列为 1，当且仅当新的第 i 行来自于原来的第 j 行。

左乘矩阵 $\mathbf{P}$ 相当于对矩阵 $\mathbf{A}$ 作行变换。

例：对矩阵

$$\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

作行变换，新行顺序为 (1, 3, 2)，对应的置换矩阵为

$$\mathbf{P} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

$$\mathbf{PA} = \begin{pmatrix} 1 & 2 & 3 \\ 7 & 8 & 9 \\ 4 & 5 & 6 \end{pmatrix}$$

$\mathbf{P}$ 是正交矩阵：

$$\mathbf{P}^T = \mathbf{P}^{-1}$$

$$\Rightarrow \mathbf{A} = \mathbf{P}^T \mathbf{LU}$$

实际计算的过程是交替进行的，即：换行、消元 + 换行、消元 + $\cdots$ + 换行、消元，即

$$\mathbf{M}_{n-1} \mathbf{P}_{n-1} \cdots \mathbf{M}_2 \mathbf{P}_2 \mathbf{M}_1 \mathbf{P}_1 \mathbf{A} = \mathbf{U}$$

其中 $\mathbf{M}$ 为消元矩阵， $\mathbf{P}$ 为置换矩阵。

现在考虑，有没有一种方法，让所有的 $\mathbf{P}$ 都在右边，所有的 $\mathbf{M}$ 都在左边？即

$$\mathbf{M}_3 \mathbf{P}_3 \mathbf{M}_2 \mathbf{P}_2 \mathbf{M}_1 \mathbf{P}_1 = \mathbf{M}'_3 \mathbf{M}'_2 \mathbf{M}'_1 \mathbf{P}'_3 \mathbf{P}'_2 \mathbf{P}'_1$$

很容易看到

$$\mathbf{M}'_3 = \mathbf{M}_3$$

$$\mathbf{M}'_2 = \mathbf{P}_3 \mathbf{M}_2 \mathbf{P}_3^{-1}$$

$$\mathbf{M}'_1 = \mathbf{P}_3 \mathbf{P}_2 \mathbf{M}_1 \mathbf{P}_2^{-1} \mathbf{P}_3^{-1}$$

可以写成

$$\mathbf{M}'_{n-1} \mathbf{M}'_{n-2} \cdots \mathbf{M}'_1 \mathbf{P}_{n-1} \mathbf{P}_{n-2} \cdots \mathbf{P}_1 \mathbf{A} = \mathbf{U}$$

定义

$$\mathbf{P} = \mathbf{P}_{n-1} \mathbf{P}_{n-2} \cdots \mathbf{P}_1$$

$$\mathbf{L} = (\mathbf{M}'_{n-1} \mathbf{M}'_{n-2} \cdots \mathbf{M}'_1)^{-1}$$

于是

$$\mathbf{PA} = \mathbf{LU}$$

2.6.4 LU 分解求解逆矩阵与行列式

对于矩阵求逆：

$$\mathbf{A} = \mathbf{L}\mathbf{U} \Rightarrow \mathbf{A}^{-1} = \mathbf{U}^{-1}\mathbf{L}^{-1}$$

于是问题变为分别求解 $\mathbf{L}$ 和 $\mathbf{U}$ 的逆矩阵。对于 $\mathbf{L}$ ：

$$\mathbf{L}\mathbf{L}^{-1} = \mathbf{I}$$

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ l_{21} & 1 & 0 & 0 \\ l_{31} & l_{32} & 1 & 0 \\ l_{41} & l_{42} & l_{43} & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ d_{21} & 1 & 0 & 0 \\ d_{31} & d_{32} & 1 & 0 \\ d_{41} & d_{42} & d_{43} & 1 \end{pmatrix} = \mathbf{I}$$

$$d_{ij} = -l_{ij} - \sum_{k=j+1}^{i-1} l_{ik}d_{kj}, i > j$$

对于 $\mathbf{U}$ ：

$$\mathbf{U}^{-1}\mathbf{U} = \mathbf{I}$$

$$\begin{pmatrix} e_{11} & e_{12} & e_{13} & e_{14} \\ 0 & e_{22} & e_{23} & e_{24} \\ 0 & 0 & e_{33} & e_{34} \\ 0 & 0 & 0 & e_{44} \end{pmatrix} \begin{pmatrix} u_{11} & u_{12} & u_{13} & u_{14} \\ 0 & u_{22} & u_{23} & u_{24} \\ 0 & 0 & u_{33} & u_{34} \\ 0 & 0 & 0 & u_{44} \end{pmatrix} = \mathbf{I}$$

$$e_{ij} = -\frac{1}{u_{jj}} \sum_{k=i}^{j-1} e_{ik}u_{kj}$$

$$e_{ii} = \frac{1}{u_{ii}}$$

矩阵求行列式：

$$\det(\mathbf{A}) = \det(\mathbf{L}) \cdot \det(\mathbf{U})$$

$$\det(\mathbf{L}) = 1$$

$$\Rightarrow \det(\mathbf{A}) = \det(\mathbf{U}) = \prod_{i=1}^n u_{ii}$$

2.6.5 三对角矩阵的 LU 分解

对于三对角矩阵 $\mathbf{A}$, 可以写作

$$\mathbf{A} = \begin{pmatrix} b_1 & c_1 & 0 & \cdots & 0 \\ a_2 & b_2 & c_2 & \cdots & 0 \\ 0 & a_3 & b_3 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & b_n \end{pmatrix}$$

以三对角矩阵为系数矩阵的线性方程组:

$$\mathbf{A}\vec{x} = \vec{f}$$

$$\Rightarrow a_i x_{i-1} + b_i x_i + c_i x_{i+1} = f_i, i = 1, 2, \cdots, n$$

其中我们规定:

$$a_1 = 0, c_n = 0$$

可以作一类特殊的 LU 分解:

$$\begin{aligned} \mathbf{A} &= \mathbf{L}\mathbf{U} \\ &= \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ \alpha_2 & 1 & 0 & \cdots & 0 \\ 0 & \alpha_3 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{pmatrix} \begin{pmatrix} \beta_1 & c_1 & 0 & \cdots & 0 \\ 0 & \beta_2 & c_2 & \cdots & 0 \\ 0 & 0 & \beta_3 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & \beta_n \end{pmatrix} \end{aligned}$$

递推关系为

$$\beta_1 = b_1$$

$$\alpha_i = \frac{a_i}{\beta_{i-1}}, i = 2, 3, \cdots, n$$

$$\beta_i = b_i - \alpha_i c_{i-1}, i = 2, 3, \cdots, n$$

它的应用是求解一维 Poisson 方程:

$$-\frac{d^2 u}{dx^2} = f(x)$$

离散化: 把区间 $[0, L]$ 等分为 $n+1$ 个小段, 每段长为 $h = \frac{L}{n+1}$, 记 $x_i = ih$, 对应着 $u_i = u(x_i), f_i = f(x_i)$ 。同时满足齐次 Dirichlet 边界条件: $u_0 = u_{n+1} = 0$ 。利用差分公式:

$$f''(x_i) = \frac{f_{i+1} - 2f_i + f_{i-1}}{h^2}$$

得到方程：

$$-f_{i+1} + 2f_i - f_{i-1} = h^2 f_i, i = 1, 2, \dots, n$$

写成矩阵形式：

$$\begin{pmatrix} 2 & -1 & 0 & \cdots & 0 \\ -1 & 2 & -1 & \cdots & 0 \\ 0 & -1 & 2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 2 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \\ \vdots \\ u_n \end{pmatrix} = \begin{pmatrix} h^2 f_1 \\ h^2 f_2 \\ h^2 f_3 \\ \vdots \\ h^2 f_n \end{pmatrix}$$

可以看到，系数矩阵是一个对称正定的三对角矩阵。

2.7 LDL^T 分解

对于对称正定矩阵，可以作 **LDL^T** 分解：若 **A** 满足

$$\mathbf{A} = \mathbf{A}^T$$

$$\mathbf{x}^T \mathbf{A} \mathbf{x} > 0, \forall \mathbf{x} \neq \mathbf{0}$$

$$\Rightarrow \mathbf{A} = \mathbf{L} \mathbf{D} \mathbf{L}^T$$

其中 **L** 为单位下三角矩阵，**D** 为对角矩阵。即

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{12} & a_{22} & a_{23} \\ a_{13} & a_{23} & a_{33} \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{pmatrix} \begin{pmatrix} d_{11} & 0 & 0 \\ 0 & d_{22} & 0 \\ 0 & 0 & d_{33} \end{pmatrix} \begin{pmatrix} 1 & l_{21} & l_{31} \\ 0 & 1 & l_{32} \\ 0 & 0 & 1 \end{pmatrix}$$

2.8 Cholesky 分解

对于对称正定矩阵 **A**，满足

$$\mathbf{A} = \mathbf{A}^T$$

$$\mathbf{x}^T \mathbf{A} \mathbf{x} > 0, \forall \mathbf{x} \neq \mathbf{0}$$

则 **A** 可以作 Cholesky 分解：

$$\mathbf{A} = \mathbf{L} \mathbf{L}^T$$

其中 **L** 为下三角矩阵，考虑三维情形：

$$\mathbf{A} = \mathbf{L} \mathbf{L}^T = \begin{pmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{pmatrix} \begin{pmatrix} l_{11} & l_{21} & l_{31} \\ 0 & l_{22} & l_{32} \\ 0 & 0 & l_{33} \end{pmatrix} = \begin{pmatrix} l_{11}^2 & l_{11}l_{21} & l_{11}l_{31} \\ l_{11}l_{21} & l_{21}^2 + l_{22}^2 & l_{21}l_{31} + l_{22}l_{32} \\ l_{11}l_{31} & l_{21}l_{31} + l_{22}l_{32} & l_{31}^2 + l_{32}^2 + l_{33}^2 \end{pmatrix}$$

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{12} & a_{22} & a_{23} \\ a_{13} & a_{23} & a_{33} \end{pmatrix}$$

先求第一列:

$$l_{11} = \sqrt{a_{11}}$$

$$l_{i1} = \frac{a_{i1}}{l_{11}}, i = 2, 3, \dots, n$$

再求第 m 列:

$$\sum_{k=1}^m l_{mk}^2 = a_{mm} \Rightarrow l_{mm} = \sqrt{a_{mm} - \sum_{k=1}^{m-1} l_{mk}^2}$$

$$\sum_{k=1}^m l_{ik} l_{mk} = a_{im} \Rightarrow l_{im} = \frac{1}{l_{mm}} \left(a_{im} - \sum_{k=1}^{m-1} l_{ik} l_{mk} \right), i = m+1, m+2, \dots, n$$

2.9 Jacobi 迭代

Jacobi 迭代是求解线性方程组 $\mathbf{A}\vec{x} = \vec{b}$ 的一种迭代方法。可以把线性方程组的第 i 个方程写作

$$a_{i1}x_1 + a_{i2}x_2 + \dots + a_{ii}x_i + \dots + a_{in}x_n = b_i$$

$$\Rightarrow x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1, j \neq i}^n a_{ij}x_j \right)$$

代入一组初始值 (例如 $(0, 0, 0, \dots, 0)$), 得到第一轮迭代的结果, 继续迭代, 直到满足某个收敛条件为止。递推关系为

$$x_i^{(k)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1, j \neq i}^n a_{ij}x_j^{(k-1)} \right), i = 1, 2, \dots, n$$

收敛条件 (例):

$$\frac{\|x^k - x^{k-1}\|_{\infty}}{\|x^k\|_{\infty}} < 10^{-3}$$

有一些改进方法, 比如 Gauss-Seidel 迭代法。它的思路是, 每次迭代求解 x_i^k 时, 因为前面的 $x_1^k, \dots, x_{i-1}^k$ 已经求出了, 可以直接利用迭代后的解, 递推关系为

$$x_i^k = \frac{1}{a_{ii}} \left[b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^n a_{ij}x_j^{(k-1)} \right], i = 1, 2, \dots, n$$

SOR 方法则引入了一个松弛因子 ω , 递推关系为

$$x_i^k = (1 - \omega)x_i^{(k-1)} + \frac{\omega}{a_{ii}} \left[b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^n a_{ij}x_j^{(k-1)} \right], i = 1, 2, \dots, n$$

可以把 Jacobi 迭代写成统一的矩阵形式

$$\mathbf{A} = \mathbf{D} - \mathbf{L} - \mathbf{U}$$

其中 $\mathbf{D}$ 是对角矩阵, $\mathbf{L}$ 是严格下三角矩阵, $\mathbf{U}$ 是严格上三角矩阵。

$$\mathbf{A} = \begin{pmatrix} a_{11} & 0 & 0 & \cdots & 0 \\ 0 & a_{22} & 0 & \cdots & 0 \\ 0 & 0 & a_{33} & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & a_{nn} \end{pmatrix} - \begin{pmatrix} 0 & 0 & 0 & \cdots & 0 \\ a_{21} & 0 & 0 & \cdots & 0 \\ a_{31} & a_{32} & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \cdots & 0 \end{pmatrix} - \begin{pmatrix} 0 & a_{12} & a_{13} & \cdots & a_{1n} \\ 0 & 0 & a_{23} & \cdots & a_{2n} \\ 0 & 0 & 0 & \cdots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 0 \end{pmatrix}$$

于是

$$\mathbf{A}\vec{x} = \vec{b} \Rightarrow \mathbf{D}\vec{x} = (\mathbf{L} + \mathbf{U})\vec{x} + \vec{b}$$

$$\vec{x} = \mathbf{D}^{-1}(\mathbf{L} + \mathbf{U})\vec{x} + \mathbf{D}^{-1}\vec{b}$$

迭代:

$$\vec{x}^{(k)} = \mathbf{D}^{-1}(\mathbf{L} + \mathbf{U})\vec{x}^{(k-1)} + \mathbf{D}^{-1}\vec{b}$$

2.10 最速下降法

最速下降法是求解线性方程组 $\mathbf{A}\vec{x} = \vec{b}$ 的一种迭代方法, 适用于系数矩阵 $\mathbf{A}$ 为对称正定矩阵的情况。它的思路是把求解线性方程组转化为求解一个二次函数的最小值问题。定义二次函数

$$g(\vec{x}) = \frac{1}{2}\vec{x}^T \mathbf{A}\vec{x} - \vec{b}^T \vec{x}$$

求梯度:

$$\nabla g(\vec{x}) = \mathbf{A}\vec{x} - \vec{b}$$

$\nabla g = 0$ 的位置即为方程组的解。考虑试探解:

$$\begin{aligned} g(\vec{x} + t\vec{v}) &= \frac{1}{2}(\vec{x} + t\vec{v})^T \mathbf{A}(\vec{x} + t\vec{v}) - \vec{b}^T(\vec{x} + t\vec{v}) \\ &= \frac{1}{2}\vec{x}^T \mathbf{A}\vec{x} + t\vec{v}^T \mathbf{A}\vec{x} + \frac{1}{2}t^2\vec{v}^T \mathbf{A}\vec{v} - \vec{b}^T \vec{x} - t\vec{b}^T \vec{v} \\ &= \left(\frac{1}{2}\vec{x}^T \mathbf{A}\vec{x} - \vec{b}^T \vec{x} \right) + t \left(\mathbf{A}\vec{x} - \vec{b} \right)^T \vec{v} + \frac{1}{2}t^2\vec{v}^T \mathbf{A}\vec{v} \\ &= g(\vec{x}) - t \left\langle \vec{b} - \mathbf{A}\vec{x}, \vec{v} \right\rangle + \frac{1}{2}t^2 \langle \vec{v}, \mathbf{A}\vec{v} \rangle \end{aligned}$$

其中, 内积的定义为

$$\left\langle \vec{b} - \mathbf{A}\vec{x}, \vec{v} \right\rangle = \vec{b}^T \vec{v} - \vec{x}^T \mathbf{A}\vec{v}$$

或者可以把内积写成下面的形式，都是一样的，本篇选用最后一种形式：

$$(\vec{b} - \mathbf{A}\vec{x}) \cdot \vec{v} = (\vec{b} - \mathbf{A}\vec{x}, \vec{v}) = \langle \vec{b} - \mathbf{A}\vec{x}, \vec{v} \rangle$$

这可以看作一个关于步长 t 的二次函数，其最小值点为

$$\hat{t} = \frac{\langle \vec{b} - \mathbf{A}\vec{x}, \vec{v} \rangle}{\langle \vec{v}, \mathbf{A}\vec{v} \rangle}$$

也就是

$$g(\vec{x} + \hat{t}\vec{v}) < g(\vec{x}), v \neq 0$$

因此，沿着 $\vec{v}$ 的方向前进 $\hat{t}$ 的距离，可以使得 g 的值减小。再进行迭代，新步长：

$$t_k = \frac{\langle \vec{v}^{(k)}, \vec{b} - \mathbf{A}\vec{x}^{(k-1)} \rangle}{\langle \vec{v}^{(k)}, \mathbf{A}\vec{v}^{(k)} \rangle}$$

新位置：

$$\vec{x}^{(k+1)} = \vec{x}^{(k)} + t_k \vec{v}^{(k)}$$

可以用残差来定义新方向：

$$\vec{v}^{(k+1)} = \vec{b} - \mathbf{A}\vec{x}^{(k)}$$

2.11 共轭梯度法

前面的最速下降法每次迭代的方向都是残差的方向，可能会出现震荡的情况。共轭梯度法的搜索方向不仅依赖于残差的方向，还依赖于上一次迭代的搜索方向，最终使得每次迭代的搜索方向都是共轭的。

定义 **A-共轭**：对于非零向量 $\vec{v}$ 和 $\vec{w}$ ，如果满足

$$\langle \vec{v}, \mathbf{A}\vec{w} \rangle = 0$$

则称 $\vec{v}$ 和 $\vec{w}$ 是 **A-共轭** 的。

共轭梯度法要求满足条件：

(i) 共轭条件

$$\langle \vec{v}^{(i)}, \mathbf{A}\vec{v}^{(j)} \rangle = 0, i \neq j$$

(ii) 残差正交条件

$$\langle \vec{r}^i, \vec{r}^j \rangle = 0, i \neq j$$

(iii) 残差与旧方向正交条件

$$\langle \vec{r}^{(k)}, \vec{v}^{(j)} \rangle = 0, j = 1, 2, \dots, k$$

递推公式为

$$\vec{v}^{(k+1)} = \vec{r}^{(k)} + s_k \vec{v}^{(k)}$$

共轭条件:

$$\begin{aligned}\langle \vec{v}^{(k)}, \mathbf{A} \vec{v}^{(k+1)} \rangle &= 0 \\ \Rightarrow \langle \vec{v}^{(k)}, \mathbf{A} \vec{r}^{(k)} + \mathbf{A} s_k \vec{v}^{(k)} \rangle &= 0 \\ \langle \vec{v}^{(k)}, \mathbf{A} \vec{r}^{(k)} \rangle + s_k \langle \vec{v}^{(k)}, \mathbf{A} \vec{v}^{(k)} \rangle &= 0\end{aligned}$$

解得

$$s_k = -\frac{\langle \vec{v}^{(k)}, \mathbf{A} \vec{r}^{(k)} \rangle}{\langle \vec{v}^{(k)}, \mathbf{A} \vec{v}^{(k)} \rangle}$$

再选择合适的步长 t_k 以保证残差条件 $\langle \vec{r}^{(i)}, \vec{r}^{(j)} \rangle = 0$, 在前面的最速下降法中已经计算得到

$$\begin{aligned}t_k &= \frac{\langle \vec{v}^{(k)}, \vec{r}^{(k-1)} \rangle}{\langle \vec{v}^{(k)}, \mathbf{A} \vec{v}^{(k)} \rangle} = \frac{\langle \vec{r}^{(k-1)} + s_{k-1} \vec{v}^{(k-1)}, \vec{r}^{(k-1)} \rangle}{\langle \vec{v}^{(k)}, \mathbf{A} \vec{v}^{(k)} \rangle} \\ &= \frac{\langle \vec{r}^{(k-1)}, \vec{r}^{(k-1)} \rangle}{\langle \vec{v}^{(k)}, \mathbf{A} \vec{v}^{(k)} \rangle} + s_{k-1} \frac{\langle \vec{v}^{(k-1)}, \vec{r}^{(k-1)} \rangle}{\langle \vec{v}^{(k)}, \mathbf{A} \vec{v}^{(k)} \rangle} \\ &= \frac{\langle \vec{r}^{(k-1)}, \vec{r}^{(k-1)} \rangle}{\langle \vec{v}^{(k)}, \mathbf{A} \vec{v}^{(k)} \rangle}\end{aligned}$$

已知

$$\begin{aligned}\vec{r}^{(k)} - \vec{r}^{(k-1)} &= -\mathbf{A} (\vec{x}^{(k)} - \vec{x}^{(k-1)}) = -t_k \mathbf{A} \vec{x}^{(k)} \\ \langle \vec{r}^{(k)}, \vec{r}^{(k)} \rangle &= \langle \vec{r}^{(k-1)}, \vec{r}^{(k)} \rangle - t_k \langle \mathbf{A} \vec{v}^{(k)}, \vec{r}^{(k)} \rangle = -t_k \langle \mathbf{A} \vec{v}^{(k)}, \vec{r}^{(k)} \rangle \\ \Rightarrow s_k &= \frac{\langle \vec{r}^{(k)}, \vec{r}^{(k)} \rangle}{\langle \vec{r}^{(k-1)}, \vec{r}^{(k-1)} \rangle}\end{aligned}$$

于是最终按顺序的递推关系为

$$\begin{aligned}\vec{r}^{(0)} &= \vec{b} - \mathbf{A} \vec{x}^{(0)} \\ \vec{v}^{(1)} &= \vec{r}^{(0)} \\ \vec{x}^{(k)} &= \vec{x}^{(k-1)} + t_k \vec{v}^{(k)} \\ \vec{r}^{(k)} &= \vec{r}^{(k-1)} - t_k \mathbf{A} \vec{v}^{(k)} \\ \vec{v}^{(k+1)} &= \vec{r}^{(k)} + s_k \vec{v}^{(k)}\end{aligned}$$

例: 考虑线性方程组

$$\begin{pmatrix} 4 & 1 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

先初始化，设

$$\vec{x}_0 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$$

残差：

$$\vec{r}_0 = \vec{b} - \mathbf{A}\vec{x}_0 = \begin{pmatrix} 1 \\ 2 \end{pmatrix} - \begin{pmatrix} 4 & 1 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} -8 \\ -3 \end{pmatrix}$$

搜索方向：

$$\vec{v}_1 = \vec{r}_0 = \begin{pmatrix} -8 \\ -3 \end{pmatrix}$$

步长：

$$t_1 = \frac{\langle \vec{v}_1, \vec{r}_0 \rangle}{\langle \vec{v}_1, \mathbf{A}\vec{v}_1 \rangle} = \frac{73}{331}$$

更新解：

$$\vec{x}_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix} + \frac{73}{331} \begin{pmatrix} -8 \\ -3 \end{pmatrix} = \begin{pmatrix} 0.2356 \\ 0.3384 \end{pmatrix}$$

残差：

$$\vec{r}_1 = \vec{r}_0 - t_1 \mathbf{A}\vec{v}_1 = \begin{pmatrix} -8 \\ -3 \end{pmatrix} - \frac{73}{331} \begin{pmatrix} 4 & 1 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} -8 \\ -3 \end{pmatrix} = \begin{pmatrix} -0.2810 \\ 0.7492 \end{pmatrix}$$

系数：

$$s_1 = \frac{\langle \vec{r}^{(1)}, \vec{r}^{(1)} \rangle}{\langle \vec{r}^{(0)}, \vec{r}^{(0)} \rangle} = 0.0088$$

搜索方向：

$$\vec{v}_2 = \vec{r}_1 + s_1 \vec{v}_1 = \begin{pmatrix} -0.2810 \\ 0.7492 \end{pmatrix} + 0.0088 \begin{pmatrix} -8 \\ -3 \end{pmatrix} = \begin{pmatrix} -0.3511 \\ 0.7229 \end{pmatrix}$$

步长：

$$t_2 = \frac{\langle \vec{v}_2, \vec{r}_1 \rangle}{\langle \vec{v}_2, \mathbf{A}\vec{v}_2 \rangle} = 0.4122$$

更新解：

$$x_2 = \begin{pmatrix} 0.2356 \\ 0.3384 \end{pmatrix} + 0.4122 \begin{pmatrix} -0.3511 \\ 0.7229 \end{pmatrix} = \begin{pmatrix} 0.0909 \\ 0.6364 \end{pmatrix} \approx \begin{pmatrix} \frac{1}{11} \\ \frac{7}{11} \end{pmatrix}$$

与精确解已经十分接近了。

2.12 QR 分解法

QR 分解法是另外一种矩阵的分解方法：

$$\mathbf{A} = \mathbf{Q}\mathbf{R}$$

其中 $\mathbf{Q}$ 为正交矩阵， $\mathbf{R}$ 为上三角矩阵。

$$\mathbf{A} = (q_1, q_2, \dots, q_n) \begin{pmatrix} r_{11} & r_{12} & \cdots & r_{1n} \\ 0 & r_{22} & \cdots & r_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & r_{nn} \end{pmatrix}$$

$$\mathbf{Q}\mathbf{Q}^T = \mathbf{I}$$

于是线性方程组可以写作

$$\mathbf{A}\vec{x} = \vec{b} \Rightarrow \mathbf{R}\vec{x} = \mathbf{Q}^T\vec{b}$$

QR 分解有很多实现方法，这里先介绍 Gram-Schmidt 正交化方法。在后面求解本征值问题时，再介绍 Householder 变换方法。

$$\mathbf{A} = (\vec{a}_1, \vec{a}_2, \dots, \vec{a}_n)$$

$$\mathbf{Q} = (\vec{e}_1, \vec{e}_2, \dots, \vec{e}_n)$$

其中矩阵 $\mathbf{A}$ 为列向量组组成的矩阵， $\mathbf{Q}$ 为一组标准正交基组成的矩阵。

可以将矩阵列向量组 $(\vec{a}_1, \vec{a}_2, \dots, \vec{a}_n)$ 变换成正交列向量组 $(\vec{u}_1, \vec{u}_2, \dots, \vec{u}_n)$ 。

$$\vec{u}_1 = \vec{a}_1, \vec{e}_1 = \frac{\vec{u}_1}{|\vec{u}_1|}$$

$$\vec{u}_2 = \vec{a}_2 - \langle \vec{a}_2, \vec{e}_1 \rangle \vec{e}_1, \vec{e}_2 = \frac{\vec{u}_2}{|\vec{u}_2|}$$

$$\vec{u}_n = \vec{a}_n - \sum_{j=1}^{n-1} \langle \vec{a}_n, \vec{e}_j \rangle \vec{e}_j, \vec{e}_n = \frac{\vec{u}_n}{|\vec{u}_n|}$$

于是

$$\mathbf{R} = \begin{pmatrix} \langle \vec{a}_1, \vec{e}_1 \rangle & \langle \vec{a}_2, \vec{e}_1 \rangle & \cdots & \langle \vec{a}_n, \vec{e}_1 \rangle \\ 0 & \langle \vec{a}_2, \vec{e}_2 \rangle & \cdots & \langle \vec{a}_n, \vec{e}_2 \rangle \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \langle \vec{a}_n, \vec{e}_n \rangle \end{pmatrix}$$

$$\mathbf{A} = (\vec{e}_1, \vec{e}_2, \dots, \vec{e}_n) \begin{pmatrix} \langle \vec{a}_1, \vec{e}_1 \rangle & \langle \vec{a}_2, \vec{e}_1 \rangle & \cdots & \langle \vec{a}_n, \vec{e}_1 \rangle \\ 0 & \langle \vec{a}_2, \vec{e}_2 \rangle & \cdots & \langle \vec{a}_n, \vec{e}_2 \rangle \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \langle \vec{a}_n, \vec{e}_n \rangle \end{pmatrix} = \mathbf{Q}\mathbf{R}$$

$$\vec{a}_k = \sum_{j=1}^k \langle \vec{a}_k, \vec{e}_j \rangle \vec{e}_j$$

例：考虑矩阵

$$\mathbf{A} = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix}$$

$$\vec{u}_1 = \vec{a}_1 = (1, 1, 0)^T$$

$$\vec{e}_1 = \frac{\vec{u}_1}{|\vec{u}_1|} = \frac{1}{\sqrt{2}} (1, 1, 0)^T$$

$$\vec{u}_2 = \vec{a}_2 - \langle \vec{a}_2, \vec{e}_1 \rangle \vec{e}_1 = \left(\frac{1}{2}, -\frac{1}{2}, 0 \right)^T$$

$$\vec{e}_2 = \frac{1}{\sqrt{6}} (1, -2, 1)^T$$

$$\vec{u}_3 = \vec{a}_3 - \langle \vec{a}_3, \vec{e}_1 \rangle \vec{e}_1 - \langle \vec{a}_3, \vec{e}_2 \rangle \vec{e}_2 = \frac{1}{\sqrt{3}} (-1, 1, 1)^T$$

得到

$$\mathbf{Q} = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{6}} & -\frac{1}{\sqrt{3}} \\ \frac{1}{\sqrt{2}} & -\frac{2}{\sqrt{6}} & \frac{1}{\sqrt{3}} \\ 0 & \frac{1}{\sqrt{6}} & \frac{1}{\sqrt{3}} \end{pmatrix}$$

$$\mathbf{R} = \begin{pmatrix} \sqrt{2} & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 0 & \frac{2}{\sqrt{6}} & \frac{1}{\sqrt{6}} \\ 0 & 0 & \frac{1}{\sqrt{3}} \end{pmatrix}$$

2.13 SVD 分解

$m \times n$ 矩阵 $\mathbf{A}$ 还可以分解为如下形式：

$$\mathbf{A} = \mathbf{U}\mathbf{D}\mathbf{V}^T$$

其中 $\mathbf{U}$ 为 $m \times m$ 正交矩阵， $\mathbf{V}$ 为 $n \times n$ 正交矩阵， $\mathbf{D}$ 为 $m \times n$ 对角矩阵，对角线上的元素为 $\mathbf{A}$ 的奇异值。可以将 $\mathbf{U}$ 的向量称为左奇异向量， $\mathbf{V}^T$ 的向量称为右奇异向量。

$$\mathbf{U}\mathbf{U}^T = \mathbf{V}\mathbf{V}^T = \mathbf{I}$$

$$\mathbf{D} = \begin{pmatrix} \omega_1 & 0 & \cdots & 0 \\ 0 & \omega_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \omega_n \end{pmatrix}$$

在实际求解线性方程组的过程中，经常用到系数矩阵的逆

$$\mathbf{A}^{-1} = \mathbf{V}\mathbf{S}^{-1}\mathbf{U}^T$$

为了减小误差，消除噪声，常常会把小于某个阈值的奇异值 ω_j 置零，这样就得到了矩阵 $\mathbf{A}$ 的一个近似逆，即

$$\mathbf{D}_0^{-1} = \begin{cases} \frac{1}{\sigma_i}, \sigma_i > \epsilon \\ 0, \sigma_i \leq \epsilon \end{cases}$$

2.14 Sherman-Morrison 公式

Sherman-Morrison 公式提供了一种计算矩阵的逆的方法。

设 $\mathbf{A}$ 为一个可逆矩阵， $\vec{u}$ 和 $\vec{v}$ 为列向量，考虑矩阵

$$\mathbf{B} = \mathbf{A} + \vec{u}\vec{v}^T$$

的逆矩阵。其中 $\vec{u}$ 是 m 维向量， $\vec{v}$ 是 n 维向量。这是一种秩-1 更新。

$$(\vec{u}\vec{v}^T)_{ij} = u_i v_j$$

$$(\mathbf{A} + \vec{u}\vec{v}^T)^{-1} = (\mathbf{I} + \mathbf{A}^{-1}\vec{u}\vec{v}^T)^{-1} \mathbf{A}^{-1}$$

考虑级数展开：

$$\begin{aligned} (\mathbf{I} - \mathbf{M})^{-1} &= \mathbf{I} + \mathbf{M} + \mathbf{M}^2 + \mathbf{M}^3 + \cdots \\ \Rightarrow (\mathbf{I} + \mathbf{A}^{-1}\vec{u}\vec{v}^T)^{-1} &= \mathbf{I} - \mathbf{A}^{-1}\vec{u}\vec{v}^T + \mathbf{A}^{-1}\vec{u}\vec{v}^T \mathbf{A}^{-1}\vec{u}\vec{v}^T - \cdots \end{aligned}$$

定义

$$\lambda = \vec{v}^T \mathbf{A}^{-1} \vec{u}$$

$$\begin{aligned} (\mathbf{A} + \vec{u}\vec{v}^T)^{-1} &= \mathbf{A}^{-1} - \mathbf{A}^{-1}\vec{u}\vec{v}^T \mathbf{A}^{-1} (1 - \lambda + \lambda^2 - \cdots) \\ &= \mathbf{A}^{-1} - \frac{\mathbf{A}^{-1}\vec{u}\vec{v}^T \mathbf{A}^{-1}}{1 + \lambda} \end{aligned}$$

$$(\mathbf{A} + \vec{u}\vec{v}^T)^{-1} = \mathbf{A}^{-1} - \frac{\mathbf{A}^{-1}\vec{u}\vec{v}^T \mathbf{A}^{-1}}{1 + \vec{v}^T \mathbf{A}^{-1} \vec{u}}$$

考虑方程组：

$$(\mathbf{A} + \vec{u}\vec{v}^T) \vec{x} = \vec{b}$$

已知方程组的解：

$$\mathbf{A}\vec{y} = \vec{b}$$

$$\mathbf{A}\vec{z} = \vec{u}$$

方程的解为：

$$\vec{x} = \vec{y} - \frac{\vec{v}^T \vec{y}}{1 + \vec{v}^T \vec{z}} \vec{z}$$

3 插值方法

3.1 拉格朗日插值

考虑拟合一个函数，其中 $n + 1$ 个点的函数值已知：

$$y_0 = f(x_0), y_1 = f(x_1), \dots, y_n = f(x_n)$$

拉格朗日插值要求拟合函数 $P(x)$ 的每个点都经过插值点：

$$P(x_k) = f(x_k), k = 0, 1, 2, \dots, n$$

拉格朗日插值函数 $P(x)$ ：

$$P(x) = f(x_0) L_{n,0}(x) + f(x_1) L_{n,1}(x) + \dots + f(x_n) L_{n,n}(x) = \sum_{k=0}^n f(x_k) L_{n,k}(x)$$

插值基函数：

$$\begin{aligned} L_{n,k}(x) &= \frac{(x - x_0)(x - x_1) \cdots (x - x_{k-1})(x - x_{k+1}) \cdots (x - x_n)}{(x_k - x_0)(x_k - x_1) \cdots (x_k - x_{k-1})(x_k - x_{k+1}) \cdots (x_k - x_n)} \\ &= \prod_{i=0, i \neq k}^n \frac{(x - x_i)}{(x_k - x_i)} \end{aligned}$$

可以看到

$$L_k(x_i) = \begin{cases} 1, k = i \\ 0, k \neq i \end{cases}$$

例：两点直线插值

$$P_1(x) = \frac{x - x_0}{x_1 - x_0} y_1 + \frac{x - x_1}{x_0 - x_1} y_0$$

三点插值

$$P_3(x) = \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}y_2 + \frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)}y_1 + \frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)}y_0$$

插值余项:

$$f(x) = P(x) + \frac{f^{(n+1)}(\xi(x))}{(n+1)!} (x-x_0)(x-x_1)\cdots(x-x_n), \xi \in (x_0, x_n)$$

例: $f(x) = x^{-1}$, 计算插值误差。插值点为 $x_0 = 2, x_1 = 2.75, x_2 = 4$ 。

拉格朗日插值拟合得:

$$P(x) = \frac{1}{22}x^2 - \frac{35}{88}x + \frac{49}{44}$$

$$f^{(3)}(x) = -6x^{-4}$$

$$R(x) = \frac{f^{(3)}(\xi)}{3!} (x-x_0)(x-x_1)(x-x_2) = -\xi^{-4} (x-2)(x-2.75)(x-4)$$

求极值:

$$\max \xi^{-4} = \frac{1}{16}$$

$$\max (x-2)(x-2.75)(x-4) = -\frac{9}{16}$$

$$\Rightarrow |R(x)| \leq 0.00586$$

3.2 Hermite 插值

Hermite 插值是拉格朗日插值的升级版, 不仅要求拟合函数和原函数在插值点的值相同, 还要求在插值点的斜率相同。插值函数

$$H_{2n+1}(x) = \sum_{i=0}^n f(x_i) H_{n,i}(x) + \sum_{i=0}^n f'(x_i) \hat{H}_{n,i}(x)$$

$H_{n,i}(x)$ 为了处理函数值:

$$H_{n,i}(x) = [1 - 2(x-x_i)L'_{n,i}(x)] L_{n,i}^2(x)$$

$$H_{n,i}(x_i) = 1, H'_{n,i}(x_i) = 0$$

$\hat{H}_{n,i}(x)$ 为了处理导数:

$$\hat{H}_{n,i}(x) = (x-x_i)L_{n,i}^2(x)$$

$$\hat{H}_{n,i}(x_i) = 0, \hat{H}'_{n,i}(x_i) = 1$$

误差:

$$f(x) = H_{2n+1}(x) + \frac{(x-x_0)^2 \cdots (x-x_n)^2}{(2n+2)!} f^{(2n+2)}(\xi)$$

3.3 三次样条插值

龙格现象是数值分析中的一个经典问题，指的是在等距节点下，使用高次多项式插值时，插值多项式在区间两端会出现剧烈震荡，导致插值误差反而比低次多项式更大，甚至完全不收敛于原函数。想要减小龙格现象，可以使用低次分段插值方法，比较经典的方法是三次样条插值。

函数有 $n+1$ 个数据点，分为 n 个区间，考虑第 i 个区间，用三次多项式函数拟合：

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3, i \in [0, n-1]$$

一共有 $4n$ 个系数。

(i) 插值条件

$$S_i(x_i) = f_i \Rightarrow a_i = f_i$$

$$S_i(x_{i+1}) = f_{i+1} \Rightarrow a_i + b_i h_i + c_i h_i^2 + d_i h_i^3 = f_{i+1}$$

给出了 $2n$ 个方程。

(ii) 一阶导数连续

$$S'_i(x_{i+1}) = S'_{i+1}(x_{i+1})$$

$$\Rightarrow b_i + 2c_i h_i + 3d_i h_i^2 = b_{i+1}$$

给出 $n-1$ 个方程。

(iii) 二阶导数连续

$$S''_i(x_{i+1}) = S''_{i+1}(x_{i+1})$$

$$\Rightarrow c_i + 3d_i h_i = c_{i+1}$$

给出 $n-1$ 个方程。

(iv) 边界条件

边界条件要根据实际情况来确定，比较常用的是自然样条边界条件：两端二阶导数为 0

$$S''_0(x_0) = 0 \Rightarrow c_0 = 0$$

$$S''_{n-1}(x_n) = 0 \Rightarrow c_{n-1} + 3d_{n-1}h_{n-1} = 0$$

或者是固定边界条件：

$$S'_0(x_0) = f'_0 \Rightarrow b_0 = f'_0$$

$$S'_{n-1}(x_n) = f'_n \Rightarrow b_{n-1} + 2c_{n-1}h_{n-1} + 3d_{n-1}h_{n-1}^2 = f'_n$$

这给出了 2 个方程。

现在来求解系数，已知方程：

$$a_{i+1} = a_i + b_i h_i + c_i h_i^2 + d_i h_i^3$$

$$b_{i+1} = b_i + 2c_i h_i + 3d_i h_i^2$$

$$c_{i+1} = c_i + 3d_i h_i$$

先消去 d_i ：

$$a_{i+1} = a_i + b_i h_i + \frac{h_i^2}{3} (2c_i + c_{i+1})$$

$$b_{i+1} = b_i + h_i (c_i + c_{i+1})$$

再消去 b_i ：

$$b_i = \frac{1}{h_i} (a_{i+1} - a_i) - \frac{h_i}{3} (2c_i + c_{i+1})$$

得到 a_i 和 c_i 的关系：

$$h_{i-1} c_{i-1} + 2(h_{i-1} + h_i) c_i + h_i c_{i+1} = \frac{3}{h_i} (a_{i+1} - a_i) - \frac{3}{h_{i-1}} (a_i - a_{i-1})$$

已知 $a_i = f_i$ ，可以求解 c_i 。

自然样条：

$$S''(x_0) = S''(x_n) = 0$$

于是可以得到线性方程组：

$$\mathbf{A} \vec{x} = \vec{b}$$

$$\mathbf{A} = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ h_0 & 2(h_0 + h_1) & h_1 & \cdots & 0 \\ 0 & h_1 & 2(h_1 + h_2) & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & h_{n-1} \\ 0 & 0 & 0 & \cdots & 1 \end{pmatrix}$$

$$\vec{b} = \begin{pmatrix} 0 \\ \frac{3}{h_1} (a_2 - a_1) - \frac{3}{h_0} (a_1 - a_0) \\ \vdots \\ \frac{3}{h_{n-1}} (a_n - a_{n-1}) - \frac{3}{h_{n-2}} (a_{n-1} - a_{n-2}) \\ 0 \end{pmatrix}$$

固定边界条件：

$$S'(x_0) = f'(a), S'(x_n) = f'(b)$$

$$\begin{aligned}
b_0 &= \frac{1}{h_0} (a_1 - a_0) - \frac{h_0}{3} (2c_0 + c_1) = f'(a) \\
\Rightarrow 2h_0c_0 + h_0c_1 &= \frac{3}{h_0} (a_1 - a_0) - 3f'(a) \\
b_n &= b_{n-1} + h_{n-1} (c_{n-1} - c_n) = f'(b) \\
\Rightarrow b_{n-1} &= \frac{1}{h_{n-1}} (a_n - a_{n-1}) - \frac{h_{n-1}}{3} (2c_{n-1} + c_n)
\end{aligned}$$

于是可以得到线性方程组

$$\mathbf{A} = \begin{pmatrix} 2h_0 & h_0 & 0 & \cdots & 0 \\ h_0 & 2(h_0 + h_1) & h_1 & \cdots & 0 \\ 0 & h_1 & 2(h_1 + h_2) & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & h_{n-1} \\ 0 & 0 & 0 & \cdots & 2h_{n-1} \end{pmatrix}$$

$$\vec{b} = \begin{pmatrix} \frac{3}{h_0} (a_1 - a_0) - 3f'(a) \\ \frac{3}{h_1} (a_2 - a_1) - \frac{3}{h_0} (a_1 - a_0) \\ \vdots \\ \frac{3}{h_{n-1}} (a_n - a_{n-1}) - \frac{3}{h_{n-2}} (a_{n-1} - a_{n-2}) \\ 3f'(b) - \frac{3}{h_{n-1}} (a_n - a_{n-1}) \end{pmatrix}$$

3.4 Akima 插值

Akima 插值仍然是分段三次样条插值法，只不过确定系数的方式有所不同，一个点的斜率应该由它的附近点确定。

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$$

需要四个方程来确定四个系数：

$$S_i(x_i) = f_i$$

$$S_i(x_{i+1}) = f_{i+1}$$

定义

$$d_i = \frac{f_{i+1} - f_i}{x_{i+1} - x_i}$$

$$w_{i-1} = |d_{i+1} - d_i|$$

$$w_i = |d_{i-1} - d_{i-2}|$$

w_{i-1} 是右侧差分, w_i 是左侧差分。一阶微分

$$S'_i = \frac{w_{i-1}d_{i-1} + w_id_i}{w_{i-1} + w_i}$$

一些其他的代替方案:

(i) $d_{i-2} = d_{i-1}, d_i \neq d_{i+1}$ 时, 取左边的稳定坡度值

$$S'_i = d_{i-1}$$

(ii) $d_i = d_{i+1}, d_{i-2} \neq d_{i-1}$ 时, 取右侧的稳定坡度值

$$S'_i = d_i$$

(iii) $d_{i-1} = d_i$ 时, 取共同的坡度值

$$S'_i = d_{i-1} = d_i$$

(iv) $d_{i-2} = d_{i-1} \neq d_i = d_{i+1}$ 时, 取平均值

$$S'_i = \frac{d_{i-1} + d_i}{2}$$

因为每个点的斜率需要它旁边五个点的坡度值 d , 而边界上的点旁边没有点了, 所以需要外推:

$$d_{-1} = 2d_0 - d_1$$

$$d_0 = 2d_1 - d_2$$

$$d_k = 2d_{k-1} - d_{k-2}$$

$$d_{k+1} = 2d_k - d_{k-1}$$

例: 拟合函数

$$y = a + b(x - x_1) + c(x - x_1)^2 + d(x - x_1)^3, x \in [x_1, x_2]$$

边界条件:

$$y(x_1) = y_1, y(x_2) = y_2$$

$$y'(x_1) = t_1, y'(x_2) = t_2$$

求解出系数:

$$a = y_1$$

$$b = t_1$$

$$c = \frac{1}{x_2 - x_1} \left[\frac{3(y_2 - y_1)}{x_2 - x_1} - 2t_1 - t_2 \right]$$

$$d = \frac{1}{(x_2 - x_1)^2} \left[t_1 + t_2 - \frac{2(y_2 - y_1)}{x_2 - x_1} \right]$$

3.5 有理函数插值

分数多项式插值可以出现无穷大, 对于导数不连续、有奇异点的函数, 它显著优于传统的多项式插值, 能有效避免龙格现象。分数多项式插值有多种方法, 一种方法是经过插值点, 即有理函数插值。已知函数的 $m+1$ 个点的数值 $(x_i, y_i), i = 1, 2, \dots, m+1$ 。插值函数:

$$R_{i(i+1)\dots(i+m)} = \frac{P_\mu(x)}{Q_\nu(x)} = \frac{p_0 + p_1x + \dots + p_\mu x^\mu}{1 + q_1x + \dots + q_\nu x^\nu}$$

其中 $m = \mu + \nu$, 其数值应根据实际情况选取, $P_\nu(x), Q_\nu(x)$ 为两个有理多项式。

确定系数:

$$\begin{aligned} R(x_i) &= \frac{P_\mu(x_i)}{Q_\nu(x_i)} = y_i \\ \Rightarrow P_\mu(x_i) - y_i Q_\nu(x_i) &= 0 \\ \sum_{j=0}^{\mu} p_j x_i^j - y_i \left(\sum_{k=1}^{\nu} q_k x_i^k + 1 \right) &= 0 \end{aligned}$$

这是关于未知系数的线性方程组:

$$\begin{aligned} p_0 + x_1 p_1 + \dots + x_1^\mu p_\mu - y_1 x_1 q_1 - \dots - y_1 x_1^\nu q_\nu &= y_1 \\ p_0 + x_2 p_1 + \dots + x_2^\mu p_\mu - y_2 x_2 q_1 - \dots - y_2 x_2^\nu q_\nu &= y_2 \\ \dots & \\ p_0 + x_m p_1 + \dots + x_m^\mu p_\mu - y_m x_m q_1 - \dots - y_m x_m^\nu q_\nu &= y_m \end{aligned}$$

3.6 Pade 近似

Pade 近似是另一种分式多项式插值方法, 已知函数 $f(x)$ 的泰勒展开级数, 我们要构造有理函数, 使其泰勒展开在 $x = 0$ 处与 $f(x)$ 匹配到尽可能高的阶数。拟合函数:

$$R(x) = \frac{a_0 + a_1 x^1 + \dots + a_m x^m}{1 + b_1 x^1 + \dots + b_n x^n}$$

$f(x)$ 的泰勒级数已知:

$$f(x) = \sum_{k=0}^{\infty} c_k x^k$$

要求两个展开的前 $m+n+1$ 项完全相同, 即

$$\begin{aligned} \frac{a_0 x^0 + a_1 x^1 + \dots + a_m x^m}{1 + b_1 x^1 + \dots + b_n x^n} &= c_0 + c_1 x^1 + \dots + c_{m+n+1} x^{m+n+1} + \dots \\ (1 + b_1 x^1 + \dots + b_n x^n) (c_0 + c_1 x^1 + c_2 x^2 + \dots) &= a_0 + a_1 x^1 + \dots + a_m x^m + O(x^{m+n+1}) \end{aligned}$$

零阶项:

$$a_0 = c_0$$

一阶项:

$$a_1 = c_1 + b_1 c_0$$

二阶项:

$$a_2 = c_2 + b_1 c_1 + b_2 c_0$$

...

$$a_m = c_m + \sum_{i=1}^{\min(m,n)} b_i c_{m-i}$$

后面的系数都为零, 因为右边没有 $m+1$ 阶以上的项。

$$\begin{pmatrix} c_{m-n+1} & c_{m-n+2} & \cdots & c_m \\ c_{m-n+2} & c_{m-n+3} & \cdots & c_{m+1} \\ \vdots & \vdots & \ddots & \vdots \\ c_m & c_{m+1} & \cdots & c_{m+n-1} \end{pmatrix} \begin{pmatrix} b_n \\ b_{n-1} \\ \vdots \\ b_1 \end{pmatrix} = - \begin{pmatrix} c_{m+1} \\ c_{m+2} \\ \vdots \\ c_{m+n} \end{pmatrix}$$

4 数值微分和积分

4.1 一阶微分公式

最简单的一阶微分公式是向前差分公式:

$$f'(x_0) = \frac{f(x_0 + h) - f(x_0)}{h} - \frac{h}{2} f''(\xi)$$

截断误差精度为一阶, 误差相对较大, 下面考虑其他微分公式来提高精度, 考虑拉格朗日插值:

$$f(x) = \sum_{k=0}^n f(x_k) L_k(x) + \frac{(x-x_0) \cdots (x-x_n)}{(n+1)!} f^{(n+1)}(\xi)$$

求导:

$$f'(x) = \sum_{k=0}^n f(x_k) L'_k(x) + \frac{d}{dx} \left[\frac{(x-x_0) \cdots (x-x_n)}{(n+1)!} \right] f^{(n+1)}(\xi) + \frac{(x-x_0) \cdots (x-x_n)}{(n+1)!} \frac{d}{dx} f^{(n+1)}(\xi)$$

其中插值基函数:

$$L_{n,k}(x) = \frac{(x-x_0) \cdots (x-x_{k-1})(x-x_{k+1}) \cdots (x-x_n)}{(x_k-x_0) \cdots (x_k-x_{k-1})(x_k-x_{k+1}) \cdots (x_k-x_n)}$$

取三个点 x_0, x_1, x_2 , 得到三点微分公式:

$$f'(x_i) = f(x_0) \frac{2x_i - x_1 - x_2}{(x_0 - x_1)(x_0 - x_2)} + f(x_1) \frac{2x_i - x_0 - x_2}{(x_1 - x_0)(x_2 - x_0)} + f(x_2) \frac{2x_i - x_0 - x_1}{(x_2 - x_0)(x_2 - x_1)}$$

截断误差:

$$R(x_i) = \frac{1}{6} f^{(3)}(\xi_i) \prod_{k=0, k \neq i}^2 (x_i - x_k)$$

若等间距取点, 即

$$x_1 = x_0 + h, x_2 = x_0 + 2h$$

得到三个点的微分公式:

$$f'(x_0) = \frac{1}{h} \left[-\frac{3}{2} f(x_0) + 2f(x_0 + h) - \frac{1}{2} f(x_0 + 2h) \right] + \frac{h^2}{3} f^{(3)}(\xi_0)$$

$$f'(x_0 + h) = \frac{1}{h} \left[-\frac{1}{2} f(x_0) + \frac{1}{2} f(x_0 + 2h) \right] - \frac{h^2}{6} f^{(3)}(\xi_1)$$

$$f'(x_0 + 2h) = \frac{1}{h} \left[\frac{1}{2} f(x_0) - 2f(x_0 + h) + \frac{3}{2} f(x_0 + 2h) \right] + \frac{h^2}{3} f^{(3)}(\xi_2)$$

平移, 得到端点三点微分公式:

$$f'(x_0) = \frac{1}{2h} [-3f(x_0) + 4f(x_0 + h) - f(x_0 + 2h)] + \frac{h^2}{3} f^{(3)}(\xi_0)$$

中心差分公式:

$$f'(x_0) = \frac{1}{2h} [-f(x_0 - h) + f(x_0 + h)] - \frac{h^2}{6} f^{(3)}(\xi_1)$$

端点三点公式:

$$f'(x_0) = \frac{1}{2h} [f(x_0 - 2h) - 4f(x_0 - h) + 3f(x_0)] + \frac{h^2}{3} f^{(3)}(\xi_2)$$

如果取的点更多, 得到相应的微分公式精度就会更高。例, 五点微分公式:

$$f'(x_0) = \frac{1}{12h} [f(x_0 - 2h) - 8f(x_0 - h) + 8f(x_0 + h) - f(x_0 + 2h)] + \frac{h^4}{30} f^{(5)}(\xi)$$

$$f'(x_0) = \frac{1}{12h} [-25f(x_0) + 48f(x_0 + h) - 36f(x_0 + 2h) + 16f(x_0 + 3h) - 3f(x_0 + 4h)] + \frac{h^4}{5} f^{(5)}(\xi)$$

4.2 二阶微分公式

$$\begin{aligned}f(x_0 + h) &= f(x_0) + f'(x_0)h + \frac{1}{2}f''(x_0)h^2 + \frac{1}{6}f'''(x_0)h^3 + \frac{1}{24}f^{(4)}(\xi_1)h^4 \\f(x_0 - h) &= f(x_0) - f'(x_0)h + \frac{1}{2}f''(x_0)h^2 - \frac{1}{6}f'''(x_0)h^3 + \frac{1}{24}f^{(4)}(\xi_{-1})h^4 \\ \Rightarrow f(x_0 + h) + f(x_0 - h) &= 2f(x_0) + f''(x_0)h^2 + \frac{1}{24}[f^{(4)}(\xi) + f^{(4)}(\xi_{-1})]h^4 \\ f''(x_0) &= \frac{1}{h^2}[f(x_0 - h) - 2f(x_0) + f(x_0 + h)] - \frac{h^2}{24}[f^{(4)}(\xi) + f^{(4)}(\xi_{-1})]\end{aligned}$$

4.3 Simpson 公式

下面来考虑积分，由拉格朗日插值公式：

$$\begin{aligned}P_n(x) &= \sum_{i=0}^n f(x_i) L_i(x) \\ \int_a^b f(x) dx &= \int_a^b \sum_{i=0}^n f(x_i) L_i(x) dx + \int_a^b \prod_{i=0}^n (x - x_i) \frac{f^{(n+1)}(\xi)}{(n+1)!} dx \\ &= \sum_{i=0}^n a_i f(x_i) + \frac{1}{(n+1)!} \int_a^b \prod_{i=0}^n (x - x_i) f^{(n+1)}(\xi) dx \\ &\approx a_i f(x_i)\end{aligned}$$

其中

$$a_i = \int_a^b L_i(x) dx$$

选三个等距点：

$$x_0 = a, x_1 = \frac{a+b}{2}, x_2 = b$$

插值多项式为

$$P_2(x) = L_0(x) f(x_0) + L_1(x) f(x_1) + L_2(x) f(x_2)$$

因此积分可写作：

$$\int_a^b f(x) dx = \sum_{k=0}^2 a_k f(x_k) = a_0 f(x_0) + a_1 f(x_1) + a_2 f(x_2)$$

$$L_k(x) = \prod_{i=0, i \neq k}^2 \left(\frac{x - x_i}{x - x_k} \right)$$

$$a_0 = \int_a^b L_0(x) dx = \frac{1}{6}(b-a)$$

$$a_1 = \int_a^b L_1(x) dx = \frac{2}{3}(b-a)$$

$$a_2 = \int_a^b L_2(x) dx = \frac{1}{6}(b-a)$$

得到 Simpson 公式:

$$\int_a^b f(x) dx \approx \frac{b-a}{6} \left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right]$$

4.4 牛顿-科特斯公式

取 n 个点, 积分公式为

$$\int_a^b f(x) dx \approx \sum_{k=0}^n a_k f(x_k)$$

$$a_k = \int_a^b L_k(x) dx = \int_a^b \prod_{i=0, i \neq k}^n \left(\frac{x - x_i}{x_k - x_i} \right) dx$$

定义科特斯系数:

$$c_k = \frac{1}{b-a} a_k$$

步长:

$$h = \frac{b-a}{n}$$

$$\Rightarrow \int_a^b f(x) dx \approx (b-a) \sum_{k=0}^n c_k f(x_k) = (b-a) \sum_{k=0}^n c_k f(a+kh)$$

$$\begin{aligned} c_k &= \frac{1}{b-a} \int_a^b \prod_{i=0, i \neq k}^n \left(\frac{x - x_i}{x_k - x_i} \right) dx = \frac{1}{nh} \int_0^h \prod_{i=0, i \neq k}^n \frac{\lambda - i}{k - i} h d\lambda \\ &= \frac{(-1)^{n-k}}{n(n-k)!k!} \int_0^n \prod_{i=0, i \neq k}^n (\lambda - i) d\lambda \end{aligned}$$

科特斯系数的性质:

(i) 对于确定的点数 n , 有

$$\sum_{k=0}^n c_k = 1$$

(ii) 对称性

$$c_k = c_{n-k}$$

(iii) $n = 8$ 时, 系数会出现负数, 因此高阶插值不稳定。

比较常用的是 $n = 4$ (科特斯公式)

$$\int_a^b f(x) dx \approx (b-a) \left[\frac{7}{90} f(x_0) + \frac{16}{45} f(x_1) + \frac{2}{15} f(x_2) + \frac{16}{45} f(x_3) + \frac{7}{90} f(x_4) \right]$$

$$R(x) = -\frac{8h^7}{945} f^{(6)}(\xi)$$

4.5 复合积分公式

复合积分公式为了减小区间过大而造成误差太大, 将积分区间等分为若干等份, 每一份为一个子区间, 分别利用低阶积分公式, 再把每个区间积分加起来。考虑复合 Simpson 公式:

$$\int_a^b f(x) dx = \sum_{k=1}^{n/2} \int_{x_{2k-2}}^{x_{2k}} f(x) dx = \sum_{k=1}^{n/2} \left\{ \frac{h}{3} \left[f(x_{2k-2}) + 4f(x_{2k-1}) + f(x_{2k}) - \frac{h^5}{90} f^{(4)}(\xi) \right] \right\}$$

合并重复项:

$$\int_a^b f(x) dx = \frac{h}{3} \left[f(a) + 2 \sum_{k=1}^{n/2-1} f(x_{2k}) + 4 \sum_{k=1}^{n/2} f(x_{2k-1}) + f(b) \right] - \frac{b-a}{180} h^4 f^{(4)}(\xi')$$

4.6 余项外推

假设精确值为 M , 用步长 h 计算近似值为 $N_1(h)$, 余项可以写成如下形式:

$$M = N_1(h) + k_1 h + k_2 h^2 + k_3 h^3 + \dots$$

换成一半步长:

$$\begin{aligned} M &= N_1\left(\frac{h}{2}\right) + k_1 \frac{h}{2} + k_2 \frac{h^2}{4} + k_3 \frac{h^3}{8} + \dots \\ \Rightarrow M &= 2N_1\left(\frac{h}{2}\right) - N_1(h) - \frac{k_2}{2} h^2 - \frac{3k_3}{4} h^3 + \dots \end{aligned}$$

定义:

$$\begin{aligned} N_2(h) &= 2N_1\left(\frac{h}{2}\right) - N_1(h) \\ \Rightarrow M &= N_2(h) - \frac{k_2}{2} h^2 - \frac{3k_3}{4} h^3 + \dots \end{aligned}$$

精度从 $O(h)$ 提高到了 $O(h^2)$ 。

下面考虑只有偶数阶余项的情形:

$$M = N_1(h) + k_1 h^2 + k_2 h^4 + \dots$$

$$M = N_1 \left(\frac{h}{2} \right) + k_1 \frac{h^2}{4} + k_2 \frac{h^4}{16} + \dots$$

$$\Rightarrow 3M = 4N_1 \left(\frac{h}{2} \right) - N_1(h) - \frac{3k_2}{4}h^4 - \frac{15k_3}{16}h^6 + \dots$$

定义:

$$N_2(h) = \frac{1}{3} \left[4N_1 \left(\frac{h}{2} \right) - N_1(h) \right]$$

$$\Rightarrow M = N_2(h) - \frac{k_2}{4}h^4 - \frac{5k_3}{16}h^6 + \dots$$

下一阶:

$$N_3(h) = \frac{1}{15} \left[16N_2 \left(\frac{h}{2} \right) - N_2(h) \right]$$

$$M = N_3(h) + \frac{k_3}{64}h^6 + \dots$$

考虑复合梯形公式:

$$\int_a^b f(x) dx = \frac{h}{2} \left[f(a) + \sum_{k=1}^{n-1} f(x_k) + f(b) \right] + k_1 h^2 + k_2 h^4 + k_3 h^6 + \dots$$

$$N_1(h) = \frac{h}{2} \left[f(a) + 2 \sum_{k=1}^{n-1} f(x_k) + f(b) \right]$$

递推关系:

$$N_2(h) = \frac{1}{3} \left[4N_1 \left(\frac{h}{2} \right) - N_1(h) \right]$$

$$N_3(h) = \frac{1}{15} \left[16N_2 \left(\frac{h}{2} \right) - N_2(h) \right]$$

$$\dots$$

$$N_i(h) = N_{i-1} \left(\frac{h}{2} \right) + \frac{1}{4^{i-1} - 1} \left[N_{i-1} \left(\frac{h}{2} \right) - N_{i-1}(h) \right]$$

4.7 龙贝格算法

龙贝格算法是一种加速算法, 可以得到更高精度的求积公式。考虑变步长梯形法, 将积分区间 $[a, b]$ 等分为 n 个区间, 有 $n+1$ 个点。

$$x_k = a + kh, h = \frac{b-a}{n}, k = 0, 1, 2, \dots, n$$

积分:

$$T_n = \sum_{k=0}^{n-1} \frac{h}{2} [f(x_k) + f(x_{k+1})]$$

对每一个子区间再加一个节点 $x_{k+1/2}$, 则每一段区间的积分为

$$\frac{h}{4} [f(x_k) + 2f(x_{k+1/2}) + f(x_{k+1})]$$

整个区间:

$$\begin{aligned} T_{2n} &= \frac{h}{4} \sum_{k=0}^{n-1} [f(x_k) + f(x_{k+1})] + \frac{h}{2} \sum_{k=0}^{n-1} f(x_{k+1/2}) \\ &= \frac{T_n}{2} + \frac{h}{2} \sum_{k=0}^{n-1} f(x_{k+1/2}) \end{aligned}$$

等间隔取点:

$$h = \frac{b-a}{n}, x_{k+1} = a + \left(k + \frac{1}{2}\right) h$$

截断误差:

$$\begin{aligned} I - T_n &= -\frac{b-a}{12} h^2 f''(\xi_n) = -\frac{b-a}{12} \left(\frac{b-a}{n}\right)^2 f''(\xi_n) \\ I - T_{2n} &= -\frac{b-a}{12} \left(\frac{b-a}{2n}\right)^2 f''(\xi_{2n}) \end{aligned}$$

如果不考虑 ξ_n 和 ξ_{2n} 的差异:

$$\begin{aligned} I - T_n &= \frac{1}{4} (I - T_{2n}) \\ \Rightarrow I - T_{2n} &= \frac{1}{3} (T_{2n} - T_n) \end{aligned}$$

当 T_n 与 T_{2n} 很接近时, 误差很小。得到

$$I = \frac{4}{3} T_{2n} - \frac{1}{3} T_n$$

考虑抛物线法:

$$\begin{aligned} I - S_n &= -\frac{b-a}{2880} h^4 f^{(4)}(\xi_n) \\ I - S_{2n} &= -\frac{b-a}{2880} \left(\frac{h}{2}\right)^4 f^{(4)}(\xi_{4n}) \\ \Rightarrow I - S_n &= 16 (I - S_{2n}) \\ I &= \frac{16}{15} S_{2n} - \frac{1}{15} S_n \end{aligned}$$

考虑科特斯公式:

$$\begin{aligned} I - C_n &= -\frac{2(b-a)}{945} \left(\frac{h}{4}\right)^6 f^{(6)}(\xi_n) \\ I - 2C_{2n} &= -\frac{2(b-a)}{945} \left(\frac{h}{8}\right)^6 f^{(6)}(\xi_{6n}) \\ \Rightarrow I &= \frac{64}{63} C_{2n} - \frac{1}{63} C_n \end{aligned}$$

4.8 高斯积分

取一组合适的点 $x_k, k = 0, 1, 2, \dots, n$, 使得积分公式

$$\int_a^b f(x) dx = \sum_{k=0}^n a_k f(x_k)$$

具有 $2n - 1$ 次代数精度, 这个积分就叫作高斯积分, 对应的点 x_k 叫作高斯点。

定理: 节点 $x_k, k = 0, 1, 2, \dots, n$ 是高斯点的充分必要条件是, 以这些点为零点的多项式

$$\omega_{n+1}(x) = \prod_{k=0}^n (x - x_k)$$

与任意不超过 n 次的多项式 $Q_n(x)$ 均正交, 即

$$\int_a^b Q_n(x) \omega_{n+1}(x) dx = 0$$

已知 Legendre 多项式在 $[-1, 1]$ 上是正交多项式, Legendre 多项式是 Legendre 方程的解

$$\frac{d}{dx} \left[(1 - x^2) \frac{dy}{dx} \right] + \lambda y = 0$$

$$L_k(x) = \frac{1}{2^k k!} \frac{d^k}{dx^k} (x^2 - 1)^k$$

Legendre 多项式是正交的:

$$\int_{-1}^1 L_i(x) L_j(x) dx = \frac{2}{2i + 1} \delta_{ij}$$

迭代关系:

$$(k + 1) L_{k+1}(x) + k L_{k-1}(x) - (2k + 1) x L_k(x) = 0$$

可以取

$$\omega_{n+1}(x) = \frac{2^{n+1} [(n + 1)!]^2}{(2n + 2)!} L_{n+1}(x) = \frac{(n + 1)!}{(2n + 2)!} \frac{d^{n+1}}{dx^{n+1}} \left[(x^2 - 1)^{n+1} \right]$$

对于不超过 n 次的多项式 $Q_n(x)$, 可以进行展开:

$$Q_n(x) = \sum_{k=0}^n \lambda_k L_k(x)$$

$$\Rightarrow \int_{-1}^1 \omega(x) Q(x) dx = C(n) \int_{-1}^1 L_k(x) L_{n+1}(x) dx = 0$$

Legendre 多项式的零点即为高斯点。使用 n 阶 Legendre 多项式 $P_n(x)$ 的零点来构造高斯公式：

$$\int_{-1}^1 f(x) dx = \sum_{k=0}^{n-1} a_k f(x_k)$$

拉格朗日插值系数：

$$a_k = \int_{-1}^1 \prod_{i=0, i \neq k}^{n-1} \left(\frac{x - x_i}{x_k - x_i} \right) dx$$

下面证明高斯积分具有 $2n-1$ 阶代数精度：考虑 $2n-1$ 阶多项式 $P_{2n-1}(x)$ ，可以写成

$$P_{2n-1}(x) = L_n(x) Q_{n-1}(x) + R_{n-1}(x)$$

其中 $L_n(x)$ 为 n 次 Legendre 多项式， $Q_{n-1}(x)$ 与 $R_{n-1}(x)$ 均为次数不超过 $n-1$ 的多项式。

考虑 $[-1, 1]$ 区间上的积分：

$$\int_{-1}^1 P_{2n-1}(x) dx = \int_{-1}^1 L_n(x) Q_{n-1}(x) dx + \int_{-1}^1 R_{n-1}(x) dx = \int_{-1}^1 R_{n-1}(x) dx$$

在高斯点 x_k 处

$$L_n(x_k) = 0 \Rightarrow P_{2n-1}(x_k) = R_{n-1}(x_k)$$

n 个点的高斯积分公式对 $n-1$ 阶多项式是精确成立的，于是有

$$\int_{-1}^1 R_{n-1}(x) dx = \sum_{k=1}^n a_k R_{n-1}(x_k)$$

而已知

$$\begin{aligned} \int_{-1}^1 P_{2n-1}(x) dx &= \int_{-1}^1 R_{n-1}(x) dx \\ \sum_{k=1}^n a_k R_{n-1}(x_k) &= \sum_{k=1}^n a_k P_{2n-1}(x_k) \end{aligned}$$

得到

$$\int_{-1}^1 P_{2n-1}(x) dx = \sum_{k=1}^n a_k P_{2n-1}(x_k)$$

可以看到 n 个高斯点的高斯公式对 $2n-1$ 阶多项式也是精确成立的，因此具有 $2n-1$ 阶精度。

现在问题变为求解高斯积分权重系数 a_k ，一个经典做法是利用 Legendre 多项式的正交性和节点上的函数值来反解系数。 $R_{n-1}(x)$ 可以由 Legendre 多项式展开：

$$R_{n-1}(x) = \sum_{i=0}^{n-1} \lambda_i L_i(x)$$

积分：

$$\int_{-1}^1 R_{n-1}(x) dx = \sum_{i=0}^{n-1} \lambda_i \int_{-1}^1 L_0(x) L_i(x) dx = 2\lambda_0$$

考虑高斯点 x_k 上的取值：

$$R_{n-1}(x_k) = \sum_{i=0}^{n-1} \lambda_i L_i(x_k), k = 0, 1, 2, \dots, n-1$$

得到关于 λ_k 的线性方程组：

$$\begin{pmatrix} L_0(x_0) & L_1(x_0) & \cdots & L_{n-1}(x_0) \\ L_0(x_1) & L_1(x_1) & \cdots & L_{n-1}(x_1) \\ \vdots & \vdots & \ddots & \vdots \\ L_0(x_{n-1}) & L_1(x_{n-1}) & \cdots & L_{n-1}(x_{n-1}) \end{pmatrix} \begin{pmatrix} \lambda_0 \\ \lambda_1 \\ \vdots \\ \lambda_{n-1} \end{pmatrix} = \begin{pmatrix} R_{n-1}(x_0) \\ R_{n-1}(x_1) \\ \vdots \\ R_{n-1}(x_{n-1}) \end{pmatrix}$$

$$\mathbf{L}\vec{\lambda} = R_{n-1}(\vec{x})$$

$$\Rightarrow \vec{\lambda} = \mathbf{L}^{-1}R_{n-1}(\vec{x})$$

写成分量形式

$$\lambda_i = \sum_{k=0}^{n-1} (\mathbf{L}^{-1})_{ik} R_{n-1}(x_k)$$

于是

$$\int_{-1}^1 R_{n-1}(x) dx = 2\lambda_0 = 2 \sum_{k=0}^{n-1} (\mathbf{L}^{-1})_{0k} R_{n-1}(x_k)$$

$$\int_{-1}^1 P_{2n-1}(x) dx = 2 \sum_{k=0}^{n-1} (\mathbf{L}^{-1})_{0k} P_{2n-1}(x_k)$$

$$\Rightarrow a_k = 2 (\mathbf{L}^{-1})_{0k}$$

例：积分

$$\int_{-1}^1 x^2 dx$$

考虑 $n = 2$ ，Legendre 多项式为

$$L_2(x) = \frac{1}{2} (3x^2 - 1)$$

高斯点为

$$x_0 = -\frac{1}{\sqrt{3}}, x_1 = \frac{1}{\sqrt{3}}$$

任意多项式 $R_{n-1}(x)$ 可以由 Legendre 多项式展开:

$$R_{n-1}(x) = \lambda_0 L_0(x) + \lambda_1 L_1(x)$$

$$R_1(x_0) = \lambda_0 - \frac{1}{\sqrt{3}}\lambda_1$$

$$R_1(x_1) = \lambda_0 + \frac{1}{\sqrt{3}}\lambda_1$$

$$\mathbf{L} = \begin{pmatrix} 1 & -\frac{1}{\sqrt{3}} \\ 1 & \frac{1}{\sqrt{3}} \end{pmatrix}$$

$$\mathbf{L}^{-1} = \frac{\sqrt{3}}{2} \begin{pmatrix} \frac{1}{\sqrt{3}} & \frac{1}{\sqrt{3}} \\ -1 & 1 \end{pmatrix}$$

权重系数:

$$a_0 = a_1 = 1$$

积分:

$$\int_{-1}^1 x^2 dx = a_0 x_0^2 + a_1 x_1^2 = \frac{1}{3} + \frac{1}{3} = \frac{2}{3}$$

刚才我们考虑的积分都是在 $[-1, 1]$ 区间内的, 如果要在其它区间积分, 就需要进行区间变换: 假设积分区间为 $[a, b]$, 令

$$t = \frac{b-a}{2}x + \frac{b+a}{2}$$

$$t(x=-1) = a, t(x=1) = b$$

$$\int_a^b f(t) dt = \frac{b-a}{2} \int_{-1}^1 f\left(\frac{b-a}{2}x + \frac{b+a}{2}\right) dx$$

无穷区间积分:

$$\int_0^\infty f(t) dt$$

考虑

$$t = \tan\left[\frac{\pi}{4}(1+x)\right]$$

相应的权重因子也要发生变化:

$$a'_k = \frac{\pi}{4} \frac{a_k}{\cos^2\left[\frac{\pi}{4}(1+x_k)\right]}$$

4.9 带权函数的高斯积分

很多情况下我们需要处理带权函数的高斯积分：

$$\int_a^b \rho(x) f(x) dx$$

此时高斯点成立的条件为：多项式

$$\omega_{n+1}(x) = \prod_{k=0}^n (x - x_k)$$

为区间 $[a, b]$ 上关于权函数 $\rho(x)$ 的正交多项式：

$$\int_a^b \rho(x) \omega_{n+1}(x) Q_n(x) dx = 0$$

下面列举一些常见的带权函数的高斯积分：

(i) 高斯-拉盖尔积分公式

$$\int_a^b e^{-x} f(x) dx \approx \sum_{k=0}^n a_k f(x_k)$$

$$a_k = -\frac{(n!)^2}{\mathcal{L}'_{n+1}(x_k) \mathcal{L}_n(x_k)}$$

$\mathcal{L}_n(x)$ 为拉盖尔多项式：

$$\int_{-\infty}^{\infty} e^{-x} \mathcal{L}_n(x)^2 dx = 1$$

递推关系为

$$(n+1) \mathcal{L}_{n+1}(x) = (2n+1-x) \mathcal{L}_n(x) - n \mathcal{L}_{n-1}(x)$$

$$\mathcal{L}_0(x) = 1$$

$$\mathcal{L}_1(x) = 1 - x$$

(ii) 高斯-厄米积分公式

$$\int_a^b e^{-x^2} f(x) dx \approx \sum_{k=0}^n a_k f(x_k)$$

$$a_k = -\frac{2^{n+1} n! \sqrt{\pi}}{H'_{n+1}(x_k) H_n(x_k)}$$

$H_n(x)$ 厄米多项式：

$$\int_{-\infty}^{\infty} e^{-x^2} H_n(x)^2 dx = 2^n n! \sqrt{\pi}$$

递推关系为

$$H_{n+1}(x) = 2xH_n(x) - 2nH_{n-1}(x)$$

$$H_0(x) = 1$$

$$H_1(x) = 2x$$

(iii) 高斯-切比雪夫公式

$$\int_{-1}^1 \frac{f(x)}{\sqrt{1-x^2}} dx = \sum_{k=0}^n a_k f(x_k)$$

$$a_k = -\frac{\pi}{n+1}$$

5 非线性方程

一些方程的求解比较复杂，需要数值求解。例如方势阱的薛定谔方程：

$$-\frac{\hbar^2}{2m} \frac{d^2 u}{dx^2} + V(x) u(x) = E u(x)$$

$$V(r) = \begin{cases} -V_0, & 0 \leq x < a \\ 0, & x > a \end{cases}$$

势阱内外的薛定谔方程不同：

$$\frac{d^2 u(x)}{dx^2} + \frac{2m}{\hbar^2} (V_0 + E) u(x) = 0, \quad x < a$$

$$\frac{d^2 u(x)}{dx^2} + \frac{2m}{\hbar^2} E u(x) = 0, \quad x > a$$

$$u(r) = \begin{cases} A \sin\left(\sqrt{2m(V_0 - |E|)}r/\hbar\right), & r < a \\ B \exp\left(-\sqrt{2m|E|}r/\hbar\right), & r > a \end{cases}$$

连续性条件是一个方程求根的问题：

$$f(E) = \sqrt{2m(V_0 - |E|)} \cot\left(\sqrt{2ma^2(V_0 - |E|)}/\hbar\right) + \sqrt{2m|E|}$$

$$f(E) = 0$$

5.1 二分法

二分法的思想是很简单的，其基本原理是介值定理：假设函数 $f(x)$ 在区间 $[a, b]$ 上连续，并且 $f(a)$ 与 $f(b)$ 异号，那么在区间 (a, b) 内至少存在一个根 r ，使得 $f(r) = 0$ 。

考虑区间 $[a, b]$ ，取区间中点 $p_1 = \frac{a+b}{2}$ ，检查 $f(p_1)$ 的符号：

(i) 如果 $f(p_1)$ 与 $f(a)$ 同号，说明根在右半区间 (p_1, b) 内，更新 $(a_2, b_2) = (p_1, b)$ ；

(ii) 如果 $f(p_1)$ 与 $f(b)$ 同号，说明根在左半区间 (a, p_1) 内，更新 $(a_2, b_2) = (a, p_1)$ 。

循环： $p_2 = \frac{a_2+b_2}{2}$ ，直到 $f(p_n) = 0$ 为止。

二分法具有全局收敛性，对于振荡、不光滑函数具有较好效果。

5.2 不动点法

不动点法将求根转化为寻找不动点的问题。求根函数：

$$f(x) = x - g(x)$$

$$f(p) = 0 \Rightarrow p = g(p)$$

选取初始值 p_0 ，迭代求解：

$$p_n = g(p_{n-1})$$

最终会收敛到方程的根：

$$p = \lim_{n \rightarrow \infty} p_n = \lim_{n \rightarrow \infty} g(p_n) = g(p)$$

对于同一个 $f(x)$ ，有不同的 $g(x)$ 选取方法， $g(x)$ 的选取和迭代的收敛速度有很大关系。如果 $g(x)$ 选择不好，迭代可能会收敛很慢甚至发散，因此要选取较好的 $g(x)$ 。

压缩映射定理：若 g 将区间 $[a, b]$ 映射到自身，且 $g(x)$ 在 $[a, b]$ 上可导，并存在常数 $0 < k < 1$ 使得 $|g'(x)| \leq k, \forall x \in (a, b)$ ，则对于任意初始值 $p_0 \in [a, b]$ ，由 $p_n = g(p_{n-1})$ 生成的序列必定收敛到 $[a, b]$ 内的唯一不动点 p 。

例：对于方程

$$x^3 + 4x^2 - 10 = 0$$

有以下五种 $g(x)$ 的取值方法：

$$g_1(x) = x - x^3 - 4x^2 + 10$$

$$g_2(x) = \left(\frac{10}{x} - 4x \right)^{1/2}$$

$$g_3(x) = \frac{1}{2} (10 - x^3)^{1/2}$$

$$g_4(x) = \left(\frac{10}{4+x} \right)^{1/2}$$

$$g_5(x) = x - \frac{x^3 + 4x^2 - 10}{3x^2 + 8x}$$

取 $p_0 = 1.5$ ，通过数值计算可以看到，第一个函数会发散，第二个会收敛到负数的平方根，从第三个到第五个函数，收敛速度会不断变快。

表 1: 五个 $g(x)$ 利用不动点法求根结果

n	$g_1(x)$	$g_2(x)$	$g_3(x)$	$g_4(x)$	$g_5(x)$
0	1.5	1.5	1.5	1.5	1.5
1	-0.875	0.8165	1.286953768	1.348399725	1.373333333
2	6.732	2.9969	1.402540804	1.367376372	1.365262015
3	-469.7	$(-8.65)^{1/2}$	1.345458374	1.364957015	1.365230014
4	1.03×10^8		1.375170253	1.365264748	1.365230013
5			1.360094193	1.365225594	
6			1.367846968	1.365230576	
7			1.363887004	1.365229942	
8			1.365916734	1.365230022	
9			1.364878217	1.365230012	
10			1.365410062	1.365230014	
15			1.365223680	1.365230013	
20			1.365230236		
25			1.365230006		
30			1.365230013		

5.3 牛顿法

5.3.1 牛顿法求解单重根问题

牛顿法是另一种求解方程的方法，其核心思想为在当前位置用切线近似原函数，将切线与 x 轴的交点作为下一个近似根。对于方程 $f(p) = 0$ ，函数 $f(p)$ 可以 Taylor 展开：

$$f(p) = f(p_0) + (p - p_0)f'(p_0) + \frac{(p - p_0)^2}{2}f''(\xi(p))$$

不考虑二阶微分项：

$$p = p_0 - \frac{f(p_0)}{f'(p_0)} = p_1$$

迭代:

$$p_n = p_{n-1} - \frac{f(p_{n-1})}{f'(p_{n-1})}$$

牛顿法的收敛速度是很快的（平方收敛），但是是局域收敛的，这要求初值不能偏离根太远。

5.3.2 牛顿法求解多重根问题

牛顿法求解多重根问题时，如果采用前面的公式，收敛速度将变为线性收敛，为了恢复平方收敛速度，可以将问题再转换为求单根问题。对于函数

$$f(x) = (x - p)^m q(x)$$

定义

$$\mu(x) = \frac{f(x)}{f'(x)}$$

将问题转化为求解 $\mu(x)$ 的单根，可以计算出

$$\mu(x) = \frac{(x - p)q(x)}{mq(x) + (x - p)q'(x)}$$

$$g(x) = x - \frac{\mu(x)}{\mu'(x)} = x - \frac{f(x)f'(x)}{f'(x)^2 - f(x)f''(x)}$$

迭代公式:

$$p_{n+1} = p_n - \frac{f(p_n)f'(p_n)}{f'(p_n)^2 - f(p_n)f''(p_n)}$$

5.4 割线法 (Secant Method)

割线法避免了牛顿法的求微分过程，可以看作牛顿法的“离散化版本”，它避免直接计算导数，而是用两点连线的斜率来近似切线斜率。

$$f'(p_{n-1}) = \lim_{x \rightarrow p_{n-1}} \frac{f(x) - f(p_{n-1})}{x - p_{n-1}}$$

因此可以近似:

$$f'(p_{n-1}) \approx \frac{f(p_{n-1}) - f(p_{n-2})}{p_{n-1} - p_{n-2}}$$

迭代:

$$p_n = p_{n-1} - \frac{f(p_{n-1})(p_{n-1} - p_{n-2})}{f(p_{n-1}) - f(p_{n-2})}$$

5.5 Aitken 加速算法

利用 Aitken 加速算法可以加速收敛。考虑序列 $p_0, p_1, p_2, \dots$ ，该序列将收敛为 p 。可以构造一个新的序列，使其收敛速度变快。假设对于足够大的 n ，有

$$\frac{p_{n+1} - p}{p_n - p} \approx \frac{p_{n+2} - p}{p_{n+1} - p}$$

$$\Rightarrow p \approx \frac{p_{n+2}p_n - p_{n+1}^2}{p_{n+2} - 2p_{n+1} + p_n} = p_n - \frac{(p_{n+1} - p_n)^2}{p_{n+2} - 2p_{n+1} + p_n}$$

可以构造新的序列 $\hat{p}_n$ ：

$$\hat{p}_n = p_n - \frac{(p_{n+1} - p_n)^2}{p_{n+2} - 2p_{n+1} + p_n}$$

如果定义

$$\Delta p_n = p_{n+1} - p_n$$

$$\Delta^2 p_n = p_{n+1} - 2p_{n+1} + p_n$$

可以写成差分形式：

$$\hat{p}_n = p_n - \frac{(\Delta p_n)^2}{\Delta^2 p_n}$$

5.6 Steffensen 方法

Steffensen 方法把 Aitken 加速和不动点迭代结合起来，每次迭代都使用加速，从而得到一种不依赖导数、具有二阶收敛速度的求根方法。

从当前近似 $p_0^{(k)}$ 出发，先做两步不动点迭代：

$$p_1^{(k)} = g(p_0^{(k)})$$

$$p_2^{(k)} = g(p_1^{(k)})$$

这三个点利用 Aitken 公式，计算出加速度的值作为下一轮的起点：

$$p_0^{(k+1)} = \hat{p}_0^{(k)} = p_0^{(k)} - \frac{(p_1^{(k)} - p_0^{(k)})^2}{p_2^{(k)} - 2p_1^{(k)} + p_0^{(k)}}$$

再重复上述步骤即可。

5.7 秦九韶算法

考虑代数多项式：

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

可以将其写作如下形式：

$$P(x) = (\cdots ((a_n x + a_{n-1})x + a_{n-2})x + \cdots + a_1)x + a_0$$

令

$$\begin{aligned} b_n &= a_n, \quad b_k = a_k + b_{k+1}x_0 \\ &\Rightarrow b_0 = p(x_0) \end{aligned}$$

考虑多项式：

$$Q(x) = b_n x^{n-1} + b_{n-1} x^{n-2} + \cdots + b_2 x + b_1$$

这个多项式可以看成 $P(x)$ 除以 $(x - x_0)$ 的除数，余数为 b_0 ，即

$$P(x) = (x - x_0)Q(x) + b_0$$

求导：

$$\begin{aligned} P'(x) &= Q(x) + (x - x_0)Q'(x) \\ &\Rightarrow P'(x_0) = Q(x_0) \end{aligned}$$

再利用牛顿法：

$$x_1 = x_0 - \frac{P(x_0)}{P'(x_0)} = x_0 - \frac{P(x_0)}{Q(x_0)}$$

收敛到根 t 时， $P(t) = 0$ ，此时对应的 $b_0 = 0$ 。于是

$$P(x) = (x - t)Q(x)$$

$Q(x)$ 包含了剩下的所有根，可以再对 $Q(x)$ 使用牛顿法求下一个根，以此类推达到逐次降阶的效果。

5.8 Brent 方法

Brent 算法是一种混合型求根方法。核心思想是：同时利用括根区间、二分法和逆二次插值，在保证可靠性的前提下获得更快的收敛速度。

算法维护三个点 a, b, c , 通常 $f(a), f(b)$ 异号, 且 $f(b)$ 的绝对值是最小的。三个点的函数值已知, 可以通过反函数插值来求根:

$$x = \frac{[y - f(a)][y - f(b)]c}{[f(c) - f(a)] - [f(c) - f(b)]} + \frac{[y - f(b)][y - f(c)]a}{[f(a) - f(b)][f(a) - f(c)]} + \frac{[y - f(c)][y - f(a)]b}{[f(b) - f(c)][f(b) - f(a)]}$$

$y = 0$ 时可以求解出对应的 x 值, 解得

$$x = b + \frac{P}{Q}$$

其中

$$\begin{aligned} R &= \frac{f(b)}{f(c)}, S = \frac{f(b)}{f(a)}, T = \frac{f(a)}{f(c)} \\ P &= S[T(R - T)(c - b) - (1 - R)(b - a)] \\ Q &= (T - 1)(R - 1)(S - 1) \end{aligned}$$

下面要考虑这个计算出的结果 x 要不要接受, 一般来说, 接受新值的具体条件:

- (i) $x \in (a, b)$;
- (ii) 与上次迭代相比, 区间收缩不会太慢。

如果上述条件不满足, 则放弃插值, 使用二分法:

$$x = \frac{a + b}{2}$$

随后再迭代, 更新三个点: 计算

$$y = f(x)$$

- (i) 如果 $f(b)$ 与 y 同号, 根在 $[a, x]$ 之间, 更新 $[a', b'] = [a, x]$
- (ii) 否则根在 $[x, b]$ 区间内, 更新 $[a', b'] = [x, b]$ 。

如果 $|f(a')| < |f(b')|$, 则需要更换 a', b' , 因为要保持 $f(b')$ 的绝对值最小。

最后再取取 $c' = b$, 重复上面的过程。

5.9 牛顿法求解非线性方程组

下面来考虑非线性方程组的求解:

$$f_1(x_1, x_2, \dots, x_n) = 0$$

$$f_2(x_1, x_2, \dots, x_n) = 0$$

...

$$f_n(x_1, x_2, \dots, x_n) = 0$$

改写为矩阵形式：

$$\mathbf{F}(\vec{x}) = \vec{0}$$

其中

$$\mathbf{F}(\vec{x}) = \begin{pmatrix} f_1(\vec{x}) \\ f_2(\vec{x}) \\ \dots \\ f_n(\vec{x}) \end{pmatrix}$$

构造 Jacobi 矩阵：

$$\mathbf{J}(\vec{x}) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \dots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \dots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \frac{\partial f_n}{\partial x_2} & \dots & \frac{\partial f_n}{\partial x_n} \end{pmatrix}$$

迭代公式：

$$G(\vec{x}) = \vec{x} - \mathbf{J}(\vec{x})^{-1} \mathbf{F}(\vec{x})$$

$$\vec{x}^{(k)} = G(\vec{x}^{(k-1)}) = \vec{x}^{(k-1)} - \mathbf{J}(\vec{x}^{(k-1)})^{-1} \mathbf{F}(\vec{x}^{(k-1)})$$

在实际的求解过程中，一般不会直接求解 Jacobi 矩阵的逆矩阵。

定义

$$\vec{y} = \vec{x}^{(k)} - \vec{x}^{(k-1)}$$

$$\Rightarrow \mathbf{J}(\vec{x}^{(k-1)}) \vec{y} = -\mathbf{F}(\vec{x}^{(k-1)})$$

求解该线性方程组即可。

例：求解非线性方程组

$$f_1(x_1, x_2, x_3) = 3x_1 - \cos(x_2x_3) - \frac{1}{2}$$

$$f_2(x_1, x_2, x_3) = x_1^2 - 81(x_2 + 0.1)^2 + \sin x_3 + 1.06$$

$$f_3(x_1, x_2, x_3) = e^{-x_1x_2} + 20x_3 + \frac{10\pi - 3}{3}$$

Jacobi 矩阵为：

$$\mathbf{J}(x_1, x_2, x_3) = \begin{pmatrix} 3 & x_3 \sin x_2x_3 & x_2 \sin x_2x_3 \\ 2x_1 & -162(x_2 + 0.1) & \cos x_3 \\ -x_2e^{-x_1x_2} & -x_1e^{-x_1x_2} & 20 \end{pmatrix}$$

取定初始值：

$$\vec{x}^{(0)} = (0.1, 0.1, -0.1)^T$$

计算得

$$\mathbf{F}(\vec{x}^{(0)}) = (-0.199995, -2.269833417, 8.462025346)^T$$

求解线性方程组：

$$\mathbf{J}(\vec{x}^{(0)})\vec{y}^{(0)} = -\mathbf{F}(\vec{x}^{(0)})$$

求解得

$$\vec{y}^{(0)} = \begin{pmatrix} 0.3998696728 \\ -0.08053315147 \\ -0.4215204718 \end{pmatrix}$$

$$\vec{x}^{(1)} = \vec{x}^{(0)} + \vec{y}^{(0)} = \begin{pmatrix} 0.4998696782 \\ 0.01946684853 \\ -0.5215204718 \end{pmatrix}$$

迭代：

$$\mathbf{J}(\vec{x}^{(k-1)})\vec{y}^{(k-1)} = -\mathbf{F}(\vec{x}^{(k-1)})$$

5.10 准牛顿法（Broyden 方法）

如果雅可比矩阵很难计算，则需要用近似矩阵 $\mathbf{A}$ 来代替。近似矩阵 $\mathbf{A}$ 需要条件：

(i) 割线条件

对于标量函数 $f(x)$ ，割线近似为：

$$f'(x_1) \approx \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

对于向量函数，自然推广为

$$\mathbf{A}_1(\vec{x}^{(1)} - \vec{x}^{(0)}) = \mathbf{F}(\vec{x}^{(1)}) - \mathbf{F}(\vec{x}^{(0)})$$

记

$$\vec{s}_1 = \vec{x}^{(1)} - \vec{x}^{(0)}$$

$$\vec{y}_1 = \mathbf{F}(\vec{x}^{(1)}) - \mathbf{F}(\vec{x}^{(0)})$$

割线条件可以写作

$$\mathbf{A}_1 \vec{s}_1 = \vec{y}_1$$

(ii) 正交条件

对于任意与 $\vec{s}_0$ 正交的向量 $\vec{z}$ 。

$$\vec{s}_1^T \vec{z} = 0$$

$$\Rightarrow \mathbf{A}_1 \vec{z} = \mathbf{A}_0 \vec{z}$$

$\mathbf{A}_1$ 需要满足秩-1 更新：

$$\mathbf{A}_1 = \mathbf{A}_0 + \frac{\vec{y}_1 - \mathbf{A}_0 \vec{s}_1}{|\vec{s}_1|^2} \vec{s}_1^T$$

验证正交条件：

$$\mathbf{A}_1 \vec{z} = \mathbf{A}_0 \vec{z} + \frac{\vec{y}_1 - \mathbf{A}_0 \vec{s}_1}{|\vec{s}_1|^2} (\vec{s}_1^T \vec{z}) = \mathbf{A}_0 \vec{z}$$

于是 $\mathbf{A}_i$ 的迭代公式：

$$\mathbf{A}_i = \mathbf{A}_{i-1} + \frac{\vec{y}_i - \mathbf{A}_{i-1} \vec{s}_i}{|\vec{s}_i|^2} \vec{s}_i^T$$

$\vec{x}_i$ 的迭代公式：

$$\vec{x}^{(i+1)} = \vec{x}^{(i)} - \mathbf{A}_i^{-1} \mathbf{F}(\vec{x}^{(i)})$$

$\vec{x}_i$ 的迭代中需要求解 $\mathbf{A}_i$ 的逆矩阵，根据 Sherman-Morrison 公式 (2.14)：

$$\begin{aligned} (\mathbf{A} + \vec{u} \vec{v}^T)^{-1} &= \mathbf{A}^{-1} - \frac{\mathbf{A}^{-1} \vec{u} \vec{v}^T \mathbf{A}^{-1}}{1 + \vec{v}^T \mathbf{A}^{-1} \vec{u}} \\ \Rightarrow \mathbf{A}_i^{-1} &= \left(\mathbf{A}_{i-1} + \frac{\vec{y}_i - \mathbf{A}_{i-1} \vec{s}_i}{|\vec{s}_i|^2} \vec{s}_i^T \right)^{-1} \\ &= \mathbf{A}_{i-1}^{-1} + \frac{(\vec{s}_i - \mathbf{A}_{i-1}^{-1} \vec{y}_i) \vec{s}_i^T \mathbf{A}_{i-1}^{-1}}{\vec{s}_i^T \mathbf{A}_{i-1}^{-1} \vec{y}_i} \end{aligned}$$

直接利用该公式就可以加速收敛。

例：求解非线性方程组

$$\vec{g}(x, y, z) = \begin{pmatrix} x^2 + y^2 + z^2 - 3 \\ x^2 + y^2 + z^2 - 1 \\ x + y + z - 3 \end{pmatrix}$$

初始值：

$$\vec{x}^{(0)} = (x^{(0)}, y^{(0)}, z^{(0)})^T = (1, 0, 1)^T$$

雅可比矩阵：

$$\mathbf{J}(x, y, z) = \begin{pmatrix} 2x & 2y & 2z \\ 2x & 2y & -1 \\ 1 & 1 & 1 \end{pmatrix}$$

第一次迭代：

$$\begin{aligned}\mathbf{J}(\vec{x}^{(0)})(\vec{x}^{(1)} - \vec{x}^{(0)}) &= -\vec{g}(\vec{x}^{(0)}) \\ \Rightarrow \vec{x}^{(1)} &= \left(\frac{3}{2}, \frac{1}{2}, 1\right)^T\end{aligned}$$

$$\mathbf{A}_1 = \mathbf{A}_0 + \frac{\vec{y}_1 - \mathbf{A}_0 \vec{s}_1}{|\vec{s}_1|^2} \vec{s}_1^T = \begin{pmatrix} \frac{5}{2} & \frac{1}{2} & 2 \\ \frac{5}{2} & \frac{1}{2} & -1 \\ 1 & 1 & 1 \end{pmatrix}$$

再迭代：

$$\begin{aligned}\mathbf{A}_1(\vec{x}^{(2)} - \vec{x}^{(1)}) &= -\vec{g}(\vec{x}^{(1)}) \\ \Rightarrow \vec{x}^{(2)} &= \left(\frac{5}{4}, \frac{3}{4}, 1\right)^T\end{aligned}$$

5.11 改进的 Broyden 方法

我燃尽了，这部分内容我使用 AI 理解极品 ppt 失败，但是没关系考试不考这部分内容。

6 优化问题

优化问题主要是对目标函数 $f(x)$ 求解全局极小或者极大值的问题。

6.1 Fletcher-Reeves 算法

Fletcher-Reeves 算法也算是共轭梯度法，它主要用于高效求解大规模无约束优化问题。与依赖二阶导数（Hessian 矩阵）的牛顿法不同，FR 算法只需利用一阶梯度信息，就能逐步构造出共轭的搜索方向，从而避免了直接计算和存储庞大的 Hessian 矩阵，在计算复杂度和内存占用上具有显著优势。

对于一般可微函数 $f(x)$ ，求解其极小值。

初始化：

$$p_0 = -\nabla f(x_0)$$

现在要沿着 p_k 方向寻找合适的步长 t_k ，使得 $f(x)$ 下降最多。迭代公式：

$$x_{k+1} = x_k + t_k p_k$$

定义

$$\beta_{k+1}^{FR} = \frac{|\nabla f(x_{k+1})|^2}{|\nabla f(x_k)|^2}$$

新方向:

$$p_{k+1} = -\nabla f(x_{k+1}) + \beta_{k+1}^{FR} p_k$$

t 一般采用一维线搜索来确定。

迭代停止条件:

$$|\nabla f(x_k)| > \varepsilon$$

6.2 线性共轭梯度法

线性共轭梯度法适用于二次正定函数:

$$f(x) = \frac{1}{2} \vec{x} \mathbf{A} \vec{x} - \vec{b}^T \vec{x} + c$$

于是

$$\nabla f = 0 \Rightarrow \mathbf{A} \vec{x} - \vec{b} = 0$$

问题转化为求解线性方程组。

初始化:

$$\vec{r}_0 = \mathbf{A} \vec{x}_0 - \vec{b}$$

$$\vec{p}_0 = -\vec{r}_0$$

根据最速下降法的结论, 步长:

$$t_k = \frac{\langle \vec{r}_k, \vec{p}_k \rangle}{\langle \vec{p}_k, \mathbf{A} \vec{p}_k \rangle}$$

更新变量:

$$\vec{x}_{k+1} = \vec{x}_k + t_k \vec{p}_k$$

更新残差:

$$\vec{r}_{k+1} = \mathbf{A} \vec{x}_{k+1} - \vec{b} = \vec{r}_k + t_k \mathbf{A} \vec{p}_k$$

计算 β :

$$\beta_{k+1} = \frac{\langle \vec{r}_{k+1}, \mathbf{A} \vec{p}_k \rangle}{\langle \vec{p}_k, \mathbf{A} \vec{p}_k \rangle}$$

更新搜索方向:

$$\vec{p}_{k+1} = -\vec{r}_{k+1} + \beta_{k+1} \vec{p}_k$$

再继续迭代。

6.3 拟牛顿法优化

一维情形比较简单, 直接使用牛顿法即可, 求解函数 $f(x)$ 的极值, 可以转化为求解非线性方程 $f'(x) = 0$, 牛顿法公式:

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}$$

多维情形要用到拟牛顿法, 考虑函数的 Tayler 展开:

$$f(\vec{x}_k + \Delta\vec{x}_k) = f(\vec{x}_k) + \nabla f(\vec{x}_k)^T \Delta\vec{x} + \frac{1}{2} \Delta\vec{x}^T \mathbf{H} \Delta\vec{x}$$

其中 H 为 Hessian 矩阵:

$$\mathbf{H}(\vec{x}) = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{pmatrix}$$

一般我们选用近似矩阵 $\mathbf{B}$ 来代替 Hessian 矩阵。

$$\nabla f(\vec{x}_k + \Delta\vec{x}_k) = \nabla f(\vec{x}_k) + \mathbf{B} \Delta\vec{x}_k$$

迭代:

$$\vec{x}_{k+1} = \vec{x}_k + \Delta\vec{x}_k$$

但现在近似矩阵 $\mathbf{B}$ 还是未知的, 需要其他条件去计算得到 $\mathbf{B}$ 的迭代关系, 不同的算法给出了不同的计算过程。

6.4 Broyden 算法

Broyden 算法是比较简单的迭代过程, 本质上是对矩阵 $\mathbf{B}_k$ 进行秩-1 更新:

$$\mathbf{B}_{k+1} = \mathbf{B}_k + \frac{\vec{y}_k - \mathbf{B}_k \vec{s}_k}{\vec{s}_k^T \vec{s}_k} \vec{s}_k^T$$

其中

$$\vec{s}_k = \Delta(\vec{x}_k) = \vec{x}_{k+1} - \vec{x}_k$$

$$\vec{y}_k = \nabla f(\vec{x}_{k+1}) - \nabla f(\vec{x}_k)$$

逆矩阵利用 Sherman-Morrison 公式即可求解:

$$\mathbf{B}_{k+1}^{-1} = \mathbf{B}_k^{-1} + \frac{(\vec{s}_k - \mathbf{B}_k^{-1} \vec{y}_k) \vec{s}_k^T \mathbf{B}_k^{-1}}{\vec{s}_k^T \mathbf{B}_k^{-1} \vec{y}_k}$$

6.5 BFGS 算法

BFGS 算法更复杂一些，这个算法的应用更加广泛，其思想是给 $\mathbf{B}$ 进行秩-2 更新：

$$\mathbf{B}_{k+1} = \mathbf{B}_k + \alpha \vec{u} \vec{u}^T + \beta \vec{v} \vec{v}^T$$

选择

$$\vec{u} = \vec{y}_k, \vec{v} = \mathbf{B}_k \vec{s}_k$$

于是割线条件可以写作

$$\mathbf{B}_{k+1} \vec{s}_k = \vec{y}_k$$

代入 $\mathbf{B}_{k+1}$ 得

$$\mathbf{B}_k \vec{s}_k + \alpha \vec{y}_k (\vec{y}_k^T \vec{s}_k) + \beta (\mathbf{B}_k \vec{s}_k) (\vec{s}_k^T \mathbf{B}_k \vec{s}_k) = \vec{y}_k$$

可以看到

$$\alpha = \frac{1}{\vec{y}_k^T \vec{s}_k}$$

$$\beta = -\frac{1}{\vec{s}_k^T \mathbf{B}_k \vec{s}_k}$$

得到 $\mathbf{B}_k$ 的迭代公式：

$$\mathbf{B}_{k+1} = \mathbf{B}_k + \frac{\vec{y}_k \vec{y}_k^T}{\vec{y}_k^T \vec{s}_k} - \frac{\mathbf{B}_k \vec{s}_k \vec{s}_k^T \mathbf{B}_k^T}{\vec{s}_k^T \mathbf{B}_k \vec{s}_k}$$

逆矩阵：

$$\mathbf{B}_{k+1}^{-1} = \mathbf{B}_k^{-1} + \frac{(\vec{s}_k^T \vec{y}_k + \vec{y}_k^T \mathbf{B}_k^{-1} \vec{y}_k)(\vec{s}_k \vec{s}_k^T)}{(\vec{s}_k^T \vec{y}_k)^2} - \frac{\mathbf{B}_k^{-1} \vec{y}_k \vec{s}_k^T + \vec{s}_k \vec{y}_k^T \mathbf{B}_k^{-1}}{\vec{s}_k^T \vec{y}_k}$$

$$x_{k+1} = x_k - \alpha_k \mathbf{B}_k^{-1} \nabla f(x_k)$$

α_k 是精确搜索步长，对二次函数：

$$f = \frac{1}{2} \vec{x}^T \mathbf{Q} \vec{x} + \vec{b}^T \vec{x}$$

$$\alpha_k = -\frac{\vec{g}_k^T \vec{p}_k}{\vec{p}_k^T \mathbf{Q} \vec{p}_k}$$

其中

$$\vec{p}_k = -\mathbf{B}_k^{-1} \vec{g}_k = -\mathbf{B}^{-1} \nabla f(\vec{x}_k)$$

为下一步的搜索方向。

6.6 线性规划问题—单纯形法

线性规划是运筹学中最基础、最重要的数学优化方法，用于在一组线性约束下，最大化或最小化一个线性目标函数。它解决的问题是：在约束条件下，如何做决策（变量）使得收益最大或成本最小，且所有关系都是直线（线性）的。比如说：某工厂生产两种产品，产品 1 每件利润 40 元，产品 2 每件利润 60 元。生产限制：

机器工时：产品 1 每件需 2 小时，产品 2 需 1 小时，每天总工时不超过 70 小时；

原材料 A：产品 1 每件需 1 单位，产品 2 需 1 单位，每天至少供应 40 单位；

原材料 B：产品 1 每件需 1 单位，产品 2 需 3 单位，每天恰好供应 90 单位。

问如何安排产量使利润最大？

数学建模：要求解函数

$$f(x_1, x_2) = 40x_1 + 60x_2$$

在约束条件

$$2x_1 + x_2 \leq 70$$

$$x_1 + x_2 \geq 40$$

$$x_1 + 3x_2 = 90$$

$$x_1, x_2 \geq 0$$

下的最大值。这就是一个线性规划问题。

单纯形法是求解线性规划问题最经典、最基础的算法。其核心思想比较直观。想象有两个约束条件：

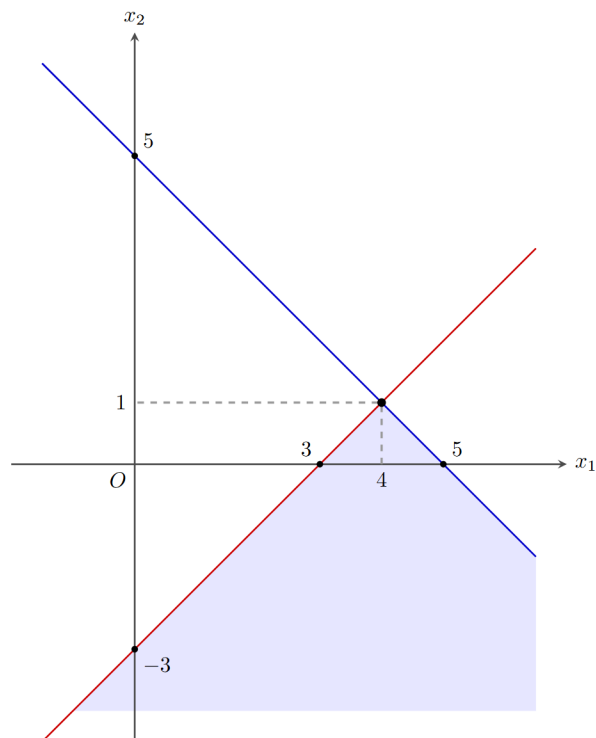
$$x_1 + x_2 \leq 5$$

$$x_1 - x_2 \geq 3$$

x_1, x_2 在这两个约束条件下，可以取到的范围是 $x_1 - x_2$ 平面内的一个二维区域，如图所示。

推广到高维情形，线性规划问题的可行区域应该是一个凸多面体，由若干半空间的交集构成，里面所有的点都是满足约束条件的点。而线性规划的最优解一定出现在某个顶点上。下面来简要说明：

目标函数 $f(x)$ 一定是线性函数，它的等值面是一族平行的超曲面。假设最优点出现在凸多面体内部某点 x' 处，因为内部点周围所有方向都是可行的（因为内部均满足约束条件），而目标函数沿着其在该点的梯度方向 c 时会增加，那么如果从 x' 沿着 c 方向移动一小步，目标函数的值会变大，且当步长足够小时，这个点仍然在内部，这与 x' 为最优位置矛盾。因此，最优解不可能在多面体内部，只能在边界上。



进一步，当如果最优点不在顶点，比如在一条棱上，那么沿着棱方向移动可能继续改进，直到碰到顶点（除非整条棱都是最优的），所以总存在一个顶点是最优的。

单纯形法从一个顶点出发，沿着棱走到相邻的、目标函数值更大的顶点，重复这个过程，直到达到最优顶点。

下面利用一个具体的例子，来演示单纯形法的操作步骤：

求解目标函数

$$f(x_1, x_2) = 4x_1 + 6x_2$$

在约束条件

$$-x_1 + x_2 \leq 11$$

$$x_1 + x_2 \leq 27$$

$$2x_1 + 5x_2 \leq 90$$

$$x_1, x_2 \geq 0$$

下的极大值。

(i) 引入松弛变量，将不等式约束变为等式。

$$-x_1 + x_2 + s_1 = 11$$

$$x_1 + x_2 + s_2 = 27$$

$$2x_1 + 5x_2 + s_3 = 90$$

$$s_1, s_2, s_3 \geq 0$$

(ii) 建立初始单纯形表

表 2: 初始单纯形表

x_1	x_2	s_1	s_2	s_3	b
-1	1	1	0	0	11
1	1	0	1	0	27
2	5	0	0	1	90
-4	-6	0	0	0	0

可以看到，基变量为 s_1, s_2, s_3 ，非基变量为 x_1, x_2 。对于最大值问题，最后一行存放的是目标函数系数的相反数。

(iii) 确定进入变量

在目标行中，找出最负的系数，该系数所在的列称为“进入列”。最负意味着增加改变量对目标函数的提升最显著。在该示例中，应该是 x_2 对应的 -6 。

(iv) 确定离开变量与主元

对进入列的每一个正系数 a_{ij} ，计算右端常数 b_i 与该系数的比值 b_i/a_{ij} 。

$$\frac{11}{1} = 11, \frac{27}{1} = 27, \frac{90}{5} = 18$$

选择最小的非负比值所在的行作为“离开行”。该比值表示在保持可行性的前提下，让当前的“基变量”能让位给进入变量的最大步长。将离开行与进入列的交叉处元素称为“主元”。在该示例中，第一行为“离开行”，第二列为“进入列”，因此主元为交叉元素 1。

(v) 行变换

先将主元所在行除以主元值，使得主元变为 1。作初等行变换，使进入列除主元外所有元素变为 0，这样进入变量成为新的基变量，离开变量被移除基。

此时基变量变为 x_2, s_2, s_3 。

(vi) 判断最优性

检查最后一行，如果所有系数非负，则当前解就是最优解。如果仍有负数，则返回步骤 (iii)，继续迭代。在示例中，最后将迭代为

(vii) 读取最优解

每个基变量对应的取值就是该行右端常数。

$$(x_1, x_2, s_1, s_2, s_3) = (15, 12, 14, 0, 0)$$

表 3: 变换一次的单纯形表

x_1	x_2	s_1	s_2	s_3	b
-1	1	1	0	0	11
2	0	-1	1	0	16
7	0	-5	0	1	35
-10	0	6	0	0	66

表 4: 最终单纯形表

x_1	x_2	s_1	s_2	s_3	b
0	1	0	$-\frac{2}{3}$	$\frac{1}{3}$	12
0	0	1	$\frac{7}{3}$	$-\frac{2}{3}$	14
1	0	0	$\frac{5}{3}$	$-\frac{1}{3}$	15
0	0	0	$\frac{8}{3}$	$\frac{2}{3}$	132

目标函数的最大值位于表格右下角。

$$f(x_1, x_2) = 4x_1 + 6x_2 = 132$$

6.7 下山单纯形法（Nelder-Mead 方法）

下山单纯形法的核心思路十分直观，可以理解为在 N 维空间中构造了一个可以伸缩变形的探针来寻找最低点。算法计算单纯形每个顶点的函数值，并通过反射、扩张、收缩和缩小四种操作，让单纯形在解空间中不断移动变形，逐渐向函数值最小的区域移动。例如，求解函数

$$f(x_1, x_2, \dots, x_n, x_{n+1})$$

的最小值。首先对顶点函数值排序：

$$f(x_1) \leq f(x_2) \leq \dots \leq f(x_{n+1})$$

$f(x_{n+1})$ 是最大的，因此是最差点。计算除最差点以外的其余 n 个点的中心：

$$x_0 = \frac{1}{n} \sum_{i=1}^n x_i$$

(i) 反射

$$x_r = x_0 + \alpha(x_0 - x_{n+1})$$

若 $f(x_1) \leq f(x_r) < f(x_n)$, 用 $x_{n+1} \rightarrow x_r$, 重复过程;

若 $f(x_r) < f(x_1)$, 进行扩张操作;

若 $f(x_r) \geq f(x_n)$, 进行收缩操作。

(ii) 扩张

$$x_e = x_0 + \gamma(x_r - x_0)$$

若 $f(x_e) < f(x_r)$, 则用 x_e 替换 x_{n+1} , 说明扩张成功; 否则扩张失败, 保留反射点, 用 x_r 替换 x_{n+1} , 重复过程。

(iii) 收缩

有两种情况:

(a) 如果 $f(x_r) < f(x_{n+1})$, 说明反射点比最差点更好, 进行外部收缩:

$$x_c = x_0 + \rho(x_r - x_0)$$

若 $f(x_c) \leq f(x_r)$, 则用 x_c 替换 x_{n+1} , 重复过程; 否则收缩失败, 进行缩小操作。

(b) 如果 $f(x_r) \geq f(x_{n+1})$, 说明反射点比最差点更差, 进行内部收缩:

$$x_c = x_0 + \rho(x_{n+1} - x_0)$$

若 $f(x_c) < f(x_{n+1})$, 则用 x_c 替换 x_{n+1} , 重复过程; 否则收缩失败, 进行缩小操作。

(iv) 缩小

如果收缩失败, 说明当前单纯形的形状不理想, 需要进行缩小操作:

$$x_i = x_1 + \sigma(x_i - x_1), i = 2, 3, \dots, n+1$$

然后重复上述过程。

参数的选择有一个经典值:

$$\alpha = 1, \gamma = 2, \rho = 0.5, \sigma = 0.5$$

这个默认值在很多实际问题中表现良好, 但在特定问题中可能需要调整以获得更好的性能。

6.8 模拟退火法

模拟退火法是一种基于概率的全局优化算法, 灵感来源于物理学中的退火过程。它通过模拟物质在高温下逐渐冷却的过程来寻找函数的全局最优解。算法的核心思想是允许在搜索过程中接受一些较差的解, 以避免陷入局部最优, 从而增加找到全局最优解的机会。

Metropolis 过程的基本步骤:

- (i) 初始化：设定初始温度 T_0 和初始点 x_0 ，计算该点的函数值 $f(x_0)$ ；
- (ii) 随机扰动：产生随机扰动 Δx ，得到新点 $x = x_0 + \Delta x$ ，计算新点的函数值 $f(x)$ 以及差值 $\Delta f = f(x) - f(x_0)$ ；
- (iii) 若 $\Delta f \leq 0$ ，则新点更优，无条件接受新点，作为下一次迭代的起点；
- (iv) 若 $\Delta f > 0$ ，则新点更差，计算接受概率

$$p = \exp\left(-\frac{\Delta f}{T}\right)$$

并生成一个在 $[0, 1]$ 区间内的随机数 r ，若 $p(\Delta f) \geq r$ ，则接受新点作为下一次模拟的初始点，否则放弃新点，仍取原来的点作为下一次模拟的初始点。

每轮迭代结束后，按照预定的降温计划降低温度 T ，例如指数降温：

$$T_{k+1} = \alpha T_k, \alpha \in (0, 1)$$

6.9 遗传算法

遗传算法（Genetic Algorithm, GA）是一种模拟生物进化中“自然选择”和“遗传学机理”的随机搜索与优化方法。它由 John Holland 在 20 世纪 70 年代提出，核心思想是：通过种群中个体的选择、交叉（杂交）和变异，使种群整体适应度逐步提高，最终收敛到最优解（或近似最优解）。

优胜劣汰，代代进化：

(i) 编码：将问题的解表示为一个个体（染色体），通常是一个二进制字符串，也可以是实数编码等形式（比如将实数映射到 1024 个二进制字符串）；

(ii) 适应度评价：每个个体对应一个目标函数值，称为“适应度”。适应度越高，说明个体越优秀；

(iii) 选择：依据适应度的高低，以一定概率选出优秀个体作为“父母”；

(iv) 交叉：从选出的父母中随机配对，进行基因交叉，生成新的“子代”个体，子代个体可以继承父母的部分特征；

(v) 变异：子代个体的某些基因以一定概率发生随机变异，增加种群的多样性，防止过早收敛；

(vi) 替代：用新产生的子代替换原有种群中较差的个体，形成新一代种群。

反复执行上述过程，种群的平均适应度会逐渐提高，最终趋近于最优解。

遗传算法通过模拟自然进化中的选择、交叉、变异，让种群一代代适应环境（目标函数），最终获得最优解。它不依赖梯度，能有效处理复杂、非线性、多峰优化问题，是进化计算中最具代表性的算法之一。实际应用中需要调节的参数包括：种群大小、交叉概率、变异概率、选择策略等。

7 本征值问题

在实际物理问题的过程中，经常需要求解矩阵的本征值和本征向量，例如在量子力学中，哈密顿算符的本征值对应系统的能量，而本征向量则对应系统的状态。本征值问题可以写作

$$\begin{aligned}\mathbf{A}\vec{x} &= \lambda\vec{x} \\ a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= \lambda x_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= \lambda x_2 \\ &\vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= \lambda x_n \\ \Rightarrow (\mathbf{A} - \lambda\mathbf{I})\vec{x} &= 0\end{aligned}$$

即要求行列式：

$$\det(\mathbf{A} - \lambda\mathbf{I}) = 0$$

求出方程的根，得到本征值

$$\lambda_1, \lambda_2, \cdots, \lambda_n$$

每个本征值对应一个或多个线性无关的本征向量。

本征值满足关系：

$$\lambda_1 + \lambda_2 + \cdots + \lambda_n = \text{tr}(\mathbf{A})$$

$$\lambda_1 \lambda_2 \cdots \lambda_n = \det(\mathbf{A})$$

实对称矩阵正交相似于对角矩阵，即存在一个正交矩阵 $\mathbf{S}$ 使得

$$\mathbf{S}^T \mathbf{A} \mathbf{S} = \text{diag}(\lambda_1, \lambda_2, \cdots, \lambda_n)$$

其中 $\mathbf{S}$ 为正交矩阵，它是由 $\mathbf{A}$ 的本征向量组成的矩阵。对于实对称矩阵，所有的本征值都是实数，并且对应的本征向量可以选择为相互正交的。 $\lambda_1, \lambda_2, \cdots, \lambda_n$ 即为 $\mathbf{A}$ 的本征值。

7.1 Jacobi 方法

Jacobi 方法是求解实对称矩阵本征值问题的经典算法。它通过一系列的旋转操作来逐步将矩阵对角化。其中每一次旋转可以看作是一个二维平面内的旋转，目的是消除矩阵中的一对非对角元素。

旋转矩阵：

$$\mathbf{S} = \begin{pmatrix} 1 & 0 & \cdots & 0 & \cdots & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 & \cdots & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \cdots & \vdots & \cdots & \vdots \\ 0 & 0 & \cdots & \cos \theta & \cdots & \sin \theta & \cdots & 0 \\ \vdots & \vdots & \cdots & \vdots & \ddots & \vdots & \cdots & \vdots \\ 0 & 0 & \cdots & -\sin \theta & \cdots & \cos \theta & \cdots & 0 \\ \vdots & \vdots & \cdots & \vdots & \cdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & \cdots & 0 & \cdots & 1 \end{pmatrix}$$

可以看到

$$s_{kk} = s_{ll} = \cos \theta$$

$$s_{kl} = -s_{lk} = \sin \theta$$

$$s_{ii} = 1 \quad (i \neq j)$$

$$s_{ij} = 0 \quad (i \neq j)$$

其中 θ 为旋转角度，旋转作用于第 k 行和第 l 行。通过选择合适的 θ ，可以将矩阵 $\mathbf{A}$ 中的第 k 行第 l 列的元素消除为零。下面来计算这个合适的角度 θ 。

考虑矩阵

$$\mathbf{B} = \mathbf{S}^T \mathbf{A} \mathbf{S}$$

矩阵 $\mathbf{B}$ 的第 i 行第 j 列元素为

$$b_{ii} = a_{ii}, \quad i \neq k, l$$

$$b_{ik} = a_{ik} \cos \theta - a_{il} \sin \theta, \quad i \neq k, l$$

$$b_{il} = a_{il} \cos \theta + a_{ik} \sin \theta, \quad i \neq k, l$$

$$b_{kk} = a_{kk} \cos^2 \theta + a_{ll} \sin^2 \theta - 2a_{kl} \sin \theta \cos \theta$$

$$b_{ll} = a_{ll} \cos^2 \theta + a_{kk} \sin^2 \theta + 2a_{kl} \sin \theta \cos \theta$$

$$b_{kl} = (a_{kk} - a_{ll}) \sin \theta \cos \theta + a_{kl}(\cos^2 \theta - \sin^2 \theta)$$

要消除第 k 行第 l 列的元素，即 $b_{kl} = 0$ ，可以得到

$$\tan 2\theta = \frac{2a_{kl}}{a_{ll} - a_{kk}}$$

通过不断选择矩阵 $\mathbf{A}$ 中最大的非对角元素进行旋转，可以逐步将矩阵 $\mathbf{A}$ 对角化，最终得到其本征值。

7.2 Householder 变换

对于实对称矩阵 $\mathbf{A}$ ，存在相似变换：

$$\mathbf{S}_1^T \mathbf{A} \mathbf{S}_1 = \begin{pmatrix} a_{11} & e_1 & 0 & \cdots & 0 \\ e_1 & a'_{22} & a'_{23} & \cdots & a'_{n2} \\ 0 & a'_{32} & a'_{33} & \cdots & a'_{n3} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & a'_{n2} & a'_{n3} & \cdots & a'_{nn} \end{pmatrix}$$

即将矩阵 $\mathbf{A}$ 的第一列（除前两个元素外）消除为零。接下来对剩余的 $(n-1) \times (n-1)$ 子矩阵进行同理相似变换：

$$(\mathbf{S}_1 \mathbf{S}_s)^T \mathbf{A} (\mathbf{S}_1 \mathbf{S}_s) = \begin{pmatrix} a_{11} & e_1 & 0 & \cdots & 0 \\ e_1 & a'_{22} & e_2 & \cdots & 0 \\ 0 & e_2 & a''_{33} & \cdots & a''_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & a''_{n3} & \cdots & a''_{nn} \end{pmatrix}$$

以此类推，最后可以得到一个三对角矩阵：

$$\mathbf{S}^T \mathbf{A} \mathbf{S} = \begin{pmatrix} a_{11} & e_1 & 0 & \cdots & 0 \\ e_1 & a'_{22} & e_2 & \cdots & 0 \\ 0 & e_2 & a''_{33} & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & a_{nn}^{(n-1)} \end{pmatrix}$$

下面来考虑如何构造 Householder 矩阵 $\mathbf{S}$ 来实现上述相似变换。定义 Householder 矩阵：

$$\mathbf{P} = \mathbf{I} - 2\vec{w}\vec{w}^T$$

其中 $\vec{w}$ 是一个单位向量

$$\vec{w}^T \vec{w} = 1$$

可以构造一个分块矩阵：

$$\mathbf{S}_1 = \begin{pmatrix} 1 & \mathbf{0}^T \\ \mathbf{0} & \mathbf{P} \end{pmatrix}$$

对原矩阵 $\mathbf{A}$ 作相似变换：

$$\mathbf{S}_1^T \mathbf{A} \mathbf{S}_1 = \begin{pmatrix} a_{11} & \vec{v}^T \mathbf{P} \\ \mathbf{P} \vec{v} & \mathbf{A}' \end{pmatrix}$$

其中

$$\vec{v} = (a_{21}, a_{31}, \dots, a_{n1})^T$$

要将 $\vec{v}$ 的后 $n-2$ 个元素消除为零, 可以选择 $\vec{w}$ 使得

$$\mathbf{P}\vec{v} = (\alpha, 0, 0, \dots, 0)^T$$

其中 α 为某个实数。可以通过以下步骤来构造 $\vec{w}$:

$$\mathbf{P}\vec{v} = \vec{v} - 2\vec{w}(\vec{w}^T\vec{v}) = \alpha\vec{e}$$

其中 $\vec{e}$ 为单位向量 $(1, 0, 0, \dots, 0)^T$ 。先求出 α 值:

$$(\mathbf{P}\vec{v})^T\mathbf{P}\vec{v} = \alpha^2$$

$$(\mathbf{P}\vec{v})^T\mathbf{P}\vec{v} = \vec{v}^T\vec{v} = |\vec{v}|^2$$

$$\Rightarrow \alpha^2 = \sum_{i=2}^n a_{i1}^2$$

再从反射公式反推 $\vec{w}$:

$$\mathbf{P}\vec{v} = \vec{v} - 2\vec{w}(\vec{w}^T\vec{v}) = \alpha\vec{e}$$

$$\Rightarrow \vec{v} - \alpha\vec{e} = 2\vec{w}(\vec{w}^T\vec{v})$$

$$\Rightarrow \vec{w} = \frac{\vec{v} - \alpha\vec{e}}{2\vec{w}^T\vec{v}}$$

所以问题转化为求解 $\vec{w}^T\vec{v}$, 令 $\beta = \vec{w}^T\vec{v}$ 。

$$\Rightarrow \beta = \vec{w}^T\vec{v} = \frac{(\vec{v} - \alpha\vec{e})^T\vec{v}}{2\beta}$$

$$\Rightarrow 2\beta^2 = (\vec{v} - \alpha\vec{e})^T\vec{v}$$

已知

$$\vec{e}^T\vec{v} = a_{21}$$

$$\vec{v}^T\vec{v} = \alpha^2$$

$$\Rightarrow 2\beta^2 = \alpha^2 - \alpha a_{21}$$

$$\beta = \pm \sqrt{\frac{\alpha^2 - \alpha a_{21}}{2}}$$

一般来说 α 的符号取 a_{21} 的相反数的符号, 以避免数值不稳定。

$$\mathbf{P} = \mathbf{I} - 2\vec{w}\vec{w}^T = \mathbf{I} - 2\frac{(\vec{v} - \alpha\vec{e})(\vec{v} - \alpha\vec{e})^T}{4\beta^2}$$

$$= \mathbf{I} - 2 \frac{(\vec{v} - \alpha \vec{e})(\vec{v} - \alpha \vec{e})^T}{(\vec{v} - \alpha \vec{e})^T(\vec{v} - \alpha \vec{e})}$$

到现在为止，我们就把第一次迭代的 Householder 矩阵 $\mathbf{S}_1$ 构造好了，即

$$\mathbf{A}^{(2)} = \mathbf{S}_1^T \mathbf{A}^{(1)} \mathbf{S}_1$$

下面考虑 Householder 矩阵的迭代，第 k 次变换：

$$\mathbf{A}^{(k+1)} = \mathbf{S}_k^T \mathbf{A}^{(k)} \mathbf{S}_k$$

$$\alpha^2 = \sum_{i=k+1}^n a_{ik}^{(k)2}$$

α 的符号仍然取 $a_{k+1,k}^{(k)}$ 的相反数的符号，以避免数值不稳定。

$$\beta = \pm \sqrt{\frac{\alpha^2 - \alpha a_{k+1,k}^{(k)}}{2}}$$

此时 w 的前 k 个元素为零（为了不改变前面已经消好的列），后面元素的计算与前面相同：

$$w_1^{(k)} = w_2^{(k)} = \cdots = w_k^{(k)} = 0$$

$$w_{k+1}^{(k)} = \frac{a_{k+1,k}^{(k)} - \alpha}{2\beta}$$

$$w_i^{(k)} = \frac{a_{ik}^{(k)}}{2\beta}, \quad i = k+2, k+3, \dots, n$$

于是可以定义 $\mathbf{P}^{(k)}$ ：

$$\mathbf{P}^{(k)} = \mathbf{I} - 2\vec{w}^{(k)}\vec{w}^{(k)T}$$

以及分块矩阵 $\mathbf{S}_k$ ：

$$\mathbf{S}_k = \begin{pmatrix} \mathbf{I}_{k-1} & \mathbf{0}^T \\ \mathbf{0} & \mathbf{P}^{(k)} \end{pmatrix}$$

相似变换得到的矩阵：

$$\begin{aligned} \mathbf{A}^{(k+1)} &= \mathbf{S}_k^T \mathbf{A}^{(k)} \mathbf{S}_k \\ &= \mathbf{S}_k^T \mathbf{S}_{k-1}^T \cdots \mathbf{S}_1^T \mathbf{A} \mathbf{S}_1 \cdots \mathbf{S}_{k-1} \mathbf{S}_k \end{aligned}$$

实对称矩阵最终可以化为三对角矩阵；

非对称矩阵可以化为上 Hessenberg 矩阵，即所有严格下三角部分（主对角线下方超过一条斜线的位置）元素全为零。

7.3 QR 分解

QR 分解在第二章中已经初步介绍过了，这里来详细说明它在求解本征值问题中的应用。对于一个实对称矩阵 $\mathbf{A}$ ，可以进行 QR 分解：

$$\mathbf{A} = \mathbf{Q}\mathbf{R}$$

其中 $\mathbf{Q}$ 是一个正交矩阵， $\mathbf{R}$ 是一个上三角矩阵。通过交换 $\mathbf{Q}$ 和 $\mathbf{R}$ 的位置，可以得到一个新的矩阵。定义

$$\mathbf{A}^{(k+1)} = \mathbf{R}^{(k)}\mathbf{Q}^{(k)}$$

于是

$$\mathbf{A}^{(k+1)} = \mathbf{Q}^{(k)T}\mathbf{A}^{(k)}\mathbf{Q}^{(k)}$$

当 k 足够大时， $\mathbf{A}^{(k)}$ 会收敛到一个上三角矩阵，其对角线元素即为 $\mathbf{A}$ 的本征值。

对于实对称矩阵，通常先将其化为三对角矩阵，然后再进行 QR 迭代，这样可以提高效率。这一步可以利用上面的 Jacobi 旋转或 Householder 变换。

实现 QR 分解的算法有很多种，其中 Givens 旋转是一种常用的方法，也可以再使用 Householder 变换来实现 QR 分解。Givens 旋转和 Jacobi 旋转不同，Jacobi 旋转是针对实对称矩阵的特定元素进行旋转，而 Givens 旋转则是更一般的二维旋转，可以用于任何矩阵的 QR 分解。Givens 旋转可以用来消除矩阵中的特定元素，使得矩阵逐渐变为上三角形。

旋转矩阵的形式与 Jacobi 旋转是一样的：

$$\mathbf{S} = \begin{pmatrix} 1 & 0 & \cdots & 0 & \cdots & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 & \cdots & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \cdots & \vdots & \cdots & \vdots \\ 0 & 0 & \cdots & \cos \theta & \cdots & \sin \theta & \cdots & 0 \\ \vdots & \vdots & \cdots & \vdots & \ddots & \vdots & \cdots & \vdots \\ 0 & 0 & \cdots & -\sin \theta & \cdots & \cos \theta & \cdots & 0 \\ \vdots & \vdots & \cdots & \vdots & \cdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & \cdots & 0 & \cdots & 1 \end{pmatrix}$$

最终使得

$$(\mathbf{S}\mathbf{A})_{ij} = 0$$

于是问题又变为如何求解合适的旋转角度 θ ，这是相对简单的。我们的旋转目标是：

$$\begin{pmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} a_{ij} \\ a_{jj} \end{pmatrix} = \begin{pmatrix} r \\ 0 \end{pmatrix}$$

$$r = \sqrt{a_{ij}^2 + a_{jj}^2}$$

可以得到

$$\begin{aligned}\tan \theta &= \frac{a_{ij}}{a_{jj}} \\ \sin \theta &= \frac{a_{ij}}{\sqrt{a_{ij}^2 + a_{jj}^2}} \\ \cos \theta &= \frac{a_{jj}}{\sqrt{a_{ij}^2 + a_{jj}^2}}\end{aligned}$$

例：考虑矩阵

$$\begin{pmatrix} 3 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 3 \end{pmatrix}$$

构造 Givens 旋转矩阵 $\mathbf{P}_2$ ，使得 $\mathbf{PA}$ 的 (2,1) 元素变为 0。

$$\tan \theta = \frac{a_{12}}{a_{22}} = \frac{1}{3}$$

$$\mathbf{S} = \begin{pmatrix} \frac{3}{\sqrt{10}} & \frac{1}{\sqrt{10}} & 0 \\ -\frac{1}{\sqrt{10}} & \frac{3}{\sqrt{10}} & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$\begin{aligned}\Rightarrow \mathbf{A}_2 &= \mathbf{S}_1 \mathbf{A}_1 = \begin{pmatrix} \frac{3}{\sqrt{10}} & \frac{1}{\sqrt{10}} & 0 \\ -\frac{1}{\sqrt{10}} & \frac{3}{\sqrt{10}} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 3 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 3 \end{pmatrix} \\ &= \begin{pmatrix} \sqrt{10} & \frac{3\sqrt{10}}{5} & \frac{\sqrt{10}}{10} \\ 0 & \frac{4\sqrt{10}}{5} & \frac{3\sqrt{10}}{10} \\ 0 & 1 & 3 \end{pmatrix}\end{aligned}$$

再把 (3,2) 元素变为 0，过程同理

$$\mathbf{A}_3 = \mathbf{S}_2 \mathbf{A}_2 = \begin{pmatrix} \sqrt{10} & \frac{3\sqrt{10}}{5} & \frac{\sqrt{10}}{10} \\ 0 & 2.7203 & 1.9851 \\ 0 & 0 & 2.4412 \end{pmatrix}$$

于是可以进行 QR 分解：

$$\mathbf{Q} = \mathbf{S}_1^T \mathbf{S}_2^T = \begin{pmatrix} 0.94868 & -0.29409 & 0.11625 \\ 0.31623 & 0.88226 & -0.36761 \\ 0 & 0.36761 & 0.92998 \end{pmatrix}$$

迭代：

$$\mathbf{A}^{(2)} = \mathbf{RQ} = \begin{pmatrix} 3.6 & 0.86024 & 0 \\ 0.86024 & 3.12973 & 0.89740 \\ 0 & 0.89740 & 2.27027 \end{pmatrix}$$

...

$$\mathbf{A}^{(13)} = \begin{pmatrix} 4.4139 & 0.01941 & 0 \\ 0.01941 & 3.0003 & 0.00095 \\ 0 & 0.00095 & 1.5858 \end{pmatrix}$$

可以看到，经过多次迭代，矩阵 $\mathbf{A}^{(k)}$ 逐渐变为上三角矩阵，其对角线元素即为原矩阵 $\mathbf{A}$ 的本征值。Householder 变换也是一样的，唯一的区别只是左乘矩阵：

$$\mathbf{P}_n \mathbf{P}_{n-1} \cdots \mathbf{P}_1 \mathbf{A}$$

因为 QR 分解需要把 $\mathbf{A}$ 化为上三角矩阵而不是上 Hessenberg 矩阵，所以初始向量 $\vec{v}$ 应该有变化，要把矩阵元素的第一项也加上！

$$\vec{v} = (a_{11}, a_{21}, \cdots, a_{n1})^T$$

7.4 带位移的 QR 分解

标准的 QR 迭代：

$$\mathbf{A}^{(k)} = \mathbf{Q}^{(k)} \mathbf{R}^{(k)}$$

$$\mathbf{A}^{(k+1)} = \mathbf{R}^{(k)} \mathbf{Q}^{(k)}$$

当 $\mathbf{A}$ 是上 Hessenberg 矩阵时，次对角线元素将逐渐趋于零，收敛速度取决于相邻特征值的比值

$$b \sim \frac{|\lambda_{k+1}|}{|\lambda_k|}$$

如果两个特征值很接近，收敛就会很慢，需要加速。带位移的 QR 迭代通过引入一个位移参数 σ 来加速收敛：首先还是先做预处理，得到上 Hessenberg 矩阵

$$\mathbf{A}_0 = \mathbf{S}_{n-2}^T \cdots \mathbf{S}_1^T \mathbf{A} \mathbf{S}_1 \cdots \mathbf{S}_{n-2}$$

再进行迭代：

$$\mathbf{A}_k - \sigma_k \mathbf{I} = \mathbf{Q}_k \mathbf{R}_k$$

$$\mathbf{A}_{k+1} = \mathbf{R}_k \mathbf{Q}_k + \sigma_k \mathbf{I}$$

此时次对角线元素的收敛速度

$$b \sim \left| \frac{\lambda_{k+1} - \sigma_k}{\lambda_k - \sigma_k} \right|$$

如果 σ_k 选得好, 接近 λ_k , 则收敛速度将大大加快。最常用的选择 σ 的方法是 Wilkinson 位移, 对于当前的上 Hessenberg 矩阵 $\mathbf{A}^{(k)}$, 选取其右下角 2×2 子矩阵:

$$\mathbf{M} = \begin{pmatrix} a_{n-1,n-1}^{(k)} & a_{n-1,n}^{(k)} \\ a_{n,n-1}^{(k)} & a_{nn}^{(k)} \end{pmatrix}$$

计算该子矩阵的两个特征值 μ_1, μ_2 , 选择更靠近 $a_{nn}^{(k)}$ 的特征值作为位移参数 σ_k , 即

$$\sigma_k = \begin{cases} \mu_1, & \text{if } |\mu_1 - a_{nn}^{(k)}| < |\mu_2 - a_{nn}^{(k)}| \\ \mu_2, & \text{otherwise} \end{cases}$$

7.5 广义特征值问题

对于两个矩阵 $\mathbf{A}$ 和 $\mathbf{B}$, 广义特征值问题可以写作

$$\mathbf{A}\vec{x} = \lambda\mathbf{B}\vec{x}$$

如果 $\mathbf{B}$ 是可逆的, 可以通过变换将其转化为标准特征值问题:

$$\mathbf{B}^{-1}\mathbf{A}\vec{x} = \lambda\vec{x}$$

但是 $\mathbf{B}^{-1}\mathbf{A}$ 不一定是对称矩阵, 这会导致数值计算的稳定性较差。当 $\mathbf{A}, \mathbf{B}$ 对称且 $\mathbf{B}$ 正定时, 可以利用 Cholesky 分解:

$$\mathbf{B} = \mathbf{L}\mathbf{L}^T$$

将原问题转化为

$$\mathbf{L}^{-1}\mathbf{A}\mathbf{L}^{-T}\vec{y} = \lambda\vec{y}$$

其中

$$\vec{y} = \mathbf{L}^T\vec{x}$$

7.6 SVD 分解

SVD 分解在第二章中已经进行了简单的讲解, 对于任意的 $m \times n$ 矩阵 $\mathbf{A}$, 无法求解其本征值, 但可以作 SVD 分解:

$$\mathbf{A} = \mathbf{U}\mathbf{D}\mathbf{V}^T$$

其中 $\mathbf{U}$ 是一个 $m \times m$ 的正交矩阵, $\mathbf{V}$ 是一个 $n \times n$ 的正交矩阵, $\mathbf{D}$ 是一个 $m \times n$ 的对角矩阵, 对角线上的元素为 $\mathbf{A}$ 的奇异值。

如果 $\mathbf{A}$ 是对称正定矩阵, 那么 $\mathbf{U} = \mathbf{V}$, SVD 分解就退化为特征值分解, 此时 $\mathbf{D}$ 的对角线元素即为 $\mathbf{A}$ 的本征值。下面来考虑如何实现 SVD 分解:

首先构造 $n \times n$ 矩阵 $\mathbf{A}^T \mathbf{A}$, 这是一个对称半正定矩阵。

$$\mathbf{A}^T = \mathbf{V} \mathbf{D} \mathbf{U}^T$$

$$\Rightarrow \mathbf{A}^T \mathbf{A} = \mathbf{V} \mathbf{D}^2 \mathbf{V}^T$$

$$\mathbf{D}^2 = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$$

$\mathbf{A}^T \mathbf{A}$ 的特征向量就是 SVD 中的右奇异向量 $\mathbf{V}$, 其特征值的平方根就是奇异值 $\omega_i = \sqrt{\lambda_i}$, 这样我们就得到了 $\mathbf{V}$ 和所有的 ω_i 。设 $k = \text{rank}(\mathbf{A})$, 则前 k 个奇异值 $\omega_1, \omega_2, \dots, \omega_k$ 为正, 后面的奇异值 $\omega_{k+1}, \dots, \omega_n$ 为零。

再来计算前 k 个左奇异向量 $\mathbf{U}$, 考虑列向量

$$\vec{u}_i = \frac{1}{\omega_i} \mathbf{A} \vec{v}_i, \quad i = 1, 2, \dots, k$$

可以验证, 这些列向量是正交归一的。

$$\vec{u}_i^T \vec{u}_j = \frac{1}{\omega_i \omega_j} \vec{v}_i^T \mathbf{A}^T \mathbf{A} \vec{v}_j = \frac{\omega_j}{\omega_i} \vec{v}_i^T \vec{v}_j = \delta_{ij}$$

补全 $\mathbf{U}$ 为正交矩阵。任取 $m - k$ 个与 $(\vec{u}_1, \vec{u}_2, \dots, \vec{u}_k)$ 线性无关的向量作为剩余的列向量, 对这些向量作 Gram-Schmidt 正交化, 得到 $\mathbf{U}$ 的完整列向量。

最后再构造对角矩阵 $\mathbf{D}$, 将前 k 个奇异值 $\omega_1, \omega_2, \dots, \omega_k$ 放在对角线上, 剩余元素为零, 即可得到 SVD 分解。

如果 $m > n$, $\mathbf{D}$ 的对角线元素为 $\omega_1, \omega_2, \dots, \omega_n$, 剩余元素为零, 即

$$\mathbf{D} = \begin{pmatrix} \omega_1 & 0 & \cdots & 0 \\ 0 & \omega_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \omega_n \\ 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \cdots & \vdots \\ 0 & 0 & \cdots & 0 \end{pmatrix}$$

如果 $m < n$, 则 $\mathbf{D}$ 的对角线元素为 $\omega_1, \omega_2, \dots, \omega_m$, 剩余元素为零, 即

$$\mathbf{D} = \begin{pmatrix} \omega_1 & 0 & \cdots & 0 & 0 & \cdots & 0 \\ 0 & \omega_2 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \cdots & \vdots \\ 0 & 0 & \cdots & \omega_m & 0 & \cdots & 0 \end{pmatrix}$$

例: 对矩阵作 SVD 分解

$$\mathbf{A} = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 0 \\ 1 & 1 & 0 \end{pmatrix}$$

首先计算 $\mathbf{A}^T \mathbf{A}$:

$$\mathbf{A}^T \mathbf{A} = \begin{pmatrix} 2 & 1 & 1 \\ 1 & 4 & 1 \\ 1 & 1 & 2 \end{pmatrix}$$

求出 $\mathbf{A}^T \mathbf{A}$ 的特征值和特征向量, 得到

$$\lambda_1 = 5, \lambda_2 = 2, \lambda_3 = 1$$

$$\mathbf{V} = \begin{pmatrix} \frac{\sqrt{6}}{6} & \frac{\sqrt{3}}{3} & -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{6}}{3} & -\frac{\sqrt{3}}{3} & 0 \\ \frac{\sqrt{6}}{6} & \frac{\sqrt{3}}{3} & \frac{\sqrt{2}}{2} \end{pmatrix}$$

于是得到奇异值:

$$\omega_1 = \sqrt{5}, \quad \omega_2 = \sqrt{2}, \quad \omega_3 = 1$$

计算前三个左奇异向量:

$$\vec{u}_1 = \frac{1}{\sqrt{5}} \mathbf{A} \vec{v}_1 = \left(\frac{\sqrt{30}}{15}, \frac{\sqrt{30}}{15}, \frac{\sqrt{30}}{10}, \frac{\sqrt{30}}{15}, \frac{\sqrt{30}}{10} \right)^T$$

$$\vec{u}_2 = \frac{1}{\sqrt{2}} \mathbf{A} \vec{v}_2 = \left(\frac{\sqrt{6}}{3}, -\frac{\sqrt{6}}{6}, 0, -\frac{\sqrt{6}}{6}, 0 \right)^T$$

$$\vec{u}_3 = \mathbf{A} \vec{v}_3 = \left(0, 0, \frac{\sqrt{2}}{2}, 0, -\frac{\sqrt{2}}{2} \right)^T$$

再进行 Gram-Schmidt 正交化, 得到完整的 $\mathbf{U}$ 矩阵。

SVD 分解可以进行图像压缩，其核心思想是，一张 $N \times N$ 的灰度图像可以看作一个 $N \times N$ 的矩阵 $\mathbf{A}$ ，每一个矩阵元对应一个像素的亮度，可以对矩阵进行 SVD 分解。为了减少存储空间，只保留前 k 个奇异值及其对应的奇异向量，其余奇异值即为 0，这样就得到了一个秩为 k 的近似矩阵 $\mathbf{A}_k$ ，可以用来近似原图像。通过调整 k 的值，可以在图像质量和存储空间之间进行权衡。

7.7 幂法 (Power Method)

幂法是一种简单而有效的迭代方法，用于求解矩阵的主特征值（即绝对值最大的特征值）及其对应的特征向量。假设一个 $n \times n$ 矩阵 $\mathbf{A}$ 有 n 个线性无关的特征向量 $\vec{v}_1, \vec{v}_2, \dots, \vec{v}_n$ ，对应的特征值为 $\lambda_1, \lambda_2, \dots, \lambda_n$ ，其中 $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$ 。

任取一个初始向量 $\vec{x}$ （要求不与 $\vec{v}_1$ 正交），可以将其表示为特征向量的线性组合：

$$\vec{x} = \beta_1 \vec{v}_1 + \beta_2 \vec{v}_2 + \dots + \beta_n \vec{v}_n = \sum_{i=1}^n \beta_i \vec{v}_i$$

左乘 $\mathbf{A}$ ：

$$\begin{aligned} \mathbf{A}\vec{x} &= \sum_{i=1}^n \beta_i \lambda_i \vec{v}_i \\ \mathbf{A}^2\vec{x} &= \sum_{i=1}^n \beta_i \lambda_i^2 \vec{v}_i \\ &\dots \\ \mathbf{A}^k\vec{x} &= \sum_{i=1}^n \beta_i \lambda_i^k \vec{v}_i \end{aligned}$$

写作

$$\mathbf{A}^k\vec{x} = \lambda_1^k \sum_{i=1}^n \beta_i \left(\frac{\lambda_i}{\lambda_1} \right)^k \vec{v}_i$$

由于 $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$ ，当 k 足够大时， $\left(\frac{\lambda_i}{\lambda_1} \right)^k \rightarrow 0$ ($i = 2, 3, \dots, n$)，因此求和只有第一项有贡献：

$$\lim_{k \rightarrow \infty} \mathbf{A}^k\vec{x} = \lambda_1^k \beta_1 \vec{v}_1$$

下面考虑幂法的算法实现过程：通常采用无穷范数归一化条件

$$\|\mathbf{A}^k\vec{x}\|_{\infty} = 1$$

即每个迭代分量的最大分量绝对值为 1，避免数值溢出。于是幂法的迭代过程如下：

(i) 任取一个初始向量 $\vec{x}^{(0)}$ ，满足 $\|\vec{x}^{(0)}\|_{\infty} = 1$ 。

(ii) 计算

$$\vec{y}^{(0)} = \mathbf{A}\vec{x}^{(0)}$$

随后要把 $\vec{y}^{(0)}$ 归一化, 得到新的迭代向量 $\vec{x}^{(1)}$:

$$\vec{x}^{(1)} = \frac{\vec{y}^{(0)}}{\|\vec{y}^{(0)}\|_\infty}$$

$$\vec{y}^{(1)} = \mathbf{A}\vec{x}^{(1)}$$

如果 $\vec{x}^{(0)}$ 恰好是特征向量, 应该满足 $\mathbf{A}\vec{x}^{(0)} = \lambda\vec{x}^{(0)}$, 所以对于其任意分量 m , 应该满足

$$\frac{(\mathbf{A}\vec{x}^{(0)})_m}{\vec{x}_m^{(0)}} = \lambda$$

刚才看到了, 初始向量 $\vec{x}^{(0)}$ 可以用特征向量的线性组合来表示:

$$\vec{x}^{(0)} = \beta_1\vec{v}_1 + \beta_2\vec{v}_2 + \cdots + \beta_n\vec{v}_n$$

$$\mathbf{A}\vec{x}^{(0)} = \beta_1\lambda_1\vec{v}_1 + \beta_2\lambda_2\vec{v}_2 + \cdots + \beta_n\lambda_n\vec{v}_n$$

于是可以估计最大特征值, 选取 $\vec{x}^{(0)}$ 的最大值对应的分量 m_0 : (注意此时 $\vec{y}^{(0)}$ 还没有归一化)

$$\begin{aligned}\mu^{(0)} &= \frac{(\mathbf{A}\vec{x}^{(0)})_{m_0}}{\vec{x}_{m_0}^{(0)}} = \frac{\beta_1\lambda_1\vec{v}_{1,m_0} + \sum_{i=2}^n \beta_i\lambda_i\vec{v}_{i,m_0}}{\beta_1\vec{v}_{1,m_0} + \sum_{i=2}^n \beta_i\vec{v}_{i,m_0}} \\ &= \lambda_1 \left[\frac{\beta_1\vec{v}_{1,m_0} + \sum_{i=2}^n \beta_i(\lambda_i/\lambda_1)\vec{v}_{i,m_0}}{\beta_1\vec{v}_{1,m_0} + \sum_{i=2}^n \beta_i\vec{v}_{i,m_0}} \right]\end{aligned}$$

(iii) 重复上述过程, 计算

$$\begin{aligned}\vec{y}^{(k)} &= \mathbf{A}\vec{x}^{(k)} \\ \vec{x}^{(k+1)} &= \frac{\vec{y}^{(k)}}{\|\vec{y}^{(k)}\|_\infty} \\ \mu^{(k)} &= \frac{(\mathbf{A}\vec{x}^{(k)})_{m_k}}{\vec{x}_{m_k}^{(k)}} = \lambda_1 \left[\frac{\beta_1\vec{v}_{1,m_k} + \sum_{i=2}^n \beta_i(\lambda_i/\lambda_1)^k\vec{v}_{i,m_k}}{\beta_1\vec{v}_{1,m_k} + \sum_{i=2}^n \beta_i(\lambda_i/\lambda_1)^{k-1}\vec{v}_{i,m_k}} \right]\end{aligned}$$

可以看到, 当 $k \rightarrow \infty$ 时, $\mu^{(k)} \rightarrow \lambda_1$, $\vec{x}^{(k)} \rightarrow \vec{v}_1$, 最终迭代得到最大特征值。

例: 求矩阵的最大本征值

$$\mathbf{A} = \begin{pmatrix} -2 & -3 \\ 6 & 7 \end{pmatrix}$$

规定

$$\vec{x}_0 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

$$\vec{x}_1 = \mathbf{A}\vec{x}_0 = \begin{pmatrix} -5 \\ 13 \end{pmatrix}$$

$$\vec{x}_2 = \mathbf{A}\vec{x}_1 = \begin{pmatrix} -29 \\ 61 \end{pmatrix}$$

$$\vec{x}_3 = \mathbf{A}\vec{x}_2 = \begin{pmatrix} -125 \\ 253 \end{pmatrix}$$

$$\vec{x}_4 = \mathbf{A}\vec{x}_3 = \begin{pmatrix} -509 \\ 1021 \end{pmatrix}$$

$$\vec{x}_5 = \mathbf{A}\vec{x}_4 = \begin{pmatrix} -2045 \\ 4093 \end{pmatrix}$$

$$\vec{x}_6 = \mathbf{A}\vec{x}_5 = \begin{pmatrix} -8189 \\ 16381 \end{pmatrix}$$

最大本征值：

$$\lambda_1^{(1)} = \frac{61}{13} = 4.6923$$

$$\lambda_1^{(2)} = \frac{253}{61} = 4.14754$$

$$\lambda_1^{(3)} = \frac{1021}{253} = 4.03557$$

$$\lambda_1^{(4)} = \frac{4093}{1021} = 4.00881$$

$$\lambda_1^{(5)} = \frac{16381}{4093} = 4.00200$$

对于对称矩阵，可以直接用瑞利商来估计特征值：

$$\mu^{(k)} = \frac{\vec{x}^{(k)T} \mathbf{A} \vec{x}^{(k)}}{\vec{x}^{(k)T} \vec{x}^{(k)}}$$

如果用 2-范数归一化条件，即 $\|\vec{x}^{(k)}\|_2 = 1$ ，则可以写作

$$\mu^{(k)} = \vec{x}^{(k)T} \mathbf{A} \vec{x}^{(k)} = \vec{x}^{(k)T} \vec{y}^{(k)}$$

收敛很快。

7.8 反幂法

反幂法是幂法的一种变体，用于计算离某个给定位移 q 最近的特征值及其特征向量。反幂法不是直接对 $\mathbf{A}$ 做幂法，而是对 $(\mathbf{A} - q\mathbf{I})^{-1}$ 做幂法。若 $\mathbf{A}$ 的特征值为 $\lambda_1, \lambda_2, \dots, \lambda_n$ ，则 $(\mathbf{A} - q\mathbf{I})^{-1}$ 的特征值为

$$\frac{1}{\lambda_1 - q}, \frac{1}{\lambda_2 - q}, \dots, \frac{1}{\lambda_n - q}$$

且特征向量保持不变。

对 $(\mathbf{A} - q\mathbf{I})^{-1}$ 做幂法，会找到模最大的特征值，这等价于找到离 q 最近的 λ_i 。过程和幂法是完全一样的，只不过每次迭代都要解一个线性方程组：

$$(\mathbf{A} - q\mathbf{I})\vec{y}^{(k)} = \vec{x}^{(k)}$$

以避免显式求解逆矩阵。

本征值：

$$\lambda = q + \frac{1}{\mu}$$

7.9 Wielandt 收缩法

我们已经用幂法求出了矩阵 $\mathbf{A}$ 的最大特征值 λ_1 和对应的特征向量 $\vec{v}_1$ ，现在想要继续求出第二大特征值 λ_2 ，以便再用幂法逐个求出所有的特征值。如果直接用幂法还是会收敛到 λ_1 ，所以需要改造矩阵，消除 λ_1 ，让 λ_2 变成新矩阵的最大特征值。构造矩阵：

$$\mathbf{B} = \mathbf{A} - \lambda_1 \vec{v}_1 \vec{x}^T$$

其中 $\vec{v}_1$ 是 $\mathbf{A}$ 的最大特征值 λ_1 对应的特征向量， $\vec{x}$ 满足：

$$\vec{x}^T \vec{v}_1 = 1$$

这个条件是为了让 $\mathbf{B}$ 恰好把 λ_1 变为 0，且不改变其他本征值。

$$\mathbf{B}\vec{v}_1 = \mathbf{A}\vec{v}_1 - \lambda_1 \vec{v}_1 (\vec{x}^T \vec{v}_1) = \lambda_1 \vec{v}_1 - \lambda_1 \vec{v}_1 = 0$$

$\vec{v}_1$ 是 $\mathbf{B}$ 属于本征值 0 的本征向量。

对于 λ_i ，构造向量

$$\vec{w}_i = \vec{v}_i - \frac{\lambda_1 \vec{x}^T \vec{v}_i}{\lambda_i} \vec{v}_1$$

其中 $\vec{v}_i$ 是 $\mathbf{A}$ 的第 i 个特征值 λ_i 对应的特征向量，满足 $\mathbf{A}\vec{v}_i = \lambda_i \vec{v}_i$ 。

计算：

$$\mathbf{B}\vec{w}_i = \mathbf{A}\vec{w}_i - \lambda_1 \vec{v}_1 \vec{x}^T \vec{w}_i$$

$$\begin{aligned}
&= \lambda_i \vec{v}_i - \frac{\lambda_1 \vec{x}^T \vec{v}_i}{\lambda_i} \lambda_1 \vec{v}_1 - \lambda_1 \vec{v}_1 (\vec{x}^T \vec{v}_i - \frac{\lambda_1 \vec{x}^T \vec{v}_i}{\lambda_i} \vec{x}^T \vec{v}_1) \\
&= \lambda_i \vec{v}_i - \frac{\lambda_1^2 \vec{x}^T \vec{v}_i}{\lambda_i} \vec{v}_1 - \lambda_1 \vec{x}^T \vec{v}_i (1 - \frac{\lambda_1}{\lambda_i}) \vec{v}_1 \\
&= \lambda_i \vec{v}_i - \lambda_1 \vec{x}^T \vec{v}_i \vec{v}_1 = \lambda_i \vec{w}_i
\end{aligned}$$

所以 λ_i 也是 $\mathbf{B}$ 的本征值, 对应的本征向量为 $\vec{w}_i$ 。

下面考虑如何构造向量 $\vec{x}$, 一种构造方法是

$$\vec{x} = \frac{1}{\lambda_1 v_{1i}} \begin{pmatrix} a_{i1} \\ a_{i2} \\ \vdots \\ a_{in} \end{pmatrix}$$

其中 v_{1i} 是向量 $\vec{v}_1$ 的第 i 个分量, a_{ij} 是矩阵 $\mathbf{A}$ 的元素。

验证:

$$\vec{x}^T \vec{v}_1 = \frac{1}{\lambda_1 v_{1i}} \sum_{j=1}^n a_{ij} v_{1j} = \frac{1}{\lambda_1 v_{1i}} (\mathbf{A} \vec{v}_1)_i = \frac{1}{\lambda_1 v_{1i}} \lambda_1 v_{1i} = 1$$

作用后 $\mathbf{B}$ 的第 i 行元素全为 0, 因此 $\mathbf{B}$ 的特征值 λ_1 被删除掉了, 剩余的特征值不变。
删除第 i 行和第 i 列, $\mathbf{B}$ 继续用幂法, 就可以求出 λ_2 。

例:

$$\mathbf{A} = \begin{pmatrix} 4 & -1 & 1 \\ -1 & 3 & -2 \\ 1 & -2 & 3 \end{pmatrix}$$

幂法求出 $\lambda_1 = 6$, 对应的特征向量为 $\vec{v}_1 = (1, -1, 1)^T$ 。

构造向量 $\vec{x}$:

$$\vec{x} = \frac{1}{6} \begin{pmatrix} 4 \\ -1 \\ 1 \end{pmatrix}$$

$$\vec{v}_1 \vec{x} = \begin{pmatrix} \frac{2}{3} & -\frac{1}{6} & \frac{1}{6} \\ -\frac{2}{3} & \frac{1}{6} & -\frac{1}{6} \\ \frac{2}{3} & -\frac{1}{6} & \frac{1}{6} \end{pmatrix}$$

$$\mathbf{B} = \mathbf{A} - \lambda_1 \vec{v}_1 \vec{x}^T = \begin{pmatrix} 0 & 0 & 0 \\ 3 & 2 & -1 \\ -3 & -1 & 2 \end{pmatrix}$$

删除第一行和第一列：

$$\mathbf{B}' = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$$

再次用幂法求出 $\lambda_2 = 3$ ，对应的特征向量为 $\vec{v}_2 = (1, -1)^T$ 。

7.10 Krylov 子空间方法

Krylov 子空间方法用于高效地求解大型稀疏矩阵的特征值或线性方程组，而不需要直接处理 $n \times n$ 的完整矩阵。 $\mathbf{A}$ 是我们要考虑的 $n \times n$ 大规模系数矩阵， $\vec{b}$ 是一个 n 维列向量，定义 Krylov 矩阵 $\mathbf{K}$ ：

$$\mathbf{K} = \begin{pmatrix} \vec{b} & \mathbf{A}\vec{b} & \mathbf{A}^2\vec{b} & \cdots & \mathbf{A}^{m-1}\vec{b} \end{pmatrix}$$

n 列线性无关时，可以构成一个 n 维空间，称为 Krylov 子空间，即由 $\{\vec{b}, \mathbf{A}\vec{b}, \mathbf{A}^2\vec{b}, \dots, \mathbf{A}^{n-1}\vec{b}\}$ 张成的子空间，记作 $\mathcal{K}_n(\mathbf{A}, \vec{b})$ 。这个子空间是 $\mathbf{A}$ -不变的，即

$$\mathbf{A}\mathcal{K}_n(\mathbf{A}, \vec{b}) \subseteq \mathcal{K}_n(\mathbf{A}, \vec{b})$$

左乘 $\mathbf{A}$ 得到

$$\mathbf{AK} = \begin{pmatrix} \mathbf{A}\vec{b} & \mathbf{A}^2\vec{b} & \mathbf{A}^3\vec{b} & \cdots & \mathbf{A}^n\vec{b} \end{pmatrix}$$

可以发现： $\mathbf{AK}$ 的前 $n-1$ 列正好是 $\mathbf{K}$ 的第 2 列到第 n 列。最后一列 $\mathbf{A}^n\vec{b}$ 是新的向量，但是必定可以写成 $\mathbf{K}$ 各列的线性组合，即

$$\mathbf{A}^n\vec{b} = c_1\vec{b} + c_2\mathbf{A}\vec{b} + c_3\mathbf{A}^2\vec{b} + \cdots + c_n\mathbf{A}^{n-1}\vec{b} = \mathbf{K}\vec{c}$$

其中

$$\vec{c} = (c_1, c_2, \dots, c_n)^T$$

可以写成：

$$\mathbf{AK} = \mathbf{KG}$$

很容易看到

$$\mathbf{G} = \begin{pmatrix} 0 & 0 & \cdots & 0 & c_1 \\ 1 & 0 & \cdots & 0 & c_2 \\ 0 & 1 & \cdots & 0 & c_3 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & c_n \end{pmatrix}$$

$\mathbf{G}$ 是一个上 Hessenberg 矩阵。于是

$$\mathbf{A} = \mathbf{K}\mathbf{G}\mathbf{K}^{-1}$$

因此 $\mathbf{A}$ 与 $\mathbf{G}$ 相似，具有相同的特征值， $\mathbf{G}$ 是稀疏的，求解其特征值比直接求解 $\mathbf{A}$ 的特征值要容易很多。直接求 $\mathbf{G}$ 理论可行，但数值上极不稳定，几乎无法在计算机上实现。于是我们需要寻找数值稳定的矩阵，考虑对矩阵 $\mathbf{K}$ 作 $\mathbf{QR}$ 分解：

$$\mathbf{K} = \mathbf{Q}\mathbf{R}$$

$\mathbf{Q}$ 为正交矩阵， $\mathbf{R}$ 为上三角矩阵。

$$\Rightarrow (\mathbf{Q}\mathbf{R})^{-1}\mathbf{A}(\mathbf{Q}\mathbf{R}) = \mathbf{G}$$

两边左乘 $\mathbf{R}$ ，再右乘 $\mathbf{R}^{-1}$

$$\mathbf{Q}^T\mathbf{A}\mathbf{Q} = \mathbf{R}\mathbf{G}\mathbf{R}^{-1}$$

记作 $\mathbf{T}$ ：

$$\mathbf{Q}^T\mathbf{A}\mathbf{Q} = \mathbf{R}\mathbf{G}\mathbf{R}^{-1} = \mathbf{T}$$

$\mathbf{T}$ 是一个上 Hessenberg 矩阵，且与 $\mathbf{A}$ 相似，具有相同的特征值。对 $\mathbf{T}$ 进行 QR 迭代，就可以求出 $\mathbf{A}$ 的特征值。

7.11 Lanczos 迭代

如果 $\mathbf{A}$ 为对称矩阵，则 $\mathbf{T}$ 是一个三对角矩阵：

$$\mathbf{T} = \mathbf{Q}^T\mathbf{A}\mathbf{Q} = \begin{pmatrix} \alpha_1 & \beta_1 & 0 & \cdots & 0 \\ \beta_1 & \alpha_2 & \beta_2 & \cdots & 0 \\ 0 & \beta_2 & \alpha_3 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & \alpha_n \end{pmatrix}$$

这个三对角矩阵 $\mathbf{T}$ 可以通过 Lanczos 迭代来构造。可以将矩阵 $\mathbf{Q}$ 写作：

$$\mathbf{Q} = (\vec{q}_1 \quad \vec{q}_2 \quad \cdots \quad \vec{q}_n)$$

于是 $\mathbf{Q}\mathbf{T} = \mathbf{A}\mathbf{Q}$ 可以写作：

$$\mathbf{A}\vec{q}_i = \beta_{i-1}\vec{q}_{i-1} + \alpha_i\vec{q}_i + \beta_i\vec{q}_{i+1}$$

其中 $\beta_0 = 0$, $\vec{q}_0 = \vec{0}$, $\beta_n = 0$, $\vec{q}_{n+1} = \vec{0}$ 。

$\mathbf{Q}$ 为正交矩阵，满足

$$\vec{q}_i^T \vec{q}_j = \delta_{ij}$$

先求 α_i ，用 $\vec{q}_i^T$ 左乘：

$$\alpha_i = \vec{q}_i^T \mathbf{A} \vec{q}_i$$

再求 β_i ，定义残量：

$$\vec{v} = \mathbf{A} \vec{q}_i - \alpha_i \vec{q}_i - \beta_{i-1} \vec{q}_{i-1} = \beta_i \vec{q}_{i+1}$$

$$\Rightarrow \beta_i = \|\vec{v}\|_2$$

$$\vec{q}_{i+1} = \frac{\vec{v}}{\beta_i}$$

也可以得到

$$\vec{q}_{i+1} \cdot \vec{v} = \frac{\vec{v} \cdot \vec{v}}{\beta_i} = \beta_i$$

如果迭代步数 m 等于矩阵维度 n ，得到的结果是精确的。

$$\mathbf{T} = \mathbf{V} \mathbf{E} \mathbf{V}^T$$

其中 $\mathbf{E}$ 为特征值对角矩阵。

$$\Rightarrow \mathbf{A} = \mathbf{Q} \mathbf{V} \mathbf{E} \mathbf{V}^T \mathbf{Q}^T$$

实际情况中，迭代步数 m 远小于 n ，得到的是近似解。

$$\mathbf{A} \approx \mathbf{Q} \mathbf{V} \mathbf{E} \mathbf{V}^T \mathbf{Q}^T$$

例：

$$\mathbf{A} = \begin{pmatrix} 3 & 2 & 5 \\ 2 & 3 & 5 \\ 5 & 5 & 0 \end{pmatrix}$$

对其作两次迭代，设初始矢量

$$\vec{q}_0 = \mathbf{0}$$

迭代得到

$$\vec{q}_1 = (1, 0, 0)^T$$

$$\vec{q}_2 = \frac{1}{\sqrt{29}}(0, 2, 5)^T$$

$$\mathbf{T} = \begin{pmatrix} 3 & \sqrt{29} \\ \sqrt{29} & \frac{112}{29} \end{pmatrix}$$

得到本征值的近似值：

$$\lambda_1 \approx 8.83, \quad \lambda_2 \approx -1.97$$

7.12 Arnoldi 迭代

Arnoldi 迭代是 Lanczos 迭代的推广, 适用于非对称矩阵。与 Lanczos 迭代类似, Arnoldi 迭代也构造一个 Krylov 子空间, 并在该子空间内构造一个上 Hessenberg 矩阵 $\mathbf{H}$, 使得 $\mathbf{A}$ 与 $\mathbf{H}$ 相似, 从而求解 $\mathbf{A}$ 的特征值。设矩阵:

$$\mathbf{H} = \begin{pmatrix} h_{11} & h_{12} & h_{13} & \cdots & h_{1n} \\ h_{21} & h_{22} & h_{23} & \cdots & h_{2n} \\ 0 & h_{32} & h_{33} & \cdots & h_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & h_{nn} \end{pmatrix}$$

$$\mathbf{H} = \mathbf{Q}^T \mathbf{A} \mathbf{Q}$$

$$\Rightarrow \mathbf{A} \vec{q}_j = \sum_{i=1}^{j+1} h_{ij} \vec{q}_i$$

第一列:

$$\mathbf{A} \vec{q}_1 = h_{11} \vec{q}_1 + h_{21} \vec{q}_2$$

$$\Rightarrow h_{11} = \vec{q}_1^T \mathbf{A} \vec{q}_1$$

$$\vec{v}_1 = \mathbf{A} \vec{q}_1 - h_{11} \vec{q}_1$$

$$h_{21} = \|\vec{v}_1\|_2$$

$$\vec{q}_2 = \frac{\vec{v}_1}{h_{21}}$$

第二列:

$$\mathbf{A} \vec{q}_2 = h_{12} \vec{q}_1 + h_{22} \vec{q}_2 + h_{32} \vec{q}_3$$

$$\Rightarrow h_{12} = \vec{q}_1^T \mathbf{A} \vec{q}_2$$

$$h_{22} = \vec{q}_2^T \mathbf{A} \vec{q}_2$$

$$\vec{v}_2 = \mathbf{A} \vec{q}_2 - h_{12} \vec{q}_1 - h_{22} \vec{q}_2$$

$$h_{32} = \|\vec{v}_2\|_2$$

$$\vec{q}_3 = \frac{\vec{v}_2}{h_{32}}$$

以此类推即可。

7.13 DMRG 算法

嘻嘻

8 蒙特卡洛算法 (Monte Carlo Method)

蒙特卡洛算法的核心思想是利用随机抽样来解决传统方法难以处理的积分优化等问题。1946 年由 Stanislaw Ulam 和 John von Neumann 等人提出，最初用于核武器设计中的中子扩散问题。现在已经广泛应用于物理学、金融学、计算机科学等领域。

8.1 统计概率基础

(1) 概率密度函数 $p(x)$

描述随机连续变量 X 在 x 附近的概率密度，满足归一化条件

$$\int_{-\infty}^{\infty} p(x) dx = 1$$

对于独立变量，有

$$p(x, y) = p(x)p(y)$$

(2) 累积分布函数 $P(x)$

给出随机变量取值不超过 x 的概率

$$P(x) = \int_{-\infty}^x p(t) dt$$

(3) 期望值 $E[f(x)]$

$$E[f(x)] = \int_{-\infty}^{\infty} f(x)p(x) dx$$

(4) n 阶中心矩

$$\langle (x - \langle x \rangle)^n \rangle = \int_{-\infty}^{\infty} (x - \langle x \rangle)^n p(x) dx$$

$n = 2$ 时给出方差。

(6) 中心极限定理

设 $x_1, x_2, \dots, x_n$ 是独立同分布的随机变量，服从任意概率分布 $p(x)$ (期望值为 μ ，方差为 σ^2)，定义样本均值：

$$z = \frac{1}{n} \sum_{i=1}^n x_i$$

当 $n \rightarrow \infty$ 时， z 的概率分布趋近于正态分布：

$$p(z) = \frac{1}{\sqrt{2\pi}\sigma/\sqrt{n}} \exp \left[-\frac{(z - \mu)^2}{2\sigma^2/n} \right]$$

(7) 随机数生成器

一个好的随机数生成器应该满足：

- (i) 均匀性：生成的数列在 $[0, 1]$ 上服从均匀分布；
- (ii) 独立性：连续随机数之间的相关性可以忽略不计；
- (iii) 周期长：数列重复的周期尽可能大，避免短时间内循环；
- (iv) 速度快：算法计算效率高。

计算机生成的随机数本质上是通过确定性的递推公式产生的，因此是伪随机数。只要初值（种子）相同，序列就会重复，这就引入了周期性。

下面给出四种常见的伪随机数生成算法：

- (i) Logistic 映射

$$x_{i+1} = rx_i(1 - x_i)$$

在一定的参数下可以产生混沌序列，但是分布不均匀，不常用作标准随机数生成器。

- (ii) 线性同余生成器（LCG）

$$N_i = (aN_{i-1} + c) \mod m$$

这是一种经典且快速的方法，但周期受限于 m 。

- (iii) 带记忆的线性同余

$$N_i = (aN_{i-1} + cN_{i-j}) \mod m$$

使用了更早的状态，可以延长周期。

- (iv) 减法的生成器

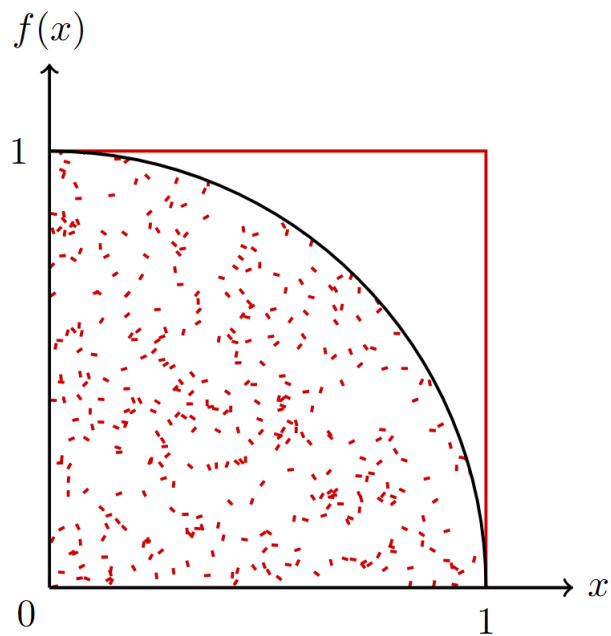
$$N_i = (N_{i-3} - N_{i-1}) \mod (2^{31} - 1)$$

8.2 随机投点法

随机投点法是一种简单的蒙特卡洛方法，用于估计几何图形的面积或体积。基本思想是，在一个已知面积或体积的区域内随机投点，统计落在目标图形内的点的比例，从而估计目标图形的面积或体积。

- (i) 在矩形区域 $[0, 1] \times [0, 1]$ 内随机均匀投点 N 次，每个点的坐标 (x_i, y_i) 独立取值；
- (ii) 判断每个点是否落在曲线 $y = f(x)$ 下方的区域 Ω 内，即条件 $y_i \leq f(x_i)$ 是否成立；
- (iii) 设 n 为满足条件的点数，则曲线下面积的蒙特卡洛估计值为

$$I = \frac{n}{N}$$



8.3 期望值估计法

函数值的期望值为

$$\langle f \rangle = \frac{1}{N} \sum_{i=1}^N f(x_i) p(x_i)$$

样本均值的方差为

$$\sigma_f^2 = \frac{1}{N} \sum_{i=1}^N (f(x_i) - \langle f \rangle)^2 p(x_i)$$

对于均匀采样

$$\sigma_f^2 = \frac{1}{N} \sum_{i=1}^N f(x_i)^2 - \left(\frac{1}{N} \sum_{i=1}^N f(x_i) \right)^2 = \langle f^2 \rangle - \langle f \rangle^2$$

N 次采样的方差:

$$\begin{aligned} \sigma_N^2 &= \frac{\sigma_f^2}{N} \\ \Rightarrow \sigma_N &\sim \frac{1}{\sqrt{N}} \end{aligned}$$

现在我们想计算积分:

$$I = \int_a^b f(x) dx$$

如果是均匀采样, 则

$$p(x) = \frac{1}{b-a}$$

$$\Rightarrow I \approx \frac{b-a}{N} \sum_{i=1}^N f(x_i)$$

多维积分:

$$I \approx \frac{V}{N} \sum_{i=1}^N f(x_i)$$

误差估计:

$$\text{Error} \approx \frac{\sigma_f}{\sqrt{N}}$$

可以看到, 误差与积分维度是无关的, 只依赖于样本量 n 和函数本身的波动。

但是如果函数 $f(x)$ 只在很小的区域内显著非零, 均匀采样的效率很低, 此时可以引入采样分布 $p(x)$ 。如果从采样分布 $p(x)$ 中抽取独立样本 $x_1, x_2, \dots, x_N$ ($\int p(x)dx = 1$), 则可以用样本均值来估计期望值:

$$I = \int_a^b f(x)dx = \int_a^b \frac{f(x)}{p(x)} p(x)dx \approx \frac{1}{N} \sum_{i=1}^N \frac{f(x_i)}{p(x_i)}$$

但是直接生成服从 $p(x)$ 的随机数有时候并不方便, 可以通过变量替换, 将积分变为均匀分布的形式, 定义累积分布函数:

$$y(x) = \int_a^x p(t)dt$$

当 x 从 a 到 b 时, y 从 0 到 1, 并且有:

$$dy = p(x)dx$$

于是积分变为:

$$I = \int_a^b p(x) \frac{f(x)}{p(x)} dx = \int_0^1 \frac{f(x(y))}{p(x(y))} dy = \frac{1}{N} \sum_{i=1}^N \frac{f(x(y_i))}{p(x(y_i))}$$

例: 如果取

$$p(x) = e^{-x}$$

$$\Rightarrow dy(x) = p(x)dx = e^{-x}dx$$

$$y(x) = \int_0^x e^{-t}dt = 1 - e^{-x}$$

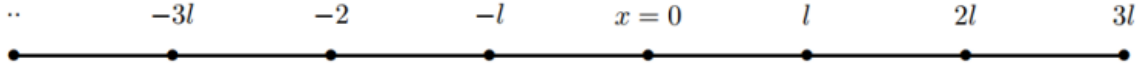
$$\Rightarrow x(y) = -\ln(1-y)$$

求积分:

$$I = \int_0^\infty e^{-x} G(x) dx = \int_0^1 G(-\ln(1-y)) dy \approx \frac{1}{N} \sum_{i=1}^N G(x(y_i))$$

8.4 一维随机行走

建立一维随机行走模型：在一维直线上，质点每一步可以向左或向右移动，步长为 l 。向右的概率为 R ，向左的概率为 L ，满足 $R + L = 1$ 。每一步的位移 $\Delta x_i = \pm l$ ，如图 3 所示。



首先考虑对称的情形，即 $R = L = \frac{1}{2}$ 。

经过 n 步之后的总位移：

$$x(n) = \sum_{i=1}^n \Delta x_i$$

期望值：

$$\langle x(n) \rangle = \sum_{i=1}^n \langle \Delta x_i \rangle = \sum_{i=1}^n \left(\frac{1}{2}l - \frac{1}{2}l \right) = 0$$

方差：因为期望值为零，于是

$$\sigma_n^2 = \langle x(n)^2 \rangle = \left\langle \sum_{i=1}^n \Delta x_i \sum_{j=1}^n \Delta x_j \right\rangle = \sum_{i=1}^n \langle \Delta x_i^2 \rangle + \sum_{i \neq j} \langle \Delta x_i \Delta x_j \rangle = nl^2$$

如果是非对称情形：

$$\langle \Delta x \rangle = n(R - L)l$$

$$\sigma_n^2 = n[l^2 - (R - L)^2 l^2] = 4nRLl^2$$

8.5 马尔科夫过程 (Markov Process)

马尔科夫过程是一种随机过程，满足无记忆性，即未来状态仅依赖于当前状态，而与过去的状态无关。定义转移概率矩阵 $\mathbf{W}_{ij}(\varepsilon)$ 为在时间 ε 内从格点 j 转移到格点 i 的概率。对于一维对称随机行走：

$$\mathbf{W}_{ij}(\varepsilon) = \begin{cases} \frac{1}{2}, & |i - j| = 1 \\ 0, & \text{else} \end{cases}$$

8.5.1 时间演化

经过一个时间步长 ε 后，粒子到达格点 i 的概率 $w_i(t + \varepsilon)$ 等于：从所有可能的上一位置 j 出发，由 j 转移到 i 的概率乘以粒子在 t 时刻位于 j 的概率，再对所有可能的 j 求和，即：

$$w_i(t + \varepsilon) = \sum_j \mathbf{W}_{ij}(\varepsilon) w_j(t)$$

多步演化：

$$\begin{aligned} w_i(2\varepsilon) &= \sum_j \mathbf{W}_{ij} w_j(\varepsilon) = \sum_j \mathbf{W}_{ij} \sum_k \mathbf{W}_{jk} w_k(0) \\ &= \sum_k \sum_j \mathbf{W}_{ij} \mathbf{W}_{jk} w_k(0) = \sum_k (\mathbf{W}^2)_{ik} w_k(0) \end{aligned}$$

以此类推：

$$w_i(n\varepsilon) = \sum_k (\mathbf{W}^n)_{ik} w_k(0)$$

考虑一个 4×4 的小系统：右移矩阵 $\mathbf{R}$ 将向量中的每个分量向右移动一个格子

$$\mathbf{R} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

左移矩阵 $\mathbf{L}$ 将向量中的每个分量向左移动一个格子

$$\mathbf{L} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

转移矩阵为

$$\mathbf{W} = \frac{1}{2}(\mathbf{R} + \mathbf{L})$$

2 步转移矩阵有四种走法：向左移 2 步，向右移 2 步，先向左移 1 步再向右移 1 步，先向右移 1 步再向左移 1 步。

$$\mathbf{W}^2 = \frac{1}{4}(\mathbf{R}^2 + \mathbf{L}^2 + \mathbf{RL} + \mathbf{LR}) = \frac{1}{4}(\mathbf{R}^2 + \mathbf{L}^2 + 2\mathbf{RL})$$

3 步转移矩阵：

$$\mathbf{W}^3 = \frac{1}{8}(\mathbf{R}^3 + \mathbf{L}^3 + 3\mathbf{R}^2\mathbf{L} + 3\mathbf{RL}^2)$$

已知二项式定理：

$$(a+b)^n = \sum_{k=0}^n \binom{n}{k} a^{n-k} b^k$$

于是 n 步转移矩阵可以写作：

$$\mathbf{W}^n = \frac{1}{2^n} \sum_{k=0}^n \binom{n}{k} \mathbf{R}^{n-k} \mathbf{L}^k$$

或者也可以写作：

$$\mathbf{W}^n = \frac{1}{2^n} \sum_{k=0}^n \binom{n}{k} \mathbf{L}^{n-2k} = \frac{1}{2^n} \sum_{k=0}^n C_n^k \mathbf{R}^{2k-n}$$

矩阵元

$$(\mathbf{R}^m)_{ij} = \delta_{i,j+m}$$

$$(\mathbf{L}^m)_{ij} = \delta_{i+m,j}$$

从起点 j 出发，经过 n 步后到达位置 i 的概率为

$$W_{ij}^n = \frac{1}{2^n} \binom{n}{\frac{n+i-j}{2}}$$

其中 $\frac{n+i-j}{2}$ 必须是整数，且满足 $0 \leq \frac{n+i-j}{2} \leq n$ 。

8.5.2 扩散方程

假设粒子从 $x = 0$ 出发，初始分布为

$$w_j(0) = \delta_{j0}$$

经过 n 步，到达格点 i 的概率为

$$w_i(n\varepsilon) = \frac{1}{2^n} \binom{n}{\frac{n+i}{2}}$$

组合数满足公式：

$$\begin{aligned} \binom{n+1}{k} &= \binom{n}{k} + \binom{n}{k-1} \\ \Rightarrow \binom{n+1}{\frac{n+1+i}{2}} &= \binom{n}{\frac{n+i+1}{2}} + \binom{n}{\frac{n+i-1}{2}} \end{aligned}$$

$$\Rightarrow \frac{1}{2^{n+1}} \binom{n+1}{\frac{n+1+i}{2}} = \frac{1}{2^{n+1}} \binom{n}{\frac{n+i+1}{2}} + \frac{1}{2^{n+1}} \binom{n}{\frac{n+i-1}{2}}$$

注意到

$$\frac{1}{2^{n+1}} \binom{n+1}{\frac{n+1+i}{2}} = w_i[(n+1)\varepsilon]$$

$$\frac{1}{2^{n+1}} \binom{n}{\frac{n+i+1}{2}} = \frac{1}{2} w_{i+1}(n\varepsilon)$$

$$\frac{1}{2^{n+1}} \binom{n}{\frac{n+i-1}{2}} = \frac{1}{2} w_{i-1}(n\varepsilon)$$

因此

$$w_i[(n+1)\varepsilon] = \frac{1}{2} w_{i+1}(n\varepsilon) + \frac{1}{2} w_{i-1}(n\varepsilon)$$

令

$$x = il, \quad t = n\varepsilon$$

$$w(x, t) = w_i(n\varepsilon)$$

$$\Rightarrow w(x, t + \varepsilon) = \frac{1}{2} w(x + l, t) + \frac{1}{2} w(x - l, t)$$

这就是离散时间的扩散方程。它表明下一时刻在 x 处的概率等于当前时刻左右邻点概率的平均。移项并整理成差分形式：

$$\frac{w(x + l, t + \varepsilon) - w(x, t)}{\varepsilon} = \frac{l^2}{2\varepsilon} \cdot \frac{w(x + l, t) - 2w(x, t) + w(x - l, t)}{l^2}$$

当

$$\varepsilon \rightarrow 0, \quad l \rightarrow 0, \quad \frac{l^2}{2\varepsilon} \rightarrow D$$

可以写作：

$$\frac{\partial w}{\partial t} = D \frac{\partial^2 w}{\partial x^2}$$

这就是一维扩散方程，其中 D 为扩散系数。

方差：

$$\langle x^2 \rangle - \langle x \rangle^2 = 2Dt$$

在无限长线上，有高斯分布解：

$$w(x, t) = \frac{1}{\sqrt{4\pi Dt}} \exp\left(-\frac{x^2}{4Dt}\right)$$

例：转移矩阵

$$\mathbf{W} = \begin{pmatrix} \frac{1}{4} & \frac{1}{8} & \frac{2}{3} \\ \frac{3}{4} & \frac{5}{8} & 0 \\ 0 & \frac{1}{4} & \frac{1}{3} \end{pmatrix}$$

初始时刻

$$\vec{w}(0) = (1, 0, 0)^T$$

时间演化：

$$\vec{w}(\varepsilon) = \mathbf{W}\vec{w}(0) = \left(\frac{1}{4}, \frac{3}{4}, 0\right)^T$$

$$\vec{w}(2\varepsilon) = \mathbf{W}\vec{w}(\varepsilon) = \left(\frac{5}{32}, \frac{21}{32}, \frac{3}{16}\right)^T$$

平衡态满足：

$$\vec{w}(\infty) = \mathbf{W}\vec{w}(\infty)$$

可以写作：

$$W_{11}w_1(\infty) + W_{12}w_2(\infty) + W_{13}w_3(\infty) = w_1(\infty)$$

$$W_{21}w_1(\infty) + W_{22}w_2(\infty) + W_{23}w_3(\infty) = w_2(\infty)$$

$$W_{31}w_1(\infty) + W_{32}w_2(\infty) + W_{33}w_3(\infty) = w_3(\infty)$$

归一化条件：

$$\sum_i w_i(\infty) = 1$$

解得

$$\vec{w}(\infty) = \left(\frac{4}{15}, \frac{8}{15}, \frac{1}{5}\right)^T$$

马尔科夫链是收敛的，将初始分布向量 $\vec{w}(0)$ 按照转移矩阵 $\mathbf{W}$ 的特征向量 $\vec{v}_i$ 展开：

$$\vec{w}(0) = \sum_i \alpha_i \vec{v}_i$$

经过 t 步后：

$$\vec{w}(t) = \mathbf{W}^t \vec{w}_0 = \sum_i \lambda_i^t \alpha_i \vec{v}_i$$

对于马尔科夫链，最大的特征值总是 1，其余的特征值的模小于 1。因此当 $t \rightarrow \infty$ 时，只有 $\lambda_i = 1$ 的项保留：

$$\vec{w}(\infty) \propto \text{对应于特征值 1 的特征向量}$$

对于刚才的例子，能够得到对角化后的特征值：

$$\mathbf{D} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \frac{1}{48}(5 - i\sqrt{311}) & 0 \\ 0 & 0 & \frac{1}{48}(5 + i\sqrt{311}) \end{pmatrix}$$

特征矢量为：

$$\begin{pmatrix} \frac{4}{3} \\ \frac{8}{3} \\ 1 \end{pmatrix}, \quad \begin{pmatrix} -\frac{1}{12} + i\frac{\sqrt{311}}{12} \\ -\frac{11}{12} - i\frac{\sqrt{311}}{12} \\ 1 \end{pmatrix}, \quad \begin{pmatrix} -\frac{1}{12} - i\frac{\sqrt{311}}{12} \\ -\frac{11}{12} + i\frac{\sqrt{311}}{12} \\ 1 \end{pmatrix}$$

将第一个特征矢量归一化，得到

$$\vec{w}(\infty) = \left(\frac{4}{15}, \frac{8}{15}, \frac{1}{5} \right)^T$$

8.6 Metropolis 算法

首先考虑马尔科夫链的平衡条件：

$$\frac{d\vec{w}_i}{dt} = \sum_j [\mathbf{W}(j \rightarrow i)\vec{w}_j - \mathbf{W}(i \rightarrow j)\vec{w}_i]$$

平衡时满足：

$$\begin{aligned} \frac{d\vec{w}_i}{dt} &= 0 \\ \Rightarrow \mathbf{W}(j \rightarrow i)\vec{w}_j &= \mathbf{W}(i \rightarrow j)\vec{w}_i, \quad \forall i, j \end{aligned}$$

这是体系达到平衡的充分不必要条件。

当直接知道 $\mathbf{W}$ 很困难时（分布复杂、维度高、归一化常数未知等），Metropolis 算法通过对称提议分布 + 接受概率，构造出满足细致平衡条件的转移矩阵，从而使采样分布收敛到目标分布。

将转移矩阵分解：

$$\mathbf{W}(j \rightarrow i) = \mathbf{T}(j \rightarrow i)\mathbf{A}(j \rightarrow i)$$

其中 $\mathbf{T}(j \rightarrow i)$ 是提议分布（如对称随机行走）， $\mathbf{A}(j \rightarrow i)$ 是接受概率。

考虑所有可能转移后，状态 i 的分布更新为：

$$\vec{w}_i(t+1) = \sum_j [w_j(t)T(j \rightarrow i)\mathbf{A}(j \rightarrow i) + w_i(t)T(i \rightarrow j)(1 - \mathbf{A}(i \rightarrow j))]$$

第一项为从 j 转移到 i 的概率，第二项为从 i 转移到 j 但被拒绝的概率。

离散时间演化微分：

$$w_i(t+1) - w_i(t) = \sum_j [w_j(t)\mathbf{T}(j \rightarrow i)\mathbf{A}(j \rightarrow i) - w_i(t)\mathbf{T}(i \rightarrow j)\mathbf{A}(i \rightarrow j)]$$

平衡时，满足：

$$w_i(t+1) = w_i(t)$$

即

$$\sum_j w_j \mathbf{T}(j \rightarrow i) \mathbf{A}(j \rightarrow i) = \sum_j w_i \mathbf{T}(i \rightarrow j) \mathbf{A}(i \rightarrow j)$$

细致平衡条件是更强的条件，每一项分别相等：

$$w_j \mathbf{T}(j \rightarrow i) \mathbf{A}(j \rightarrow i) = w_i \mathbf{T}(i \rightarrow j) \mathbf{A}(i \rightarrow j)$$

等价于：

$$\frac{\mathbf{W}(j \rightarrow i)}{\mathbf{W}(i \rightarrow j)} = \frac{w_i}{w_j}$$

$$\frac{\mathbf{T}(j \rightarrow i) \mathbf{A}(j \rightarrow i)}{\mathbf{T}(i \rightarrow j) \mathbf{A}(i \rightarrow j)} = \frac{w_i}{w_j}$$

Metropolis 算法通常取对称提议：

$$\mathbf{T}(j \rightarrow i) = \mathbf{T}(i \rightarrow j)$$

并设：

$$\mathbf{A}(j \rightarrow i) = \min \left(1, \frac{w_i}{w_j} \right)$$

8.6.1 平衡态统计物理应用

在温度 T 下，系统处于微观态 i 的概率满足 Boltzmann 分布：

$$w_i = \frac{e^{-\beta E_i}}{Z}, \quad \beta = \frac{1}{k_B T}$$

期望值：

$$\langle A \rangle = \sum_i A_i w_i = \frac{\sum_i A_i e^{-\beta E_i}}{Z}$$

直接计算期望值需要遍历所有状态，这通常是难以实现的。Metropolis 算法只需要考虑能量差：

$$\frac{w_i}{w_j} = e^{-\beta(E_i - E_j)}$$

接收概率：

$$A(j \rightarrow i) = \min \left(1, e^{-\beta(E_i - E_j)} \right) = \begin{cases} 1, & E_i \leq E_j \\ e^{-\beta(E_i - E_j)}, & E_i > E_j \end{cases}$$

新状态能量更低或相等，总是接受；若能量更高，则以概率 $e^{-\beta(E_i - E_j)}$ 接受。

8.7 MCMC 积分

可以用马尔科夫链蒙特卡洛方法来计算高维积分。首先来介绍“后验分布”的概念，在贝叶斯统计中，它表示“在给定观测数据的条件下，某个参数或一组参数的概率分布”。具体来说，后验分布是指在已有观测数据的基础上，更新对某个参数或模型的认识。贝叶斯定理给出：

$$P(\theta|D) = \frac{P(D|\theta)P(\theta)}{P(D)}$$

$P(\theta|D)$ 是参数 θ 的后验分布，表示在给定数据 D 后，更新的参数 θ 的概率分布。

$P(D|\theta)$ 是似然函数，表示在给定参数 θ 时，数据 D 出现的概率。

$P(\theta)$ 是参数 θ 的先验分布，表示在没有观测数据之前，对参数 θ 的初始认识。

$P(D)$ 是数据 D 的边际似然或证据，是一个 1 常数，用来保证后验分布的归一化。

举一个典型的例子：扔硬币。假设硬币正面的概率为 p ，但是硬币不一定是公平的。先验分布是在实验前，我们对 p 的看法，可以认为在 $0 \sim 1$ 之间都有可能，先验分布为均匀分布。

$$P(p) = \begin{cases} 1, & 0 \leq p \leq 1 \\ 0, & \text{else} \end{cases}$$

收集证据：抛 10 次硬币，结果是 7 次正面，3 次反面。似然函数为：

$$P(D|p) = \binom{10}{7} p^7 (1-p)^3$$

根据贝叶斯定理，后验分布为：

$$P(p|D) \propto p^7 (1-p)^3$$

这是一个 Beta 分布，表示在观察到数据后，对参数 p 的更新认识。

后验分布的期望值为：

$$E(p|D) = \int_0^1 p P(p|D) dp = \frac{2}{3}$$

这表明在观察到数据后，我们对 p 的估计是 $\frac{2}{3}$ ，即认为硬币正面出现的概率的平均值大约是 $\frac{2}{3}$ 。但是在实际的后验分布期望值计算中，后验分布通常是难以求解的，尤其是在高维参数空间中。此时可以使用 MCMC 方法来近似计算后验分布的期望值。

MCMC 积分不直接给出后验分布的解析表达式，而是通过构造一条马尔科夫链，使其平稳分布等于后验分布 $p(\theta|D)$ 。然后从链上抽取大量样本，这些样本就可以看作是来自后验分布的近似独立样本。

$$\int g(\theta) p(\theta|D) d\theta \approx \frac{1}{N} \sum_{i=1}^N g(\theta_i)$$

(i) 直接采样：优点是样本之间不相关，统计效率高；缺点是只能处理少数简单分布，对于复杂的多体波函数没有现成的直接采样方法。

(ii) 马尔科夫链采样：缺点是样本相关，有效样本量减少，效率较低；优点是适用范围广，几乎可以对任意分布采样，只要能定义从当前状态生成下一状态的规则。

8.8 哈密顿蒙特卡洛

哈密顿蒙特卡洛是对 Metropolis 算法的一种改进，通过引入动量变量和哈密顿动力学来更高效地探索状态空间。将目标分布 $\pi(q)$ 视作势能分布：

$$\pi(q) \propto e^{-U(q)}$$

引入辅助动量变量 p ，通常取独立正态分布：

$$\pi(p) \propto e^{-K(p)}$$

$$K(p) = \frac{p^2}{2m}$$

联合分布：

$$\pi(q, p) = \pi(q)\pi(p) \propto e^{-H(q, p)}$$

$$H(q, p) = U(q) + K(p)$$

哈密顿方程：

$$\begin{aligned} \frac{dq}{dt} &= \frac{\partial H}{\partial p} = \frac{p}{m} \\ \frac{dp}{dt} &= -\frac{\partial H}{\partial q} = -\frac{\partial U}{\partial q} \end{aligned}$$

它们定义了一个从 (q, p) 到 (q', p') 的时间演化映射。接受概率

$$A = \min(1, e^{-H(q', p') + H(q, p)})$$

8.9 量子蒙特卡洛 (QMC)

我们想要求解多体系统的基态能量 E_0 ，但无法直接对角化巨大的哈密顿量。根据变分原理，对于任意尝试的依赖于参数 $\vec{\alpha}$ 的波函数 $\vec{\phi}$ ，其能量期望值满足：

$$E_0 \leq \frac{\int \vec{\phi}^* \mathbf{H} \vec{\phi} d\vec{R}}{\int |\vec{\phi}|^2 d\vec{R}}$$

其中 $\vec{R}$ 是所有 N 个粒子的坐标， $\vec{\phi} = \vec{\phi}(\vec{R}, \vec{\alpha})$ 是尝试波函数， $\vec{\alpha}$ 是变分参数； $\mathbf{H}$ 是体系哈密顿量。通过优化参数 $\vec{\alpha}$ 使得能量期望值最小化，可以得到一个接近基态的波函数 $\vec{\phi}$ ，从而估计 E_0 。

算法的循环过程：

- (i) 构造：选定一个含参数的 $\vec{\alpha}$ 和 $\vec{\phi}$ ；
- (ii) 计算能量期望值：用蒙特卡洛方法从概率密度

$$P(\vec{R}) = \frac{|\vec{\phi}|^2}{\int |\vec{\phi}|^2 d\vec{R}}$$

中抽取样本，然后估计

$$\langle H \rangle = \int P(\vec{R}) \frac{\mathbf{H}\vec{\phi}}{\vec{\phi}} d\vec{R}$$

- (iii) 优化：利用某种最小化算法，调整参数 $\vec{\alpha}$ ，使得 $\langle H \rangle$ 最小化。
- 一般多体问题的哈密顿量可以写作：

$$H = -\frac{\hbar^2}{2m} \sum_{i=1}^N \nabla_i^2 + \sum_i V_{\text{onebody}}(\vec{r}_i) + \sum_{i<j} V_{\text{int}}(|\vec{r}_i - \vec{r}_j|)$$

定义局部能量：

$$E_L(\vec{R}) = \frac{\mathbf{H}\vec{\phi}(\vec{R})}{\vec{\phi}(\vec{R})}$$

代入得

$$E_L(\vec{R}) = \frac{1}{\vec{\phi}(\vec{R})} \left(-\frac{\hbar^2}{2m} \sum_i \nabla_i^2 \vec{\phi}(\vec{R}) \right) + \sum_i V_{\text{onebody}}(\vec{r}_i) + \sum_{i<j} V_{\text{int}}(|\vec{r}_i - \vec{r}_j|)$$

采样后即可直接计算积分：

$$\langle H \rangle = \int P(\vec{R}) E_L(\vec{R}) d\vec{R} \approx \frac{1}{N} \sum_{k=1}^N E_L(\vec{R}_k)$$

下面来考虑如何从 $P(\vec{R}) \propto |\vec{\phi}|^2$ 中取样，使用最经典的 Metropolis 算法即可。如果当前构型为 $\vec{R}$ ，提议新构型 $\vec{R}_p = \vec{R} + \Delta\vec{R}$ ，其中 $\Delta\vec{R}$ 为随机步长，计算比值：

$$w = \frac{P(\vec{R}_p)}{P(\vec{R})} = \frac{|\phi(\vec{R}_p)|^2}{|\phi(\vec{R})|^2}$$

若 $w \geq 1$ ，总是接受；若 $w < 1$ ，以概率 w 接受，否则拒绝。

例：一维谐振子

$$H = -\frac{d^2}{dx^2} + \frac{1}{2}kx^2$$

若严格求解波函数：

$$\phi_0(x) = \frac{1}{\pi^{1/4}} e^{-x^2/2}$$

试探波函数：

$$\phi(x) = \frac{\sqrt{\alpha}}{\pi^{1/4}} e^{-x^2 \alpha^2 / 2}$$

$$E_L(x) = \alpha^2 + x^2(1 - \alpha^4)$$
$$\langle H \rangle = \int |\phi(x)|^2 E_L(x) dx = \frac{\alpha^2}{2} + \frac{1}{2\alpha^2}$$

$\alpha = 1$ 时达到最小值，得到基态能量 $E_0 = 1$ 。

注意此时解析积分的时候必须要将波函数归一化，但如果用蒙特卡洛算法，就无需进行归一化。

8.10 扩散蒙特卡洛 (DMC)

虚时投影 + 扩散分支，哈哈，自己看吧！

9 微分方程

微分方程分为两类：初值问题和边值问题。初值问题是指在某个时间点给定系统的状态，求解系统在未来时间的演化；边值问题是指在空间或时间的边界上给定系统的状态，求解系统在整个区域内的行为。我们先来考虑初值问题的求解。

9.1 欧拉法 (Euler's Method)

考虑一阶常微分方程：

$$\frac{dy}{dt} = f(t, y), \quad a \leq t \leq b$$

初值为

$$y(a) = \alpha$$

现在要求解 $y(t)$ 在 $[a, b]$ 上的近似值。

首先要离散化，将 $[a, b]$ 等分为 N 段，步长

$$h = \frac{b - a}{N}$$

离散点：

$$t_i = a + ih, \quad i = 0, 1, \dots, N$$

Taylor 展开：

$$y(t_{i+1}) = y(t_i) + hy'(t_i) + \frac{h^2}{2}y''(\xi_i)$$

由于

$$\begin{aligned} y'(t_i) &= f(t_i, y(t_i)) \\ \Rightarrow y(t_{i+1}) &= y(t_i) + hf(t_i, y(t_i)) + \frac{h^2}{2}y''(\xi_i) \end{aligned}$$

忽略高阶项，得到递推公式：

$$w_0 = \alpha, \quad w_{i+1} = w_i + hf(t_i, w_i), \quad i = 0, 1, \dots, N-1$$

9.2 高阶 Taylor 展开法

高阶 Taylor 展开法是欧拉法的推广，做 Taylor 展开：

$$\begin{aligned} y(t_{i+1}) &= y(t_i) + hy'(t_i) + \frac{h^2}{2!}y''(t_i) + \dots + \frac{h^n}{n!}y^{(n)}(t_i) + \frac{h^{n+1}}{(n+1)!}y^{(n+1)}(\xi_i) \\ &= y(t_i) + hf(t_i, y_i) + \frac{h^2}{2}f'(t_i, y_i) + \frac{h^3}{6}f''(t_i, y_i) + \dots + \frac{h^n}{n!}f^{(n-1)}(t_i, y_i) + \frac{h^{n+1}}{(n+1)!}f^{(n)}(\xi_i) \end{aligned}$$

定义：

$$\phi(t_i, w_i) = f(t_i, w_i) + \frac{h}{2}f'(t_i, w_i) + \dots + \frac{h^{n-1}}{n!}f^{(n-1)}(t_i, w_i)$$

递推公式为：

$$w_0 = \alpha, \quad w_{i+1} = w_i + h\phi(t_i, w_i), \quad i = 0, 1, \dots, N-1$$

定义局部截断误差：

$$\tau_{i+1}(h) = \frac{y(t_{i+1}) - y(t_i) - h\phi(t_i, y(t_i))}{h}$$

精确解：

$$y(t_{i+1}) = y(t_i) + h\phi(t_i, y(t_i)) + h\tau_{i+1}(h) + \frac{h^{n+1}}{(n+1)!}f^{(n)}(\xi_i, y(\xi_i))$$

因此

$$\tau_{i+1}(h) = \frac{h^n}{(n+1)!}f^{(n)}(\xi_i, y(\xi_i))$$

误差在 $O(h^n)$ 级别。

总之，我们可以把上述结论写成以下形式：

$$w_0 = \alpha, \quad w_{i+1} = w_i + hT^{(n)}(t_i, w_i), \quad i = 0, 1, \dots, N-1$$

$$T^{(n)}(t_i, w_i) = f(t_i, w_i) + \frac{h}{2}f'(t_i, w_i) + \dots + \frac{h^{n-1}}{n!}f^{(n-1)}(t_i, w_i)$$

例：求解常微分方程

$$y' = y - t^2 + 1, \quad 0 \leq t \leq 2, \quad y(0) = 0.5$$

$$f'(t, y(t)) = y' - 2t = y - t^2 - 2t + 1$$

取到二阶项：

$$T^{(2)}(t_i, w_i) = f(t_i, w_i) + \frac{h}{2}f'(t_i, w_i) = (1 + \frac{h}{2})(w_i - t_i^2 + 1) - ht_i$$

递推公式为：

$$w_{i+1} = w_i + hT^{(2)}(t_i, w_i)$$

代入 $h = 0.2, t_i = 0.2i$ ，得到

$$w_{i+1} = w_i + 0.2[(1 + 0.1)(w_i - 0.04i^2 + 1) - 0.2i] = 1.22w_i - 0.0088i^2 - 0.04i + 0.22$$

Taylor 展开的阶次越高，递推的精度也会越高。

Taylor Order 2			Taylor Order 4		
t_i	w_i	$ y(t_i) - w_i $	t_i	w_i	$ y(t_i) - w_i $
0.0	0.500000	0	0.0	0.500000	0
0.2	0.830000	0.000701	0.2	0.829300	0.000001
0.4	1.215800	0.001712	0.4	1.214091	0.000003
0.6	1.652076	0.003135	0.6	1.648947	0.000006
0.8	2.132333	0.005103	0.8	2.127240	0.000010
1.0	2.648646	0.007787	1.0	2.640874	0.000015
1.2	3.191348	0.011407	1.2	3.179964	0.000023
1.4	3.748645	0.016245	1.4	3.732432	0.000032
1.6	4.306146	0.022663	1.6	4.283529	0.000045
1.8	4.846299	0.031122	1.8	4.815238	0.000062
2.0	5.347684	0.042212	2.0	5.305555	0.000083

9.3 Runge-Kutta 方法

Runge-Kutta 方法的核心思想是用函数值的组合来近似泰勒级数以避免直接计算偏导数。

考虑二阶 Taylor 展开方法:

$$T^{(2)}(t, y) = f(t, y) + \frac{h}{2} f'(t, y)$$

其中 $f'(t, y)$ 可以写作:

$$f'(t, y) = \frac{\partial f}{\partial t} + \frac{\partial f}{\partial y} \cdot f$$

于是

$$T^{(2)}(t, y) = f + \frac{h}{2} \frac{\partial f}{\partial t} + \frac{h}{2} \frac{\partial f}{\partial y} \cdot f$$

9.3.1 中点法

Runge-Kutta 方法希望引入一个函数值的线性组合来近似上面的 Taylor 展开式, 比如:

$$a_1 f(t + \alpha_1, y + \beta_1)$$

Taylor 展开:

$$a_1 f(t + \alpha_1, y + \beta_1) = a_1 f + a_1 \alpha_1 \frac{\partial f}{\partial t} + a_1 \beta_1 \frac{\partial f}{\partial y} + R_1$$

其中 R_1 为高阶余项, 对比系数得到

$$a_1 = 1, \quad \alpha_1 = \frac{h}{2}, \quad \beta_1 = \frac{h}{2} f(t, y)$$

代入得到中点法的递推公式:

$$w_{n+1} = w_n + h \cdot f \left(t_n + \frac{h}{2}, w_n + \frac{h}{2} f(t_n, w_n) \right)$$

9.3.2 改进的欧拉法

还有一些其他的递推方法, 比如改进的欧拉法。

$$y''(t) = \frac{d}{dt} f(t, y) = f_t(t, y) + f_y(t, y) \cdot y'(f, t) = f_t + f_y f$$

$$\Rightarrow y(t + h) = y(t) + hf + \frac{h^2}{2} (f_t + f_y f) + O(h^3)$$

选取两个函数值的线性组合为增量函数 Φ :

$$\Phi = a_1 f(t, y) + a_2 f(t + \alpha h, y + \beta h f)$$

Taylor 展开:

$$f(t + \alpha h, y + \beta h f) = f + \alpha h f_t + \beta h f f_y + O(h^2)$$

可以取

$$\alpha_1 = \alpha_2 = \frac{1}{2}, \quad \alpha = \beta = 1$$

得到递推公式:

$$\begin{aligned}\Phi &= \frac{1}{2}f(t, y) + \frac{1}{2}f(t+h, y+hf) \\ w_{i+1} &= w_i + \frac{h}{2}[f(t_i, w_i) + f(t_i+h, w_i+hf(t_i, w_i))]\end{aligned}$$

9.3.3 三阶 Runge-Kutta 方法

考虑三阶 Taylor 展开:

$$y(t+h) = y(t) + hy'(t) + \frac{h^2}{2}y''(t) + \frac{h^3}{6}f'''(t) + O(h^4)$$

经过推导得到递推公式:

$$w_{i+1} = w_i + \frac{h}{4} \left(f(t_i, w_i) + 3f \left(t_i + \frac{2h}{3}, w_i + \frac{2h}{3}f \left(t_i + \frac{h}{3}, w_i + \frac{h}{3}f(t_i, w_i) \right) \right) \right)$$

9.3.4 四阶 Runge-Kutta 方法

递推公式为:

$$\begin{aligned}k_1 &= hf(t_i, w_i) \\ k_2 &= hf \left(t_i + \frac{h}{2}, w_i + \frac{1}{2}k_1 \right) \\ k_3 &= hf \left(t_i + \frac{h}{2}, w_i + \frac{1}{2}k_2 \right) \\ k_4 &= hf(t_i+h, w_i+k_3) \\ w_{i+1} &= w_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)\end{aligned}$$

9.3.5 Runge-Kutta-Fehlberg 方法

递推公式为:

$$\begin{aligned}k_1 &= hf(t_i, w_i) \\ k_2 &= hf \left(t_i + \frac{h}{4}, w_i + \frac{1}{4}k_1 \right) \\ k_3 &= hf \left(t_i + \frac{3h}{8}, w_i + \frac{3}{32}k_1 + \frac{9}{32}k_2 \right) \\ k_4 &= hf \left(t_i + \frac{12h}{13}, w_i + \frac{1932}{2197}k_1 - \frac{7200}{2197}k_2 + \frac{7296}{2197}k_3 \right)\end{aligned}$$

$$\begin{aligned}
k_5 &= hf \left(t_i + h, w_i + \frac{439}{216}k_1 - 8k_2 + \frac{3680}{513}k_3 - \frac{845}{4104}k_4 \right) \\
k_6 &= hf \left(t_i + \frac{h}{2}, w_i - \frac{8}{27}k_1 + 2k_2 - \frac{3544}{2565}k_3 + \frac{1859}{4104}k_4 - \frac{11}{40}k_5 \right) \\
w_{i+1} &= w_i + \frac{16}{135}k_1 + \frac{6656}{12825}k_3 + \frac{28561}{56430}k_4 - \frac{9}{50}k_5 + \frac{2}{55}k_6
\end{aligned}$$

9.4 多步法——Adams 预测-校正方法

此处只给出递推公式，就不作推导了。

9.4.1 Adams-Moulton 三阶隐式方法

$$w_{i+1} = w_i + \frac{h}{24}[9f(t_{i+1}, w_{i+1}) + 19f(t_i, w_i) - 5f(t_{i-1}, w_{i-1}) + f(t_{i-2}, w_{i-2})]$$

一般精度更高、稳定性更好，但是需要迭代求解。

9.4.2 Adams-Bashforth 三阶显式方法

$$w_{i+1} = w_i + \frac{h}{12}[23f(t_i, w_i) - 16f(t_{i-1}, w_{i-1}) + 5f(t_{i-2}, w_{i-2})]$$

9.4.3 Adams-Bashforth 四阶显式方法

$$w_{i+1} = w_i + \frac{h}{24}[55f(t_i, w_i) - 59f(t_{i-1}, w_{i-1}) + 37f(t_{i-2}, w_{i-2}) - 9f(t_{i-3}, w_{i-3})]$$

预测-校正：首先用四阶 Runge-Kutta 方法计算出 w_0, w_1, w_2, w_3 的值。再用四阶显式 Adams-Bashforth 方法预测下一个值 w'_4 ，再代入三阶隐式 Adams-Moulton 方法校正得到 w_4 ，这样可以提高精度。以此类推，得到整个区间的数值解。

9.5 扩展的四阶 Runge-Kutta 方法

扩展的四阶 Runge-Kutta 方法可以用来求解一阶常微分方程组。一阶常微分方程组可以写作：

$$\frac{dy_i}{dt} = f_i(t, y_1, y_2, \dots, y_m), \quad i = 1, 2, \dots, m$$

初始条件:

$$y_i(a) = \alpha_i$$

将递推写为向量形式即可:

$$\begin{aligned} k_{1i} &= hf_i(t_n, w_{1n}, w_{2n}, \dots, w_{mn}) \\ k_{2i} &= hf_i\left(t_n + \frac{h}{2}, w_{1n} + \frac{1}{2}k_{11}, w_{2n} + \frac{1}{2}k_{12}, \dots, w_{mn} + \frac{1}{2}k_{1n}\right) \\ k_{3i} &= hf_i\left(t_n + \frac{h}{2}, w_{1n} + \frac{1}{2}k_{21}, w_{2n} + \frac{1}{2}k_{22}, \dots, w_{mn} + \frac{1}{2}k_{2n}\right) \\ k_{4i} &= hf_i(t_n + h, w_{1n} + k_{31}, w_{2n} + k_{32}, \dots, w_{mn} + k_{3n}) \\ w_{i,n+1} &= w_{in} + \frac{1}{6}(k_{1i} + 2k_{2i} + 2k_{3i} + k_{4i}) \end{aligned}$$

9.6 高阶常微分方程

高阶常微分方程可以转化为一个一阶微分方程组, 然后再代入上面的方法求解。考虑 m 阶常微分方程:

$$y^{(m)}(t) = f(t, y, y', \dots, y^{(m-1)})$$

初始条件:

$$y(a) = \alpha_1, \quad y'(a) = \alpha_2, \quad \dots, \quad y^{(m-1)}(a) = \alpha_m$$

定义变量:

$$\begin{aligned} u_1(t) &= y(t), \quad u_2(t) = y'(t), \quad \dots, \quad u_m(t) = y^{(m-1)}(t) \\ \frac{du_k}{dt} &= u_{k+1}, \quad k = 1, 2, \dots, m-1 \end{aligned}$$

于是可以得到一阶微分方程组:

$$\begin{aligned} u_1' &= u_2, \quad u_1(a) = \alpha_1 \\ u_2' &= u_3, \quad u_2(a) = \alpha_2 \\ &\dots \\ u_{m-1}' &= u_m, \quad u_{m-1}(a) = \alpha_{m-1} \\ u_m' &= f(t, u_1, u_2, \dots, u_m), \quad u_m(a) = \alpha_m \end{aligned}$$

例: 求解二阶常微分方程

$$y'' - 2y' + 2y = e^{2t} \sin t, \quad 0 \leq t \leq 1$$

初始条件:

$$y(0) = -0.4, \quad y'(0) = -0.6$$

引入新变量:

$$u_1 = y, \quad u_2 = y'$$

得到一阶微分方程组:

$$u_1' = u_2, \quad u_1(0) = -0.4$$

$$u_2' = 2u_2 - 2u_1 + e^{2t} \sin t, \quad u_2(0) = -0.6$$

$$f_1(t, u_1, u_2) = u_2$$

$$f_2(t, u_1, u_2) = 2u_2 - 2u_1 + e^{2t} \sin t$$

设定步长 $h = 0.1$, 使用四阶 Runge-Kutta 方法:

$$k_{11} = hf_1(t_0, w_{10}, w_{20}) = -0.06$$

$$k_{12} = hf_2(t_0, w_{10}, w_{20}) = -0.04$$

$$k_{21} = hf_1\left(t_0 + \frac{h}{2}, w_{10} + \frac{1}{2}k_{11}, w_{20} + \frac{1}{2}k_{12}\right) = -0.062$$

$$k_{22} = hf_2\left(t_0 + \frac{h}{2}, w_{10} + \frac{1}{2}k_{11}, w_{20} + \frac{1}{2}k_{12}\right) = -0.03247644757$$

$$k_{31} = hf_1\left(t_0 + \frac{h}{2}, w_{10} + \frac{1}{2}k_{21}, w_{20} + \frac{1}{2}k_{22}\right) = -0.06162832238$$

$$k_{32} = hf_2\left(t_0 + \frac{h}{2}, w_{10} + \frac{1}{2}k_{21}, w_{20} + \frac{1}{2}k_{22}\right) = -0.03152409237$$

$$k_{41} = hf_1(t_0 + h, w_{10} + k_{31}, w_{20} + k_{32}) = -0.06315240924$$

$$k_{42} = hf_2(t_0 + h, w_{10} + k_{31}, w_{20} + k_{32}) = -0.02178637298$$

$$w_{11} = w_{10} + \frac{1}{6}(k_{11} + 2k_{21} + 2k_{31} + k_{41}) = -0.4617333423$$

$$w_{21} = w_{20} + \frac{1}{6}(k_{12} + 2k_{22} + 2k_{32} + k_{42}) = -0.06316312421$$

9.7 刚性微分方程

刚性方程指的是不同模式的时间尺度差别巨大的微分方程。例如，解中包含快速衰减项和慢速变化项。如果要准确捕捉快变部分，数值方法的步长必须非常小。此时，显式方法（如欧拉法、Runge-Kutta 方法）可能会因为数值不稳定而失败。隐式方法（如后向欧拉法、Adams-Moulton 方法）通常更适合处理刚性方程，因为它们具有更好的稳定性特性。例如二维线性系统：

$$\begin{aligned} u_1' &= 9u_1 + 24u_2 + 5 \cos t - \frac{1}{3} \sin t \\ u_2' &= -24u_1 - 51u_2 - 9 \cos t + \frac{1}{3} \sin t \end{aligned}$$

初始条件：

$$u_1(0) = \frac{4}{3}, \quad u_2(0) = \frac{2}{3}$$

精确解为：

$$\begin{aligned} u_1(t) &= 2e^{-3t} - e^{-39t} + \frac{1}{3} \cos t \\ u_2(t) &= -e^{-3t} + 2e^{-39t} - \frac{1}{3} \cos t \end{aligned}$$

有两个指数衰减项。

再例如标量测试方程：

$$\begin{aligned} y' &= -30y, \quad 0 \leq t \leq 1.5 \\ y(0) &= \frac{1}{3} \end{aligned}$$

精确解为：

$$\begin{aligned} y(t) &= \frac{1}{3} e^{-30t} \\ y(1.5) &= 9.54 \times 10^{-21} \end{aligned}$$

取步长 $h = 0.1$ ，显式方法求解得到的结果会产生巨大误差。

方法	计算结果 $y(1.5)$
Exact solution	9.54173×10^{-21}
Euler's method	-1.09225×10^4
Runge-Kutta method	3.95730×10^1
Predictor-corrector method	8.03840×10^5

表 5: 刚性方程不同数值方法的结果比较（步长 $h = 0.1$ ， $t = 1.5$ ）

下面考虑如何通过隐式方法来求解刚性方程。

9.7.1 隐式梯形法

考虑初值问题：

$$y' = f(t, y), \quad y(t_j) = y_j$$

将方程从 t_j 到 t_{j+1} 积分：

$$y(t_{j+1}) - y(t_j) = \int_{t_j}^{t_{j+1}} f(t, y) dt$$

用梯形公式近似：

$$\begin{aligned} \int_{t_j}^{t_{j+1}} f(t, y) dt &\approx \frac{h}{2} [f(t_j, y_j) + f(t_{j+1}, y_{j+1})] \\ \Rightarrow y_{j+1} &= y_j + \frac{h}{2} [f(t_j, y_j) + f(t_{j+1}, y_{j+1})] \end{aligned}$$

得到隐式梯形法的递推公式：

$$w_{j+1} = w_j + \frac{h}{2} [f(t_j, w_j) + f(t_{j+1}, w_{j+1})]$$

下面考虑如何求解 w_{j+1} ，定义函数：

$$F(w) = w - w_j - \frac{h}{2} [f(t_j, w_j) + f(t_{j+1}, w)]$$

求解 $F(w) = 0$ ，可以使用牛顿迭代法：

$$\begin{aligned} w_{j+1}^{(k)} &= w_{j+1}^{(k-1)} - \frac{F(w_{j+1}^{(k-1)})}{F'(w_{j+1}^{(k-1)})} \\ F'(w) &= 1 - \frac{h}{2} f_w(t_{j+1}, w) \\ \Rightarrow w_{j+1}^{(k)} &= w_{j+1}^{(k-1)} - \frac{w_{j+1}^{(k-1)} - w_j - \frac{h}{2} [f(t_j, w_j) + f(t_{j+1}, w_{j+1}^{(k-1)})]}{1 - \frac{h}{2} f_w(t_{j+1}, w_{j+1}^{(k-1)})} \end{aligned}$$

9.7.2 Gear 方法

Gear 方法是一类线性多步法，其一般形式为

$$w_{n+k} = - \sum_{j=0}^{k-1} a_j w_{n+j} + h b f(t_{n+k}, w_{n+k})$$

其中 a_j, b 是已知系数。

二阶 Gear 方法：

$$w_{n+2} = \frac{4w_{n+1} - w_n + 2hf(t_{n+2}, w_{n+2})}{3}$$

三阶 Gear 方法：

$$w_{n+3} = \frac{18w_{n+2} - 9w_{n+1} + 2w_n + 6hf_{t_{n+3}, w_{n+3}}}{11}$$

再利用牛顿法迭代求解即可。

9.8 显性与隐性欧拉法的稳定性分析

先考虑模型方程：

$$y' = -cy (c > 0)$$

精确解为

$$y(t) = y_0 e^{-ct}$$

对于显性欧拉法：

$$y_{n+1} = y_n + hy'_n = (1 - ch)y_n$$

$$\Rightarrow y_n = (1 - ch)^n y_0$$

如果想要收敛，就必须要求

$$0 < ch < 2$$

刚性方程 c 很大，这就要求步长 h 必须特别小，否则就会不稳定。

对于隐性欧拉法：

$$y_{n+1} = y_n + hy'_{n+1} = y_n - chy_{n+1}$$

$$y_{n+1} = \frac{y_n}{1 + ch}$$

$$\Rightarrow y_n = \frac{y_0}{(1 + ch)^n}$$

无论 ch 多大，都能保证数值解的稳定性，更适合刚性方程。

推广到线性方程组：

$$\vec{y}' = -\mathbf{C}\vec{y}$$

其中 $\mathbf{C}$ 为常数矩阵，假设矩阵 $\mathbf{C}$ 可对角化，则存在可逆矩阵 $\mathbf{T}$ 使得

$$\mathbf{TCT}^{-1} = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_m)$$

引入变量

$$\vec{z} = \mathbf{T}^{-1}\vec{y}$$

$$\Rightarrow \vec{z}' = -\text{diag}(\lambda_1, \lambda_2, \dots, \lambda_m)\vec{z}$$

$$z_i(t) = z_i(0)e^{-\lambda_i t}$$

回代得到

$$\vec{y}(t) = \mathbf{T} \cdot \text{diag}(e^{-\lambda_1 t}, e^{-\lambda_2 t}, \dots, e^{-\lambda_m t}) \cdot \mathbf{T}^{-1}\vec{y}(0)$$

线性系统的稳定性由特征值 λ_i 决定，如果所有 λ_i 的实部为正，系统是稳定的。刚性系统的特征值实部相差巨大，显式方法步长受到最大实部限制，而隐式方法可以大步长求解。

对于显式欧拉法：

$$\vec{y}_{n+1} = (\mathbf{I} - h\mathbf{C})\vec{y}_n$$

特征向量展开：

$$\begin{aligned}\vec{y}_0 &= \sum_{i=1}^m a_i \vec{\xi}_i \\ \mathbf{C}\vec{\xi}_i &= \lambda_i \vec{\xi}_i \\ \Rightarrow \vec{y}_n &= \sum_{i=1}^m a_i (1 - h\lambda_i)^n \vec{\xi}_i\end{aligned}$$

数值收敛要求

$$h < \frac{2}{\lambda_{\max}}$$

步长受限于最大特征值，这是显式方法是稳定性限制。

对于隐式欧拉法：

$$\begin{aligned}\vec{y}_{n+1} &= \vec{y}_n + h\vec{y}'_{n+1} \\ \Rightarrow \vec{y}_{n+1} &= (\mathbf{I} + h\mathbf{C})^{-1}\vec{y}_n\end{aligned}$$

要求

$$\frac{1}{1 + h\lambda_i} < 1$$

隐式方法无条件稳定，优于显式方法。

9.9 线性打靶法

下面来考虑微分方程的边值问题考虑二阶线性常微分方程的边值问题：

$$y'' = p(x)y' + q(x)y + r(x), \quad a \leq x \leq b$$

边界条件：

$$y(a) = \alpha, \quad y(b) = \beta$$

线性打靶法的思想是将边值问题转化为初值问题，我们不知道在 $x = a$ 点处的导数 $y'(a)$ ，但是可以猜测一个 t ，然后求解初值问题，得到方程在 $x = b$ 处的解 $y(b, t)$ ，调整 t 使得 $y(b, t) = \beta$ 。由于方程是线性的，可以利用叠加原理，通过两个特殊的初值问题来构造解。构造两个辅助初值问题：

$$y_1'' = p(x)y_1' + q(x)y_1 + r(x), \quad y_1(a) = \alpha, \quad y_1'(a) = 0$$

$$y_2'' = p(x)y_2' + q(x)y_2, \quad y_2(a) = 0, \quad y_2'(a) = 1$$

得到两个解 $y_1(x)$ 和 $y_2(x)$ ，则原问题的解可以表示为

$$y(x) = y_1(x) + Cy_2(x)$$

其中常数 C 由右端边界条件确定：

$$\begin{aligned} y(b) &= y_1(b) + Cy_2(b) = \beta \\ \Rightarrow C &= \frac{\beta - y_1(b)}{y_2(b)} \end{aligned}$$

于是得到解：

$$\begin{aligned} y(x) &= y_1(x) + \frac{\beta - y_1(b)}{y_2(b)}y_2(x) \\ y(a) &= y_1(a) + \frac{\beta - y_1(b)}{y_2(b)}y_2(a) = \alpha \\ y(b) &= y_1(b) + \frac{\beta - y_1(b)}{y_2(b)}y_2(b) = \beta \end{aligned}$$

9.10 非线性打靶法

对于二阶边值问题：

$$y'' = f(x, y, y'), \quad a \leq x \leq b$$

边界条件：

$$y(a) = \alpha, \quad y(b) = \beta$$

我们不知道在 $x = a$ 处的导数 $y'(a)$ ，但可以将其设为一个待定参数 t 。于是转化为一个以 t 为参数的初值问题：

$$y'' = f(x, y, y'), \quad y(a) = \alpha, \quad y'(a) = t$$

对于每一个给定的 t ，可以通过数值方法得到 $y(b, t)$ ，找到合适的 t 使得

$$F(t) = y(b, t) - \beta = 0$$

于是问题转化为关于 t 的非线性方程求根问题，可以用割线法、牛顿法等方法来迭代求解：

$$t_k = t_{k-1} - \frac{(y(b, t_{k-1}) - \beta)(t_{k-1} - t_{k-2})}{y(b, t_{k-1}) - y(b, t_{k-2})}$$

或者

$$t_k = t_{k-1} - \frac{y(b, t_{k-1}) - \beta}{\left. \frac{\partial y(b, t)}{\partial t} \right|_{t=t_{k-1}}}$$

对于牛顿法，需要把微分求解出来，下面考虑如何计算

$$\frac{\partial y}{\partial t}(b, t)$$

已知原方程：

$$y'' = f(x, y, y')$$

对 t 求导：

$$\frac{\partial y''}{\partial t} = \frac{\partial f}{\partial x} \frac{\partial x}{\partial t} + \frac{\partial f}{\partial y} \frac{\partial y}{\partial t} + \frac{\partial f}{\partial y'} \frac{\partial y'}{\partial t} = \frac{\partial f}{\partial y} \frac{\partial y}{\partial t} + \frac{\partial f}{\partial y'} \frac{\partial y'}{\partial t}$$

定义：

$$\begin{aligned} z(x, t) &= \frac{\partial y}{\partial t}(x, t) \\ \Rightarrow z' &= \frac{\partial y'}{\partial t}, \quad z'' = \frac{\partial y''}{\partial t} \end{aligned}$$

得到新的线性 ODE：

$$z'' = \frac{\partial f}{\partial y} z + \frac{\partial f}{\partial y'} z'$$

边界条件：

$$z(a) = 0, \quad z'(a) = 1$$

于是牛顿法迭代可以写作：

$$t_k = t_{k-1} - \frac{y(b, t_{k-1}) - \beta}{z(b, t_{k-1})}$$

同时求解出 $z(b, t)$ 和 $y(b, t)$ ，即可得到 t_k 。由此也可以看到，边值问题和初值问题是可以相互转化的。

9.11 Numerov 方法

Numerov 方法适用于如下形式的方程：

$$y''(x) = f(x)y(x)$$

Taylor 展开：

$$y(x+h) = y(x) + hy'(x) + \frac{h^2}{2}y''(x) + \frac{h^3}{6}y'''(x) + \frac{h^4}{24}y^{(4)}(x) + \frac{h^5}{120}y^{(5)}(x) + \frac{h^6}{720}y^{(6)}(x) + O(h^7)$$

$$y(x-h) = y(x) - hy'(x) + \frac{h^2}{2}y''(x) - \frac{h^3}{6}y'''(x) + \frac{h^4}{24}y^{(4)}(x) - \frac{h^5}{120}y^{(5)}(x) + \frac{h^6}{720}y^{(6)}(x) + O(h^7)$$

两式相加得到：

$$y(x+h) + y(x-h) = 2y(x) + h^2y''(x) + \frac{h^4}{12}y^{(4)}(x) + \frac{h^6}{360}y^{(6)}(x) + O(h^8)$$

引入辅助函数：

$$w(x) = \left(1 - \frac{h^2}{12}f(x)\right) y(x)$$

近似：

$$y^{(4)}(x) = \frac{y''(x+h) - 2y''(x) + y''(x-h)}{h^2}$$

得到递推公式：

$$w(x+h) + w(x-h) - 2w(x) = h^2 f(x)y(x) + O(h^6)$$

起点公式：

$$2hy'(0) = \left[1 - \frac{h^2 f(h)}{6}\right] y(h) - \left[1 - \frac{h^2 f(-h)}{6}\right] y(-h) + O(h^5)$$

可以将 $y(-h)$ 消掉，得到

$$\begin{aligned} x(h) &= \frac{A}{B} \\ A &= \left[2 + \frac{5h^2 f(0)}{6}\right] \left[1 - \frac{h^2 f(-h)}{12}\right] y(0) + 2hy'(0) \left[1 - \frac{h^2 f(-h)}{6}\right] \\ B &= \left[1 - \frac{h^2 f(h)}{12}\right] \left[1 - \frac{h^2 f(-h)}{6}\right] + \left[1 - \frac{h^2 f(-h)}{12}\right] \left[1 - \frac{h^2 f(h)}{6}\right] \end{aligned}$$

Numerov 方法也可以用来求解形如

$$y''(x) + k(x)y(x) = F(x), \quad a \leq x \leq b$$

的方程，边界条件：

$$y(a) = \alpha, \quad y(b) = \beta$$

二阶导数可以写作：

$$y''(x) = \frac{y(x+h) + y(x-h) - 2y(x)}{h^2} - \frac{h^2}{12}y^{(4)}(x) + O(h^4)$$

已知

$$\begin{aligned} y''(x) &= -k(x)y(x) + F(x) \\ \Rightarrow y^{(4)}(x) &= -\frac{d^2}{dx^2}(ky - F) \end{aligned}$$

中心差分公式近似：

$$\left. \frac{d^2}{dx^2}(ky - F) \right|_{x_i} \approx \frac{(k_{i+1}y_{i+1} - F_{i+1}) - 2(k_i y_i - F_i) + (k_{i-1}y_{i-1} - F_{i-1})}{h^2}$$

代入得

$$\frac{y_{i+1} - 2y_i + y_{i-1}}{h^2} + \frac{h^2}{12} \left[\frac{(k_{i+1}y_{i+1} - F_{i+1}) - 2(k_i y_i - F_i) + (k_{i-1}y_{i-1} - F_{i-1})}{h^2} \right] = -k_i y_i + F_i$$

整理得到递推公式：

$$\left(1 + \frac{h^2}{12}k_{i+1}\right) y_{i+1} - 2\left(1 - \frac{5h^2}{12}k_i\right) y_i + \left(1 + \frac{h^2}{12}k_{i-1}\right) y_{i-1} = \frac{h^2}{12} (F_{i+1} + 10F_i + F_{i-1}) + O(h^6)$$

9.12 Green 函数法

考虑一维 Poisson 方程:

$$-u''(x) = f(x), \quad 0 \leq x \leq 1$$

边界条件:

$$u(0) = u(1) = 0$$

已知不定积分:

$$\begin{aligned} u(x) &= c_1 + \int_0^x u'(y) dy \\ u'(y) &= c_2 + \int_0^y u''(z) dz = c_2 - \int_0^y f(z) dz \\ \Rightarrow u(x) &= c_1 + c_2 x - \int_0^x \int_0^y f(z) dz dy \end{aligned}$$

令

$$F(y) = \int_0^y f(z) dz$$

原式可写作

$$\begin{aligned} \int_0^x F(y) dy &= \int_0^x \int_0^y f(z) dz dy = \int_0^x \left(\int_z^x dy \right) f(z) dz = \int_0^x (x - z) f(z) dz \\ \Rightarrow u(x) &= c_1 + c_2 x - \int_0^x (x - y) f(y) dy \end{aligned}$$

代入边界条件:

$$\begin{aligned} u(0) = 0 &\Rightarrow c_1 = 0 \\ u(1) = 0 &\Rightarrow c_2 = \int_0^1 (1 - y) f(y) dy \end{aligned}$$

于是

$$u(x) = x \int_0^1 (1 - y) f(y) dy - \int_0^x (x - y) f(y) dy$$

引入 Green 函数:

$$G(x, y) = \begin{cases} x(1 - y), & 0 \leq x \leq y \leq 1 \\ y(1 - x), & 0 \leq y \leq x \leq 1 \end{cases}$$

可以写作:

$$u(x) = \int_0^1 G(x, y) f(y) dy$$

Green 函数的性质:

(1) 连续性: $G(x, y)$ 在 $x = y$ 处连续 (但导数不连续);

- (2) 对称性: $G(x, y) = G(y, x)$;
 (3) 边界条件: $G(0, y) = G(1, y) = 0$;
 (4) 分段线性;
 (5) 非负性: $G(x, y) \geq 0, \forall x, y \in [0, 1]$;
 (6) 满足微分方程:

$$\frac{d^2 G(x, y)}{dx^2} = -\delta(x - y)$$

下面考虑 Green 函数的一般构造, Green 函数满足点源相应:

$$\frac{d^2 G(x, y)}{dx^2} = -\delta(x - y)$$

对于齐次方程:

$$\frac{d^2 u}{dx^2} = 0$$

两个线性无关解为

$$u_1(x) = x, \quad u_2(x) = 1 - x$$

满足导数跳跃条件:

$$\left. \frac{dG}{dx} \right|_{x=y^+} - \left. \frac{dG}{dx} \right|_{x=y^-} = -1$$

于是可以设

$$G(x, y) = \begin{cases} A(y)x, & 0 \leq x \leq y \leq 1 \\ B(y)(1 - x), & 0 \leq y \leq x \leq 1 \end{cases}$$

代入连续性和跳跃条件:

$$G(x, y) = \begin{cases} x(1 - y), & 0 \leq x \leq y \leq 1 \\ y(1 - x), & 0 \leq y \leq x \leq 1 \end{cases}$$

解为

$$u(x) = \int_0^1 G(x, y) f(y) dy$$

考虑二阶线性非齐次微分方程:

$$y''(x) + a_1(x)y'(x) + a_0(x)y(x) = f(x), \quad a \leq x \leq b$$

边界条件:

$$F_1(a) = 0, \quad F_2(b) = 0$$

考虑对应的齐次微分方程:

$$y''(x) + a_1(x)y'(x) + a_0(x)y(x) = 0$$

设 $u_1(x)$ 和 $u_2(x)$ 是齐次方程的两个线性无关解, 满足边界条件:

$$F_1(a) = 0, \quad F_2(b) = 0$$

特解可以写作:

$$y(x) = c_1(x)y_1(x) + c_2(x)y_2(x)$$

经过推导可以得到

$$y(x) = \int_a^x \frac{y_1(\xi)y_2(x)}{W(\xi)} f(\xi) d\xi + \int_x^b \frac{y_1(x)y_2(\xi)}{W(\xi)} f(\xi) d\xi$$

其中 $W(\xi)$ 为 Wronskian 行列式:

$$W(\xi) = \begin{vmatrix} y_1(\xi) & y_2(\xi) \\ y_1'(\xi) & y_2'(\xi) \end{vmatrix} = y_1(\xi)y_2'(\xi) - y_1'(\xi)y_2(\xi)$$

定义 Green 函数:

$$G(x, \xi) = \begin{cases} \frac{y_1(\xi)y_2(x)}{W(\xi)}, & a \leq \xi \leq x \leq b \\ \frac{y_1(x)y_2(\xi)}{W(\xi)}, & a \leq x \leq \xi \leq b \end{cases}$$

方程的解可以写作:

$$y(x) = \int_a^b G(x, \xi) f(\xi) d\xi$$

9.13 有限差分法

考虑微分方程:

$$y''(x) + p(x)y'(x) + q(x)y(x) = f(x), \quad a \leq x \leq b$$

边界条件:

$$y(a) = \alpha, \quad y(b) = \beta$$

离散化:

$$h = \frac{b-a}{N}, \quad x_n = a + nh$$

记作

$$y_n = y(x_n), \quad p_n = p(x_n), \quad q_n = q(x_n), \quad r_n = r(x_n)$$

中心差分公式:

$$y'(x_n) = \frac{y_{n+1} - y_{n-1}}{2h} + O(h^2)$$

$$y''(x_n) = \frac{y_{n+1} - 2y_n + y_{n-1}}{h^2} + O(h^2)$$

代入微分方程:

$$\frac{y_{n+1} - 2y_n + y_{n-1}}{h^2} + p_n \frac{y_{n+1} - y_{n-1}}{2h} + q_n y_n = r_n$$

$$\left(\frac{1}{h^2} - \frac{p_n}{2h}\right) y_{n-1} + \left(-\frac{2}{h^2} + q_n\right) y_n + \left(\frac{1}{h^2} + \frac{p_n}{2h}\right) y_{n+1} = r_n$$

定义

$$l_i = \frac{1}{h^2} - \frac{p_i}{2h}$$

$$d_i = -\frac{2}{h^2} + q_i$$

$$u_i = \frac{1}{h^2} + \frac{p_i}{2h}$$

方程变为:

$$l_i y_{i-1} + d_i y_i + u_i y_{i+1} = r_i$$

边界条件:

$$y_0 = \alpha, \quad y_N = \beta$$

得到线性方程组:

$$\begin{pmatrix} d_1 & u_1 & 0 & \cdots & 0 \\ l_2 & d_2 & u_2 & \cdots & 0 \\ 0 & l_3 & d_3 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & u_{N-2} \\ 0 & 0 & 0 & \cdots & d_{N-1} \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{N-2} \\ y_{N-1} \end{pmatrix} = \begin{pmatrix} r_1 - l_1 \alpha \\ r_2 \\ r_3 \\ \vdots \\ r_{N-2} \\ r_{N-1} - u_{N-1} \beta \end{pmatrix}$$

10 偏微分方程

10.1 有限差分法

考虑二维 Poisson 方程:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = f(x, y), \quad (x, y) \in \Omega$$

将网格离散化, 步长为 h_x, h_y , 网格点为 (x_i, y_i) , 记:

$$u_{ij} = u(x_i, y_i), \quad f_{ij} = f(x_i, y_i)$$

二阶偏导数的中心差分公式:

$$u_{xx}(x_i, y_i) \approx \frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{h_x^2}$$

$$u_{yy}(x_i, y_i) \approx \frac{u_{i,j-1} - 2u_{i,j} + u_{i,j+1}}{h_y^2}$$

代入方程:

$$\frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{h_x^2} + \frac{u_{i,j-1} - 2u_{i,j} + u_{i,j+1}}{h_y^2} = f_{ij}$$

整理系数:

$$\frac{u_{i-1,j}}{h_x^2} + \frac{u_{i+1,j}}{h_x^2} + \frac{u_{i,j-1}}{h_y^2} + \frac{u_{i,j+1}}{h_y^2} - \left(\frac{2}{h_x^2} + \frac{2}{h_y^2} \right) u_{i,j} = f_{ij}$$

$$i = 1, 2, 3, \dots, m-1, \quad j = 1, 2, 3, \dots, n-1$$

如果选取 $h_x = h_y = h$, 则方程可以写作:

$$u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1} - 4u_{ij} = h^2 f_{ij}$$

这构成了一个线性方程组:

$$\mathbf{A}\mathbf{U} = \mathbf{F}$$

可以将二维编号投影到一维:

$$k = i + (m-1)(j-1)$$

例如一个 4×4 的二维网格, 只考虑内部的 3×3 的内部点, k 的顺序为:

$$j = 1 : x_1 = u_{11}, x_2 = u_{21}, x_3 = u_{31}$$

$$j = 2 : x_4 = u_{12}, x_5 = u_{22}, x_6 = u_{32}$$

$$j = 3 : x_7 = u_{13}, x_8 = u_{23}, x_9 = u_{33}$$

线性方程组可以写作:

$$\frac{1}{h^2} \begin{pmatrix} -4 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & -4 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & -4 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & -4 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & -4 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & -4 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & -4 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & -4 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & -4 \end{pmatrix} \begin{pmatrix} u_{11} \\ u_{21} \\ u_{31} \\ u_{12} \\ u_{22} \\ u_{32} \\ u_{13} \\ u_{23} \\ u_{33} \end{pmatrix} = \begin{pmatrix} f_{11} \\ f_{21} \\ f_{31} \\ f_{12} \\ f_{22} \\ f_{32} \\ f_{13} \\ f_{23} \\ f_{33} \end{pmatrix}$$

或者把 9×9 矩阵写作:

$$\mathbf{A} = \frac{1}{h^2} \begin{pmatrix} \mathbf{B} & \mathbf{I} & 0 \\ \mathbf{I} & \mathbf{B} & \mathbf{I} \\ 0 & \mathbf{I} & \mathbf{B} \end{pmatrix}$$

其中

$$\mathbf{B} = \begin{pmatrix} -4 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & -4 \end{pmatrix}$$

再考虑边界点的处理：如果是 Dirichlet 边界条件，直接将边界值代入即可，体现在矩阵右边的 f 上。

如果是 Neumann 边界条件，就需要更新边界点的求解。在左边界 $x = x_0$ 上给定导数：

$$\left. \frac{\partial u}{\partial x} \right|_{x_0} = g(y_j)$$

中心差分近似：

$$\begin{aligned} \frac{u_{1j} - u_{-1,j}}{2h} &= g_j \\ \Rightarrow u_{-1,j} &= u_{1j} - 2hg_j \end{aligned}$$

在边界点处，Poisson 方程为：

$$\begin{aligned} \frac{u_{-1,j} - 2u_{0j} + u_{1j}}{h^2} + \frac{u_{0,j-1} - 2u_{0j} + u_{0,j+1}}{h^2} &= f_{0j} \\ \Rightarrow (u_{1j} - 2hg_j) + u_{1j} + u_{0,j-1} + u_{0,j+1} - 4u_{0j} &= h^2 f_{0j} \end{aligned}$$

再利用 Jacobi 迭代等方法求解方程组即可。

例：考虑二维 Poisson 方程

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 2 \cos \pi y - \pi^2 (1+x)^2 \cos \pi y, \quad (x, y) \in [0, 1] \times [0, 1]$$

边界条件：

$$\begin{aligned} u(x, 0) &= (1+x)^2, \quad u(x, 1) = -(1+x)^2 \\ u(0, y) &= \cos \pi y, \quad u(1, y) = 4 \cos \pi y \end{aligned}$$

解析解为

$$u(x, y) = (1+x)^2 \cos \pi y$$

选取网格步长 $h = \frac{1}{4}$ ，节点坐标

$$x_i = \frac{i}{4}, \quad y_j = \frac{j}{4}, \quad i, j = 0, 1, 2, 3, 4$$

内部节点为 3×3 共 9 个节点。

内部节点：

$$-u_{i-1,j} + 4u_{ij} - u_{i+1,j} - u_{i,j-1} - u_{i,j+1} = h^2 f_{ij}$$

边界节点：

$$\begin{aligned}
-u_{01} + 4u_{11} - u_{21} - u_{10} - u_{12} &= h^2 f_{11}, \quad i=1, j=1 \\
\Rightarrow 4u_{11} - u_{21} - u_{12} &= h^2 f_{11} + \cos \frac{\pi}{4} + \left(1 + \frac{1}{4}\right)^2 \\
-u_{11} + 4u_{21} - u_{31} - u_{20} - u_{22} &= h^2 f_{21}, \quad i=2, j=1 \\
\Rightarrow -u_{11} + 4u_{21} - u_{31} - u_{22} &= h^2 f_{21} + \left(1 + \frac{1}{2}\right)^2 \\
-u_{21} + 4u_{31} - u_{41} - u_{30} - u_{32} &= h^2 f_{31}, \quad i=3, j=1 \\
\Rightarrow -u_{21} + 4u_{31} - u_{32} &= h^2 f_{31} + 4 \cos \frac{\pi}{4} + \left(1 + \frac{3}{4}\right)^2
\end{aligned}$$

在实际计算中，可以采用更高阶的微分公式：

$$\frac{\partial^2 u}{\partial x^2} = \frac{-u_{i+2} + 16u_{i+1} + 30u_i - 16u_{i-1} + u_{i-2}}{12h^2} + O(h^4)$$

10.2 Fourier 变换—Fourier 微分算符

Fourier 变换可以用来数值求解导数。在周期性边界条件下，可以把函数的导数计算从实空间转换到傅里叶空间以简便计算。

考虑一维等间距格点坐标：

$$x_i = -\frac{L}{2} + i \cdot dx = \left(i - \frac{N}{2}\right) dx, \quad \nu = 0, 1, 2, \dots, N-1$$

网格点均匀分布在 $[-L/2, L/2)$ 的区间，其中 $L = Ndx$ 。

满足周期性边界条件：

$$f(x+L) = f(x)$$

FFT 中的经典波数排列为：

$$k_n = \frac{2\pi n}{L}, \quad n = 0, 1, 2, \dots, N-1$$

利用周期性，可以将 $n > N/2$ 的波数解释为负频率：

$$k_n = \begin{cases} \frac{2\pi n}{L}, & n = 0, 1, 2, \dots, N/2 \\ \frac{2\pi(n-N)}{L}, & n = N/2 + 1, N/2 + 2, \dots, N-1 \end{cases}$$

$$dk = \frac{2\pi}{L}$$

离散 Fourier 变换 (DFT) 为:

$$\hat{f}(k_n) = \sum_{i=0}^{N-1} e^{-ik_n x_i} f(x_i)$$

逆变换:

$$f(x_i) = \frac{1}{N} \sum_{n=0}^{N-1} e^{ik_n x_i} \hat{f}(k_n)$$

对于任意函数 $f(x)$, 其 m 阶导数的 Fourier 变换为

$$\widehat{f^{(m)}}(k_n) = (ik_n)^m \hat{f}(k_n)$$

因此, 如果要求解 $f^{(m)}(x_i)$, 可以先对 $f(x_i)$ 进行 Fourier 变换, 再乘上 $(ik_n)^m$, 再逆变换回来, 就得到了 $f^{(m)}(x_i)$ 的数值近似。

$$\begin{aligned} f^{(m)}(x_i) &= \frac{1}{N} \sum_{n=0}^{N-1} e^{ik_n x_i} (ik_n)^m \sum_{j=0}^{N-1} e^{-ik_n x_j} f(x_j) \\ &= \sum_{j=0}^{N-1} \left[\frac{1}{N} \sum_{n=0}^{N-1} e^{ik_n (x_i - x_j)} (ik_n)^m \right] f(x_j) = \sum_{j=0}^{N-1} D_{ij}^{(m)} f(x_j) \end{aligned}$$

考虑一阶微分矩阵元:

$$\begin{aligned} D_{ij}^{(1)} &= \frac{1}{N} \sum_{n=0}^{N-1} e^{ik_n (x_i - x_j)} \cdot ik_n \\ &= \frac{1}{N} \sum_{n=0}^{N/2-1} e^{ik_n h(i-j)} \cdot \frac{i2\pi n}{L} + \frac{1}{N} \sum_{n=N/2+1}^{N-1} e^{ik_n h(i-j)} \cdot \frac{i2\pi(n-N)}{L} + \frac{1}{N} e^{i\pi N h(i-j)/L} \cdot \frac{i\pi N}{L} \end{aligned}$$

$D_{ij}^{(1)}$ 应该是实数, 直接取实部:

$$\begin{aligned} \text{Re}(D_{ij}^{(1)}) &= -\frac{1}{N} \sum_{n=0}^{N/2-1} \sin\left(\frac{2\pi n h}{L}(i-j)\right) \cdot \frac{2\pi n}{L} \\ &\quad - \frac{1}{N} \sum_{n=N/2+1}^{N-1} \sin\left(\frac{2\pi(n-N)h}{L}(i-j)\right) \cdot \frac{2\pi(n-N)}{L} - \frac{1}{N} \sin\left(\frac{\pi N h}{L}(i-j)\right) \cdot \frac{\pi N}{L} \\ &= -\frac{2}{N} \sum_{n=0}^{N/2-1} \sin\left(\frac{2\pi n h}{L}(i-j)\right) \cdot \frac{2\pi n}{L} - \frac{\pi}{L} \sin\left(\frac{\pi N h}{L}(i-j)\right) \\ &= -\frac{2dk}{N} \sum_{n=1}^{N/2-1} n \sin(k_n h(i-j)) - \frac{dk}{2} \sin\left(\frac{Ndkh}{2}(i-j)\right) \end{aligned}$$

同理可以写出二阶微分矩阵元：

$$D_{ij}^{(2)} = \frac{1}{N} \sum_{n=0}^{N-1} e^{ik_n(x_i - x_j)} (-k_n^2) \\ = -\frac{2dk^2}{N} \sum_{n=1}^{N/2-1} n^2 \cos(k_n h(i-j)) - \frac{dk^2}{N} \left(\frac{N}{2}\right)^2 \cos\left(\frac{Ndkh}{2}(i-j)\right)$$

还可以推广到二维情形：

$$\frac{\partial \varphi}{\partial x}(x_i, y_j) = \sum_{k=0}^{N-1} D_{ik}^{(1)} \varphi(x_k, y_j) \\ \frac{\partial \varphi}{\partial y}(x_i, y_j) = \sum_{k=0}^{N-1} D_{jk}^{(1)} \varphi(x_i, y_k)$$

10.3 基矢展开法

基矢展开法常用于求解薛定谔方程，它的思想是将未知波函数用一组正交完备基展开，从而把微分方程转化为矩阵的本征值问题，通过对角化得到本征值和展开系数。考虑波函数，用正交完备基展开：

$$|\psi\rangle = \sum_j c_j |\psi_j\rangle$$

其中 $\{|\psi_j\rangle\}$ 是一组正交完备基， c_j 为展开系数。

代入薛定谔方程：

$$\hat{H}|\psi\rangle = E|\psi\rangle$$

左乘 $\langle\psi_i|$ ：

$$\sum_j c_j \langle\psi_i|\hat{H}|\psi_j\rangle = E \langle\psi_i|\psi\rangle = E c_i$$

定义矩阵元：

$$H_{ij} = \langle\psi_i|\hat{H}|\psi_j\rangle$$

得到矩阵方程：

$$\sum_j H_{ij} c_j = E c_i$$

10.3.1 轴对称量子系统

哈密顿量为

$$\hat{H} = -\frac{\hbar^2}{2m} \left(\frac{\partial^2}{\partial z^2} + \frac{\partial^2}{\partial \rho^2} + \frac{1}{\rho} \frac{\partial}{\partial \rho} + \frac{1}{\rho^2} \frac{\partial^2}{\partial \phi^2} \right) + \frac{1}{2}(\omega_z^2 z^2 + \omega_\perp^2 \rho^2)$$

选取基函数：

$$\Psi_{\alpha}(r, \sigma) = \psi_{n_r}^m(\rho) \cdot \psi_{n_z}(z) \cdot \frac{e^{im\phi}}{\sqrt{2\pi}} \cdot \chi_{\Sigma}(\sigma)$$

其中 $\alpha = (n_r, m, n_z, \sigma)$, χ_{Σ} 是自旋波函数。

径向波函数：

$$\psi_{n_r}^m(\rho) = \beta_{\perp} \tilde{\psi}_{n_r}^m(\eta) = N_{n_r}^m \beta_{\perp} \sqrt{2\eta}^{|m|/2} e^{-\eta/2} L_{n_r}^{|m|}(\eta)$$

其中

$$\beta = \sqrt{\frac{m\omega_{\perp}}{\hbar}} = \frac{1}{b_{\perp}}$$

$\omega_{\perp}$ 是径向频率, $b_{\perp}$ 是径向长度尺度。

$$N_{n_r}^m = \sqrt{\frac{n_r!}{(n_r + |m|)!}}$$

$$L_{n_r}^{|m|}(\eta) = \frac{e^{\eta}}{n_r! \eta^{|m|}} \frac{d^{n_r}}{d\eta^{n_r}} (e^{-\eta} \eta^{n_r + |m|})$$

$$\eta = \beta_{\perp}^2 \rho^2$$

轴向波函数：

$$\psi_{n_z}(z) = \beta_z^{1/2} \tilde{\psi}_{n_z}(\xi) = N_{n_z} \beta_z^{1/2} e^{-\xi^2/2} H_{n_z}(\xi)$$

其中

$$\beta_z = \sqrt{\frac{m\omega_z}{\hbar}} = \frac{1}{b_z}$$

$$N_{n_z} = (\sqrt{\pi} 2^{n_z} n_z!)^{-1/2}$$

$$H_{n_z}(\xi) = (-1)^{n_z} e^{\xi^2} \frac{d^{n_z}}{d\xi^{n_z}} e^{-\xi^2}$$

$$\xi = \beta_z z$$

这组基函数就是无相互作用谐振子势的精确本征函数。如果哈密顿量就是谐振子势, 在这组基下, 哈密顿量矩阵就是对角的, 本征值就是各量子数对应的能量之和:

$$E = \hbar\omega_{\perp}(2n_r + |m| + 1) + \hbar\omega_z \left(n_z + \frac{1}{2} \right)$$

如果实际势能偏离谐振子, 则需要在此基下计算矩阵元, 再对角化得到真实能级和波函数。

10.3.2 三维直角坐标系下的基矢展开法

在直角坐标系 (x, y, z) 中, 选择一组正交完备基, 可以将未知波函数表示为基函数的线性组合:

$$\psi_i(x, y, z, \sigma) = \sum_{n_x} \sum_{n_y} \sum_{n_z} \sum_{s_z} A_i^{n_x n_y n_z s_z} \psi_{n_x}(x) \psi_{n_y}(y) \psi_{n_z}(z) \delta_{s_z, \sigma}$$

其中 $\psi_{n_\mu}(x_\mu)$ 是一维谐振子本征函数, $\delta_{s_z, \sigma}$ 是自旋部分。

一维谐振子本征函数:

$$\phi_{n_\mu}(x_\mu) = b_\mu^{1/2} H_N^{(0)}(\xi_\mu) e^{-\xi_\mu^2/2}$$

其中

$$b_\mu = \sqrt{\frac{m\omega_\mu}{\hbar}} = \frac{1}{b_\mu}$$

$$\xi_\mu = b_\mu x_\mu$$

$$H_N^{(0)}(\xi_\mu) = (\sqrt{\pi} 2^{n_\mu} n_\mu!)^{-1/2} H_{n_\mu}(\xi_\mu)$$

其中 $H_{n_\mu}(\xi_\mu)$ 是 Hermite 多项式:

$$H_{n_\mu}(\xi_\mu) = (-1)^{n_\mu} e^{\xi_\mu^2} \frac{d^{n_\mu}}{d\xi_\mu^{n_\mu}} e^{-\xi_\mu^2}$$

满足:

$$\int_{-\infty}^{\infty} H_{n_\mu}^{(0)}(\xi_\mu) H_{n'_\mu}^{(0)}(\xi_\mu) e^{-\xi_\mu^2} d\xi_\mu = \delta_{n_\mu, n'_\mu}$$

10.4 离散变量表示 (DVR)

离散变量表示 (DVR) 是一种将量子力学问题离散到网格上的数值方法, 其核心思想是, 构造一组特殊的正交基函数, 使得任意波函数在这组基下的展开系数正比于波函数在特定网格上的值, 并且内积可以精确表示为加权求和, 从而避免数值积分。

可以寻找一组特殊的位置本征态 $|x_\alpha\rangle$, 它对应于网格点 x_α , 投影算符 P 将任意态投影到网格点张成的离散空间上。投影后的态叫作 DVR 原始态:

$$|\Delta_\alpha\rangle = P|x_\alpha\rangle$$

Δ_α 满足:

$$\langle \Delta_\alpha | \Delta_\beta \rangle = N_\alpha \delta_{\alpha\beta}$$

其中 N_α 是归一化常数。因此可以定义归一化的 DVR 基矢:

$$|F_\alpha\rangle = \frac{1}{\sqrt{N_\alpha}} |\Delta_\alpha\rangle$$

DVR 基矢是完备正交归一的，满足正交归一化条件：

$$\langle F_\alpha | F_\beta \rangle = \delta_{\alpha\beta}$$

可以得到

$$F_\beta(x_\alpha) = \langle x_\alpha | F_\beta \rangle = \sqrt{N_\alpha} \delta_{\alpha\beta}$$

每个 DVR 基函数只在对应的网格点上非零。

考虑任意态矢 $|\psi\rangle$ ，可以在 DVR 基下展开：

$$|\psi\rangle = \sum_{\alpha} |F_\alpha\rangle \langle F_\alpha | \psi \rangle$$

DVR 基是正交归一基，于是

$$\begin{aligned} \psi(x_\alpha) &= \langle x_\alpha | \psi \rangle = \sum_{\beta} \langle x_\alpha | F_\beta \rangle \langle F_\beta | \psi \rangle = \sum_{\beta} \sqrt{N_\alpha} \delta_{\alpha\beta} \langle F_\beta | \psi \rangle = \sqrt{N_\alpha} \langle F_\alpha | \psi \rangle \\ \Rightarrow \langle F_\alpha | \psi \rangle &= \frac{1}{\sqrt{N_\alpha}} \psi(x_\alpha) \end{aligned}$$

代入得到

$$|\psi\rangle = \sum_{\alpha} \frac{1}{N_\alpha} \psi(x_\alpha) |F_\alpha\rangle$$

两个波函数的内积可以写作

$$\langle \phi | \psi \rangle = \sum_{\alpha} \langle \phi | F_\alpha \rangle \langle F_\alpha | \psi \rangle = \sum_{\alpha} \frac{1}{N_\alpha} \phi^*(x_\alpha) \psi(x_\alpha)$$

定义权重因子

$$\omega_\alpha = \frac{1}{N_\alpha}$$

于是内积可以写作加权求和：

$$\langle \phi | \psi \rangle = \sum_{\alpha} \omega_\alpha \phi^*(x_\alpha) \psi(x_\alpha)$$

这和高斯求积公式是很像的，可以类比，DVR 基的特殊位置本征态可以选取为 Legendre 多项式的零点。

下面考虑如何构造 DVR 的奇函数，取一组完备正交基 $\{\phi_n(x)\}$ （可以是归一的厄米多项式、Legendre 多项式等等），定义投影算符：

$$P = \sum_{n=0}^{\infty} |\phi_n\rangle \langle \phi_n|$$

将其作用于位置本征态 $|x_\alpha\rangle$ ，得到未归一化的 DVR 原始态：

$$|\Delta_\alpha\rangle = P|x_\alpha\rangle = \sum_n |\phi_n\rangle \langle \phi_n|x_\alpha\rangle = \sum_n |\phi_n\rangle \phi_n^*(x_\alpha)$$

在位置表象中写出分量：

$$\Delta_\alpha(x) = \langle x|\Delta_\alpha\rangle = \sum_n \phi_n(x) \phi_n^*(x_\alpha)$$

计算内积：

$$\langle \Delta_\alpha|\Delta_\beta\rangle = \sum_n \phi_n(x_\alpha) \phi_n^*(x_\beta)$$

现在要选择特定的基函数 ϕ_n 和网格点 $\{x_\alpha\}$ ，使得 Δ_α 相互正交，可以取

$$\phi_n(x) = \sqrt{W(x)} P_n(x)$$

其中 $P_n(x)$ 是 n 次正交多项式， $W(x)$ 是权重函数，满足

$$\int W(x) P_n(x) P_m(x) dx = \delta_{nm}$$

网格点取为 n 阶正交多项式 $P_n(x)$ 的零点，此时

$$\sum_n \phi_n(x_\alpha) \phi_n^*(x_\beta) = N_\alpha \delta_{\alpha\beta}$$

归一化的 DVR 基为

$$F_\alpha(x) = \sqrt{w_\alpha} \sum_n \sqrt{W(x)W(x_\alpha)} P_n(x) P_n(x_\alpha)$$

同时，DVR 也可以将势能矩阵对角化：

$$\langle F_\alpha|V|F_\beta\rangle = \sum_k w_k F_\alpha^*(x_k) V(x_k) F_\beta(x_k) = \delta_{\alpha\beta} V(x_\alpha)$$

在 DVR 基下，势能矩阵本身就是对角的，对角元就是势能在网格点上的值。

例：利用平面波基（正交完备基）构造 DVR 基。取动量截断

$$p \in [-p_0, p_0]$$

投影算符为

$$P = \int_{-p_0}^{p_0} |p\rangle \langle p| dp$$

网格点

$$x_\alpha = \frac{\pi}{p_0} \alpha, \quad \alpha \in \mathbb{Z}$$

未归一化的 DVR 基:

$$|\Delta_\alpha\rangle = P|x_\alpha\rangle = \int_{-p_0}^{p_0} dp |p\rangle \frac{e^{-ipx_\alpha/\hbar}}{\sqrt{2\pi\hbar}}$$

计算内积:

$$\langle\Delta_\alpha|\Delta_\beta\rangle = \int_{-p_0}^{p_0} dp \frac{e^{ip(x_\alpha - x_\beta)/\hbar}}{2\pi\hbar} = \frac{p_0}{\pi\hbar} \cdot \frac{\sin[p_0(x_\alpha - x_\beta)/\hbar]}{p_0(x_\alpha - x_\beta)/\hbar} = \frac{p_0}{\pi\hbar} \delta_{\alpha\beta}$$

于是可以得到归一化的 DVR 基

$$|F_\alpha\rangle = \sqrt{\frac{\pi\hbar}{p_0}} |\Delta_\alpha\rangle$$

在 DVR 基下, 写出位置表象下的波函数:

$$\begin{aligned} F_\alpha(x) = \langle x|F_\alpha\rangle &= \sqrt{\frac{\pi\hbar}{p_0}} \int_{-p_0}^{p_0} \frac{dp}{2\pi\hbar} e^{ip(x-x_\alpha)/\hbar} = \frac{1}{\sqrt{\pi\hbar p_0}} \frac{\sin[p_0(x-x_\alpha)/\hbar]}{(x-x_\alpha)/\hbar} \\ &= \sqrt{\frac{p_0}{\pi\hbar}} \cdot \text{sinc}\left(\frac{p_0(x-x_\alpha)}{\hbar}\right) \end{aligned}$$

动量表象下的波函数:

$$\langle p|F_\alpha\rangle = \sqrt{\frac{\pi\hbar}{p_0}} \langle p|\Delta_\alpha\rangle = \frac{1}{\sqrt{2p_0}} e^{-ipx_\alpha/\hbar}$$

任意波函数的展开:

$$|\psi\rangle = \sum_{\alpha} |F_\alpha\rangle \langle F_\alpha|\psi\rangle = \sum_{\alpha} |F_\alpha\rangle \sqrt{\frac{\pi\hbar}{p_0}} \psi(x_\alpha)$$

10.5 B-spline 展开法

B-spline 展开法是一种用于函数逼近和求解微分方程的数值方法, 尤其在计算物理中广泛应用于处理非均匀网格、需要局部控制和高阶光滑性的问题。

B-spline 基函数是一类分段多项式函数, 具有局部支撑 (只在有限区间非零) 和高光滑性。递推定义为

$$b_{i,0}(x) = \begin{cases} 1, & x_i \leq x < x_{i+1} \\ 0, & \text{else} \end{cases}$$

$$b_{i,k}(x) = \frac{x - x_i}{x_{i+k} - x_i} b_{i,k-1}(x) + \frac{x_{i+k+1} - x}{x_{i+k+1} - x_{i+1}} b_{i+1,k-1}(x)$$

$b_{i,k}(x)$ 是 k 次分段多项式, 局部支撑区间为 $[x_i, x_{i+k+1}]$ 。

诶, 突然发现这部分内容不考, 哈哈

10.6 小波分析

诶，突然发现这部分内容也不考，哈哈哈

10.7 有限元法 (FEM)

有限差分法处理复杂边界（如曲线边界）比较麻烦，而有限元法通过将区域划分成许多小单元（比如三角形、四边形），能很好地适应复杂几何形状。

有限元法的基本流程是：

- (i) 将求解区域划分为许多简单的小单元，这些单元互不重叠且覆盖整个区域；
- (ii) 在每个单元上，用低阶多项式（线性、二次）近似待求的场量；
- (iii) 通过变分原理或加权残量法，将微分方程离散为代数方程组求解。

Delaunay 三角剖分是一种常用的生成三角形网格的方法，具有以下性质：

- (i) 最大化最小角：在所有可能的三角剖分中，Delaunay 三角剖分使三角形的最小内角尽可能大，从而避免出现特别瘦长的三角形，保证数值计算的稳定性；
- (ii) 凸包性：所有三角形的并集等于点集的凸包（最外围点连成的多边形）；
- (iii) 最近邻关系：对于任意一点 p ，它的最近邻点 b 一定与 p 相连（属于剖分的某个三角形）。

考虑求解二维 Poisson 方程：

$$\nabla^2 \phi(\vec{r}) = f(\vec{r}), \quad \vec{r} \in \Omega$$

直接求解偏微分方程较为困难，转而求解其对应的变分问题：

$$\delta J = \delta \int_D \left[\frac{1}{2} (\nabla \phi)^2 + f \phi \right] d^2 r = 0$$

该泛函的极值点对应着原方程的解。

将区域 D 划分为许多小三角形。在每个三角形上，用多项式近似 ϕ ：

线性单元：

$$\phi(x, y) = a_i + b_i x + c_i y$$

三个系数由三个顶点唯一确定。

二次单元：

$$\phi(x, y) = a_i + b_i x + c_i y + d_i xy + e_i x^2 + f_i y^2$$

为了保证连续性，相邻三角形在公共顶点上的数值必须相等。每个三角形就是一个单元，顶点称为节点。

为了简化单元上的积分和插值，可以引入自然坐标 ξ_a, ξ_b, ξ_c ，定义为

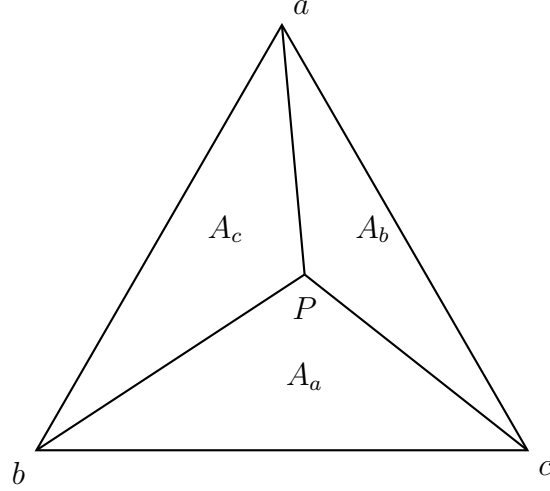
$$\xi_a = \frac{A_a}{A}, \quad \xi_b = \frac{A_b}{A}, \quad \xi_c = \frac{A_c}{A}$$

其中 A 是三角形总面积, A_a, A_b, A_c 分别是三角形内部点 (x, y) 与三角形的对边构成的子三角形的面积, 满足

$$\xi_a + \xi_b + \xi_c = 1$$

在顶点 a 处, 有

$$\xi_a = 1, \quad \xi_b = 0, \quad \xi_c = 0$$



于是, P 点坐标 (x, y) 可以用自然坐标表示:

$$\begin{pmatrix} 1 \\ x \\ y \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 \\ x_a & x_b & x_c \\ y_a & y_b & y_c \end{pmatrix} \begin{pmatrix} \xi_a \\ \xi_b \\ \xi_c \end{pmatrix}$$

逆变换:

$$\begin{pmatrix} \xi_a \\ \xi_b \\ \xi_c \end{pmatrix} = \frac{1}{2A} \begin{pmatrix} x_b y_c - x_c y_b & y_b - y_c & x_c - x_b \\ x_c y_a - x_a y_c & y_c - y_a & x_a - x_c \\ x_a y_b - x_b y_a & y_a - y_b & x_b - x_a \end{pmatrix} \begin{pmatrix} 1 \\ x \\ y \end{pmatrix}$$

其中

$$2A = \begin{vmatrix} 1 & 1 & 1 \\ x_a & x_b & x_c \\ y_a & y_b & y_c \end{vmatrix}$$

若已知三角形顶点的插值函数 ϕ_a, ϕ_b, ϕ_c , 则可以通过自然坐标插值得到单元内任意点的函数值:

$$\phi(x, y) = \xi_a \phi_a + \xi_b \phi_b + \xi_c \phi_c$$

面积坐标的幂次积分公式:

$$\int_A \xi_a^k \xi_b^l \xi_c^m d\Omega = 2A \cdot \frac{k!l!m!}{(k+l+m+2)!}$$

下面考虑将 Poisson 方程离散化，已知

$$\begin{aligned}
\phi(x, y) &= \xi_a \phi_a + \xi_b \phi_b + \xi_c \phi_c \\
\Rightarrow \nabla \phi(x, y) &= \nabla \xi_a \phi_a + \nabla \xi_b \phi_b + \nabla \xi_c \phi_c \\
\nabla^2 \phi(x, y) &= \sum_{i,j=a,b,c} \phi_i \phi_j (\nabla \xi_i \cdot \nabla \xi_j) \\
\Rightarrow J[\phi] &= \int_A \frac{1}{2} |\nabla \phi|^2 d\Omega + \int_A f \phi d\Omega = \frac{1}{2} \sum_{i,j=a,b,c} \phi_i \phi_j \int_A \nabla \xi_i \cdot \nabla \xi_j d\Omega + \sum_{i=a,b,c} \phi_i \int_A f \xi_i d\Omega \\
&= \frac{1}{2} (\vec{\phi})^T \mathbf{K} \vec{\phi} + \vec{r}^T \vec{\phi}
\end{aligned}$$

其中

$$\begin{aligned}
\vec{\phi} &= (\phi_a, \phi_b, \phi_c)^T \\
\mathbf{K}_{ij} &= \int_A \nabla \xi_i \cdot \nabla \xi_j d\Omega \\
r_i &= \int_A f \xi_i d\Omega
\end{aligned}$$

$\mathbf{K}$ 称作对称刚度矩阵， $\vec{r}$ 为载荷向量。

变分后得到线性方程组：

$$\mathbf{K} \vec{\phi} = \vec{r}$$

下面考虑如何计算 $\mathbf{K}$ 和 $\vec{r}$ ，已知

$$\begin{aligned}
\xi_a &= \frac{1}{2A} [(x_b y_c - x_c y_b) + (y_b - y_c)x + (x_c - x_b)y] \\
2A &= (x_b - x_a)(y_c - y_a) - (x_c - x_a)(y_b - y_a) \\
\Rightarrow \nabla \xi_a &= \left(\frac{\partial \xi_a}{\partial x}, \frac{\partial \xi_a}{\partial y} \right) = \frac{1}{2A} (y_b - y_c, x_c - x_b)
\end{aligned}$$

同理

$$\begin{aligned}
\nabla \xi_b &= \frac{1}{2A} (y_c - y_a, x_a - x_c) \\
\nabla \xi_c &= \frac{1}{2A} (y_a - y_b, x_b - x_a)
\end{aligned}$$

于是

$$\mathbf{K}_{ij} = \int_A \nabla \xi_i \cdot \nabla \xi_j d\Omega = A (\nabla \xi_i \cdot \nabla \xi_j)$$

代入可得

$$\mathbf{K} = \frac{1}{4A} (\vec{bb}^T + \vec{cc}^T)$$

其中

$$\vec{b} = (y_b - y_c, y_c - y_a, y_a - y_b)^T$$

$$\vec{c} = (x_c - x_b, x_a - x_c, x_b - x_a)^T$$

可以令

$$y_{ab} = y_a - y_b, \quad y_{bc} = y_b - y_c, \quad y_{ca} = y_c - y_a$$

$$x_{ab} = x_a - x_b, \quad x_{bc} = x_b - x_c, \quad x_{ca} = x_c - x_a$$

对称刚性矩阵可以写作

$$\mathbf{K} = \frac{1}{4A} \begin{pmatrix} y_{bc}^2 + x_{bc}^2 & y_{bc}y_{ca} + x_{bc}x_{ca} & y_{bc}y_{ab} + x_{bc}x_{ab} \\ y_{ca}y_{bc} + x_{ca}x_{bc} & y_{ca}^2 + x_{ca}^2 & y_{ca}y_{ab} + x_{ca}x_{ab} \\ y_{ab}y_{bc} + x_{ab}x_{bc} & y_{ab}y_{ca} + x_{ab}x_{ca} & y_{ab}^2 + x_{ab}^2 \end{pmatrix}$$

再来求 $\vec{r}$, 对于函数 $f(\vec{r})$, 同样可以插值得

$$f(\vec{r}) = \xi_a f_a + \xi_b f_b + \xi_c f_c$$

利用幂次积分公式

$$r_a = \int_A f \xi_a d\Omega = \int_A (\xi_a f_a + \xi_b f_b + \xi_c f_c) \xi_a d\Omega = 2A \left(\frac{f_a}{12} + \frac{f_b}{24} + \frac{f_c}{24} \right)$$

同理

$$r_b = 2A \left(\frac{f_b}{12} + \frac{f_c}{24} + \frac{f_a}{24} \right)$$

$$r_c = 2A \left(\frac{f_c}{12} + \frac{f_a}{24} + \frac{f_b}{24} \right)$$

对每个三角元, 计算 $\mathbf{K}$ 和 $\vec{r}$, 然后组装成全局的刚性矩阵和载荷向量, 最后求解线性方程组得到数值解。

11 动力学方程

下面我们来考虑时间演化问题的数值求解方法。

11.1 一维扩散方程

一维扩散方程为:

$$\nabla^2 u(x, t) = \frac{\partial u}{\partial t}, \quad 0 < x < L$$

初始条件:

$$u(x, 0) = g(x)$$

这个方程可以解析求解, 在初始条件为单位点源 δ 函数时, 基本解为:

$$w(x, t) = \frac{1}{\sqrt{4\pi t}} e^{-x^2/4t}$$

对于该初始条件, 扩散方程的解是基本解与初始条件的卷积:

$$u(x, t) = \int_{-\infty}^{\infty} w(x - x', t) g(x') dx'$$

下面考虑数值解。

11.1.1 有限差分法

记

$$u_{i,j} = u(x_i, t_j)$$

时间向前差分:

$$\frac{\partial u}{\partial t} = \frac{u_{i,j+1} - u_{i,j}}{\Delta t}$$

空间中心差分:

$$\nabla^2 u = \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{\Delta x^2}$$

定义

$$\alpha = \frac{\Delta t}{\Delta x^2}$$

得到递推公式:

$$u_{i,j+1} = \alpha u_{i-1,j} + (1 - 2\alpha) u_{i,j} + \alpha u_{i+1,j}$$

稳定要求

$$\alpha \leq \frac{1}{2}$$

第一步的时间计算:

$$u_{i,1} = \alpha g(x_{i-1}) + (1 - 2\alpha) g(x_i) + \alpha g(x_{i+1})$$

11.1.2 Crank-Nicolson 格式

在 Crank-Nicolson 格式中, 时间导数采用向后差分:

$$\frac{\partial u}{\partial t} = \frac{u_{i,j} - u_{i,j-1}}{\Delta t}$$

空间二阶导数取为两个时间层的加权平均：

$$\nabla^2 u = \theta \cdot \frac{\partial^2 u}{\partial x^2} \Big|_{t=t_j} + (1-\theta) \cdot \frac{\partial^2 u}{\partial x^2} \Big|_{t=t_{j-1}}$$

于是离散后的方程变为：

$$\theta \cdot \frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{\Delta x^2} + (1-\theta) \cdot \frac{u_{i-1,j-1} - 2u_{i,j-1} + u_{i+1,j-1}}{\Delta x^2} = \frac{u_{i,j} - u_{i,j-1}}{\Delta t}$$

$\theta = 0$ 时是显式格式，和上面的有限差分法完全一致；

$\theta = 1$ 时为全隐式格式，是无条件稳定的，但时间精度只有一阶；

$\theta = \frac{1}{2}$ 时为 Crank-Nicolson 格式，时间精度达到二阶，同时也是无条件稳定的。

代入后整理得到：

$$-\alpha u_{i-1,j} + 2(1+\alpha)u_{i,j} - \alpha u_{i+1,j} = \alpha u_{i-1,j-1} + 2(1-\alpha)u_{i,j-1} + \alpha u_{i+1,j-1}$$

将等式左边与右边写成矩阵形式：

$$LHS = \begin{pmatrix} 2+2\alpha & -\alpha & 0 & \cdots & 0 \\ -\alpha & 2+2\alpha & -\alpha & \cdots & 0 \\ 0 & -\alpha & 2+2\alpha & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 2+2\alpha \end{pmatrix} \begin{pmatrix} u_{1,j} \\ u_{2,j} \\ u_{3,j} \\ \vdots \\ u_{n,j} \end{pmatrix}$$

$$RHS = \begin{pmatrix} 2-2\alpha & \alpha & 0 & \cdots & 0 \\ \alpha & 2-2\alpha & \alpha & \cdots & 0 \\ 0 & \alpha & 2-2\alpha & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 2-2\alpha \end{pmatrix} \begin{pmatrix} u_{1,j-1} \\ u_{2,j-1} \\ u_{3,j-1} \\ \vdots \\ u_{n,j-1} \end{pmatrix}$$

记三对角矩阵：

$$\mathbf{B} = \begin{pmatrix} 2 & -1 & 0 & \cdots & 0 \\ -1 & 2 & -1 & \cdots & 0 \\ 0 & -1 & 2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 2 \end{pmatrix}$$

于是线性方程组可以写作：

$$(2\mathbf{I} + \alpha\mathbf{B})\vec{u}_j = (2\mathbf{I} - \alpha\mathbf{B})\vec{u}_{j-1}$$

$$\vec{u}_j = (2\mathbf{I} + \alpha\mathbf{B})^{-1}(2\mathbf{I} - \alpha\mathbf{B})\vec{u}_{j-1}$$

11.2 波动方程

11.2.1 一维波动方程的有限差分法

一维波动方程为:

$$\frac{\partial^2 u}{\partial x^2} = \frac{\partial^2 u}{\partial t^2}$$

定解条件为:

$$u(x, 0) = g(x)$$

$$\frac{\partial u}{\partial t}(x, 0) = 0$$

$$u(0, t) = u(1, t) = 0$$

中心差分:

$$\frac{\partial^2 u}{\partial x^2} = \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{\Delta x^2}$$

$$\frac{\partial^2 u}{\partial t^2} = \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{\Delta t^2}$$

得到递推公式:

$$u_{i,j+1} = 2u_{i,j} - u_{i,j-1} + \frac{\Delta t^2}{\Delta x^2}(u_{i+1,j} - 2u_{i,j} + u_{i-1,j})$$

初始条件:

$$\frac{\partial u}{\partial t}(x, 0) = 0$$

$$\Rightarrow \frac{u_{i,1} - u_{i,-1}}{2\Delta t} = 0$$

$$\Rightarrow u_{i,1} = u_{i,-1} \quad (\text{自然边界条件})$$

代入递推公式:

$$u_{i,1} = g(x_i) + \frac{\Delta t^2}{2\Delta x^2} [g(x_{i+1}) - 2g(x_i) + g(x_{i-1})]$$

11.2.2 二维波动方程的有限差分法

二维波动方程为:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = \frac{\partial^2 u}{\partial t^2}, \quad (x, y) \in (0, 1) \times (0, 1)$$

定解条件为:

$$u(x, y, 0) = g(x, y)$$

$$u(0, 0, t) = u(1, 1, t) = 0$$

$$\frac{\partial u}{\partial t}(x, y, 0) = 0$$

离散化：

$$t_l = l\Delta t, \quad x_i = i\Delta x, \quad y_j = j\Delta y$$

$$\Delta x = \Delta y = h$$

中心差分：

$$u_{xx} = \frac{u_{i+1,j}^l - 2u_{i,j}^l + u_{i-1,j}^l}{h^2}$$

$$u_{yy} = \frac{u_{i,j+1}^l - 2u_{i,j}^l + u_{i,j-1}^l}{h^2}$$

$$u_{tt} = \frac{u_{i,j}^{l+1} - 2u_{i,j}^l + u_{i,j}^{l-1}}{\Delta t^2}$$

代入得到递推公式：

$$u_{i,j}^{l+1} = 2u_{i,j}^l - u_{i,j}^{l-1} + \frac{\Delta t^2}{h^2} (u_{i+1,j}^l + u_{i-1,j}^l + u_{i,j+1}^l + u_{i,j-1}^l - 4u_{i,j}^l)$$

边界条件：

$$u_{i,j}^{-1} = u_{i,j}^1$$

$$\Rightarrow u_{i,j}^1 = g(x_i, y_j) + \frac{\Delta t^2}{2h^2} [g(x_{i+1}, y_j) + g(x_{i-1}, y_j) + g(x_i, y_{j+1}) + g(x_i, y_{j-1}) - 4g(x_i, y_j)]$$

11.2.3 变系数波动方程的有限差分法

海啸波的传播通常用变系数的波动方程描述：

$$\frac{\partial^2 u}{\partial t^2} = \nabla \cdot (\lambda(x, y) \nabla u)$$

初始条件：

$$u(x, y, 0) = g(x, y)$$

$$\frac{\partial u}{\partial t}(x, y, 0) = 0$$

差分公式为：

$$\frac{\partial}{\partial x} \left(\lambda(x, y) \frac{\partial u}{\partial x} \right) = \frac{1}{\Delta x} \left(\lambda_{i+1/2,j} \frac{u_{i+1,j}^l - u_{i,j}^l}{\Delta x} - \lambda_{i-1/2,j} \frac{u_{i,j}^l - u_{i-1,j}^l}{\Delta x} \right)$$

$$\frac{\partial}{\partial y} \left(\lambda(x, y) \frac{\partial u}{\partial y} \right) = \frac{1}{\Delta y} \left(\lambda_{i,j+1/2} \frac{u_{i,j+1}^l - u_{i,j}^l}{\Delta y} - \lambda_{i,j-1/2} \frac{u_{i,j}^l - u_{i,j-1}^l}{\Delta y} \right)$$

11.3 量子力学中的时间演化

11.3.1 时间演化算符

考虑量子力学中的时间演化，可以定义时间演化算符 $\hat{U}(t, t_0)$ ：

$$|\psi(t)\rangle = \hat{U}(t, t_0)|\psi(t_0)\rangle$$

时间演化算符是么正算符：

$$\hat{U}^\dagger(t, t_0)\hat{U}(t, t_0) = \hat{I}$$

且满足

$$\hat{U}(t_0, t) = \hat{U}^{-1}(t, t_0)$$

考虑薛定谔方程：

$$i\frac{\partial}{\partial t}\hat{U}(t, t_0) = \hat{H}\hat{U}(t, t_0)$$

满足积分方程：

$$\hat{U}(t, t_0) = \hat{I} - i \int_{t_0}^t \hat{H}\hat{U}(t', t_0)dt'$$

现在考虑其数值解，将时间切成 Δt 的小步，在每一个小步内，Hamilton 量变化不大，可以近似为：

$$\hat{U}(t_{k+1}, t_k) \approx e^{-i\hat{H}(t_k)\Delta t}$$

这是指数近似，误差为 $O(\Delta t^2)$ 。

也可以使用更高精度的 Crank-Nicolson 格式：

$$\hat{U}(t_{k+1}, t_k) \approx \left(\hat{I} + \frac{i\Delta t}{2} \hat{H}(t_{k+1/2}) \right)^{-1} \left(\hat{I} - \frac{i\Delta t}{2} \hat{H}(t_{k+1/2}) \right)$$

11.3.2 有限差分法

考虑薛定谔方程：

$$\frac{\partial}{\partial t}\psi(x, t) = \left[-\frac{1}{2} \frac{\partial^2}{\partial x^2} + V(x) + \epsilon(t)x \right] \psi(x, t)$$

二阶导数可以空间离散：

$$\frac{\partial^2}{\partial x^2}\psi(x_j, t) = \frac{\psi(x_{j+1}, t) - 2\psi(x_j, t) + \psi(x_{j-1}, t)}{\Delta x^2}$$

时间演化可以写作：

$$\hat{U}^-(t_{k+1/2})\psi(t_{k+1}) = \hat{U}^+(t_{k+1/2})\psi(t_k)$$

其中矩阵的对角元：

$$\hat{U}_{jj}^{\pm} = 1 \mp i\Delta t \left[\frac{1}{\Delta x^2} + V(x_j) + \epsilon(t_{k+1/2})x_j \right]$$

次对角元：

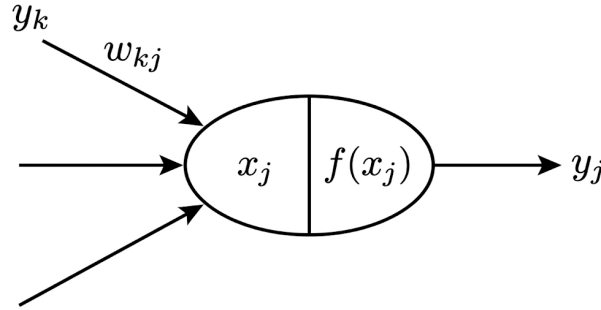
$$U_{j,j\pm 1}^{\pm} = \pm i \frac{\Delta t}{2\Delta x^2}$$

其余为 0。

于是时间演化问题转化为了求解线性方程组。

12 神经网络

12.1 bp 神经网络



如图所示，神经元先接收上层的所有神经元信号 y_k ，分别乘以对应的连接权重 w_{kj} ，然后求和，得到

$$x_j = \sum_k w_{kj} y_k$$

把求和结果代入激活函数 f ，得到神经元的最终输出

$$y_j = f(x_j)$$

若真实答案为 t_j ，网络猜的结果为 y_j ，可以定义损失函数：

$$E = \frac{1}{2} \sum_j (t_j - y_j)^2$$

学习的最终目标是最小化损失函数 E 。利用梯度下降法，可以调整权重：

$$\Delta w_{kj} = -\alpha \frac{\partial E}{\partial w_{kj}}$$

链式法则：

$$\frac{\partial E}{\partial w_{kj}} = \frac{\partial E}{\partial y_j} \cdot \frac{\partial y_j}{\partial x_j} \cdot \frac{\partial x_j}{\partial w_{kj}}$$

分别代表总误差对神经元输出的偏导、输出对加权输入的偏导和加权输入对权重的偏导。

定义神经元的本地误差信号：

$$\delta_j = -\frac{\partial E}{\partial y_j} \cdot \frac{\partial y_j}{\partial x_j}$$

权重更新可以写作：

$$\Delta w_{kj} = \alpha \delta_j y_k$$

输出层的 $f(x)$ 可取为 Sigmoid 函数：

$$y = f(x) = \frac{1}{1 + e^{-x}}$$

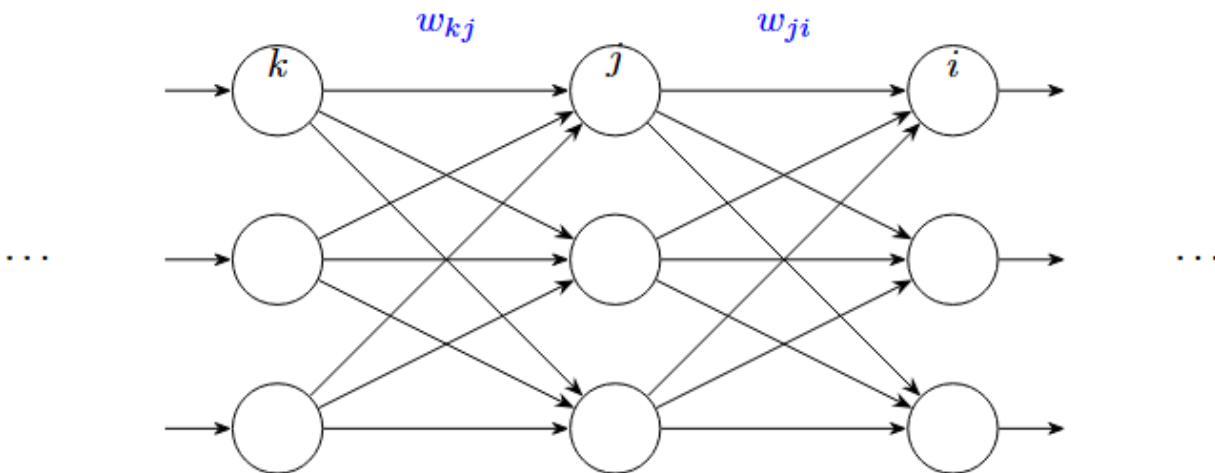
其导数为

$$f'(x) = y(1 - y)$$

于是

$$\frac{\partial E}{\partial y_j} = -(t_j - y_j)$$

$$\frac{\partial E}{\partial w_{kj}} = -(t_j - y_j) \cdot y_j(1 - y_j) \cdot y_k$$



多层网络会复杂一些，如图所示， j 为隐藏层，损失函数并不直接由 j 层决定。想要确定 j 层对误差函数的作用，需要看 i 层的误差：

$$\frac{\partial E}{\partial y_j} = \sum_i \frac{\partial E}{\partial y_i} \cdot \frac{\partial y_i}{\partial x_i} \cdot \frac{\partial x_i}{\partial y_j} = -\sum_i \delta_i w_{ji}$$

隐藏层的误差是由上层所有神经元的误差加权叠加回来的。

对于输出层：

$$\delta_j = (t_j - y_j) \cdot y_j(1 - y_j)$$

对于隐藏层：误差反向传播

$$\delta_j = \sum_i \delta_i w_{ji} \cdot y_j(1 - y_j)$$

12.2 贝叶斯神经网络

对于标准神经网络，每个权重对应一个具体的数值，训练的过程是改变权重，直到找到最佳的权重；

贝叶斯神经网络中，每个权重不再是一个数，而是一个概率分布。其核心数学公式是贝叶斯定理：

$$p(\omega|D) = \frac{p(D|\omega)p(\omega)}{p(D)}$$

其中 ω 是权重， D 是数据集；

$p(\omega)$ 是先验概率：没有看到数据之前，对权重的初步猜测；

$p(D|\omega)$ 是似然函数：在权重为 ω 的情况下，数据集 D 出现的概率；

$p(\omega|D)$ 是后验概率：看完数据之后，对权重修正后的判断。

定义实验测量值与理论值之差：

$$t_n = M^{(exp)}(x) - M^{(th)}(x)$$

激活函数：

$$y = f(x, \omega) + \sigma \epsilon$$

σ 为噪声强度。

$$f(x, \omega) = a + \sum_j b_j \tanh \left(c_j + \sum_i d_{ji} x_i \right)$$

其中

$$\omega = \{a, b_j, c_j, d_{ji}\}$$

a 为输出层的偏置； b_j 为隐藏层到输出层的权重； c_j 为隐藏层神经元的偏置； d_{ji} 为输入层到隐藏层的权重。

似然函数：

$$p(D|\omega) = \exp \left(-\frac{\chi^2}{2} \right)$$
$$\chi^2 = \sum_{n=1}^N \left(\frac{t_n - f(x, \omega)}{\Delta t_n} \right)^2$$

其中 Δt_n 是噪声。

最终的平均预测值是所有可能权重的平均：

$$\langle f(x) \rangle = \int f(x, \omega) p(\omega|D) d\omega$$

预测的不确定度：

$$\Delta f = \sqrt{\langle f^2(x) \rangle - \langle f(x) \rangle^2}$$

这个积分的完成需要用到蒙特卡洛积分。

考虑一个最简单的模型：

$$y = \theta + \epsilon$$

现在有一组数据 $y_1, \dots, y_n$ ，根据这些数据来修正对 θ 的判断。

正态分布：

$$X \sim N(\mu, \sigma^2) \Rightarrow p(X) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(X - \mu)^2}{2\sigma^2}\right)$$

似然函数为：

$$p(y|\theta) \sim N(\theta, \sigma^2)$$

显然分布为：

$$p(\theta) \sim N(\mu, \delta^2)$$

后验分布仍然为正态分布：

$$p(\theta|y) \propto \exp\left(-\frac{1}{2\delta^2}(\theta - \mu)^2\right) \times \exp\left(-\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \theta)^2\right)$$

可以写作：

$$p(\theta|y) \sim N(\nu, \gamma^2)$$

后验均值为：

$$\nu = \frac{n\bar{y}/\sigma^2 + \mu/\delta^2}{n/\sigma^2 + 1/\delta^2}$$

后验方差为：

$$\gamma^2 = \left(\frac{n}{\sigma^2} + \frac{1}{\delta^2}\right)^{-1}$$

总之，贝叶斯推断有三种方法：

(1) MLE（最大似然估计）

$$\theta_{MLE} = \arg \max_{\theta} p(y|\theta)$$

寻找使得数据出现概率最大的参数。

(2) MAP (最大后验估计)

$$\theta_{MAP} = \arg \max_{\theta} p(y|\theta)p(\theta)$$

(3) 完全贝叶斯估计

保留整个后验分布 $P(\theta|y)$ ，把所有点进行加权积分。

12.3 正则化

正则化是机器学习中常用的一种技术，旨在防止模型过拟合。过拟合是指模型在训练数据上表现很好，但在测试数据上表现很差的现象。正则化通过在损失函数中添加一个惩罚项来限制模型的复杂度，从而解决过拟合的问题。

似然函数：

$$p(D|\omega) = \frac{\exp(-\beta E_D)}{Z_D(\beta)}$$

先验分布：

$$p(\omega) = \frac{\exp(-\alpha E_w)}{Z_w(\alpha)}$$

其中 E_D 衡量模型预测和真实标签之间的差距，即

$$E_D = \frac{1}{2} \sum_n (y(x, w) - t_n)^2$$

E_w 衡量模型自身的复杂度，通常取为权重的平方和：

$$E_w = \frac{1}{2} \sum_i w_i^2$$

β 反映对数据的信任程度， α 反映对“权重应该很小”的先验信念的信任程度。

根据贝叶斯定理，后验概率：

$$p(\omega|D) \propto p(D|\omega)p(\omega) = \exp(-\beta E_D - \alpha E_w)$$

采用 MAP 估计，要让后验概率最大，即要让

$$f(\omega) = \beta E_D + \alpha E_w$$

最小化，具体展开：

$$f(\omega) = \frac{\beta}{2} \sum_n (y(x, w) - t_n)^2 + \frac{\alpha}{2} \sum_i w_i^2$$

第一项为标准的均方误差损失，第二项为 L2 正则化。如果把 α 调大，就相当于增加了惩罚力度。

12.4 奥卡姆剃刀

在机器学习里，如果两个模型都能把训练数据拟合得一样好，更简单的那个往往在测试集上表现更好。越复杂的模型，在贝叶斯框架下会被自动惩罚。

定义边缘似然：

$$p(D) = \int p(D|\omega)p(\omega)d\omega$$

衡量的是在考虑所有权重可能性的情况下，这个模型生成当前数据 D 的平均能力。比较两个不同复杂度的模型时，谁的 $p(D)$ 更大，谁就胜出。

高斯近似：

$$p(D) \approx p(D|\omega_{MAP}) \times \frac{\Delta\omega_{\text{后验}}}{\Delta\omega_{\text{先验}}}$$

取对数：

$$\ln p(D) \approx \ln p(D|\omega_{MAP}) + \ln \frac{\Delta\omega_{\text{后验}}}{\Delta\omega_{\text{先验}}}$$

第一项衡量模型在最优参数附近，对数据拟合得有多好。显然，模型越复杂，能达到的拟合度就越高，这一项会增大；第二项为奥卡姆因子，当模型越复杂，这个负数的绝对值就越大，惩罚越严重。

当增加神经网络的层数时，拟合度会变好，但奥卡姆惩罚会变重。最优的模型，是使得两者求和最大的地方。

13 量子计算

13.1 MPI

MPI（消息传递接口）是一个库，程序能在多台计算机、多个 CPU 核心之间互相发送消息、同步进度。有两个最基础的函数：

MPI_Comm_size：该函数返回当前团队里总共有多少进程在干活；

MPI_Comm_rank：该函数返回当前进程自己的编号，从 0 到 size-1。

MPI 中最经典的协作模式为主从并行模式。如果想让多个 CPU 核心协同完成一个任务，必须要有一个“总指挥”来派发任务、收集结果。

```
1 if rank==0:
2     print("I am the master process, I will distribute tasks.")
3     for i in range(1,size):
4         comm.send(i,dest=i,tag=10)
5     # master process does some work here
```

```

6     results=[]
7     for i in range(1,size):
8         result=comm.recv(source=i,tag=20)
9         results.append(result)
10 else:
11     task=comm.recv(source=0,tag=10)    #收到任务
12     print(f"I am worker process {rank}, I received task {task}.")
13     # worker process does some work here
14     result=task**2    #例：计算平方
15     comm.send(result,dest=0,tag=20)    #返回结果

```

阻塞通信 (Blocking): MPI_send 和 MPI_recv 是阻塞通信函数, 调用它们的进程会一直等待, 直到消息发送或接收完成。如果老大 (rank=0) 和工人 (rank≠0) 都同时在 send, 就会发生死锁 (Deadlock), 因为双方都在等对方完成发送。

```

1 if rank==0:
2     data="Hello, worker!"
3     comm.send(data,dest=1,tag=10)    #必须等rank 1接收才结束
4 elif rank==1:
5     data=comm.recv(source=0,tag=10)    #必须等rank 0发送才结束

```

非阻塞通信 (Non-blocking): MPI_Isend 和 MPI_Irecv 是非阻塞通信函数, 调用它们的进程不会等待消息发送或接收完成, 而是立即返回一个请求对象 (Request)。进程可以继续执行其他任务, 直到需要检查通信是否完成时, 再使用 MPI_Wait 来等待或测试请求对象。

```

1 if rank==0:
2     data="Hello, worker!"
3     request=comm.isend(data,dest=1,tag=10)    #立即返回, 不等待接收完成
4     # master process can do other work here
5     request.wait()    #等到发送完成
6     print("Data sent to worker.")
7 elif rank==1:
8     request=comm.irecv(source=0,tag=10)    #立即返回, 不等待发送完成
9     # worker process can do other work here
10    data=request.wait()    #等到接收完成, 拿到数据
11    print(f"Received data from master: {data}")

```

集体通信 (Collective Communication): MPI_Bcast 函数, 类似于广播, 允许一个进

程将数据发送给所有其他进程。MPI_Barrier 函数，所有进程必须在此处等待，直到所有进程都到达这个点，才会继续执行。这对于同步非常有用。

13.2 OpenMP

在 OpenMP（共享内存并行）中，所有线程共享内存。主线程创建一个庞大的数组，然后分配给不同的线程处理。每个线程都可以访问这个数组的任意部分，进行读写操作。

```
1  from numba import njit,prange
2  import numba
3
4  @njit(parallel=True)
5  def hello():
6      for i in prange(4):
7          print(f"Hello from thread {i}!")
```

实际上经常会采用混合并行，把 MPI 和 OpenMP 结合起来使用。MPI 负责在不同计算节点之间通信，OpenMP 负责在每个节点内部的多线程并行。

13.3 量子计算

量子计算的基本单元是量子比特（qubit），它可以同时处于 0 和 1 的叠加态。量子计算利用量子叠加和量子纠缠等现象，能够在某些问题上实现指数级的加速。

量子纠缠最典型的例子是贝尔态：

$$\frac{1}{\sqrt{2}} (|0\rangle_A \otimes |1\rangle_B - |1\rangle_A \otimes |0\rangle_B)$$

测量 A 时，如果 A 坍缩为 0，那么 B 必然坍缩为 1。

量子计算中最基本的数学操作是门，单比特门：

(1) Pauli—X 门（非门）：把 $|0\rangle$ 变成 $|1\rangle$ ，把 $|1\rangle$ 变成 $|0\rangle$ 。

$$X|0\rangle = |1\rangle, \quad X|1\rangle = |0\rangle$$

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

(2) Pauli—Y 门：同时涉及相位和翻转。

$$Y|0\rangle = i|1\rangle, \quad Y|1\rangle = -i|0\rangle$$

$$Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$$

(3) Pauli—Z 门：只改变相位，不改变概率幅。

$$Z|0\rangle = |0\rangle, \quad Z|1\rangle = -|1\rangle$$

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

(4) Hadamard 门：把 $|0\rangle$ 和 $|1\rangle$ 变成叠加态。

$$H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad H|1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

双比特门：CNOT 门。有两个量子比特，分别为控制位和目标位。只有当控制位是 $|1\rangle$ 时，才对目标位执行翻转。

$$\text{CNOT}|00\rangle = |00\rangle, \quad \text{CNOT}|01\rangle = |01\rangle$$

$$\text{CNOT}|10\rangle = |11\rangle, \quad \text{CNOT}|11\rangle = |10\rangle$$

两个独立量子态：

$$\psi_1 = \begin{pmatrix} \alpha_1 \\ \beta_1 \end{pmatrix}, \quad \psi_2 = \begin{pmatrix} \alpha_2 \\ \beta_2 \end{pmatrix}$$

组合为双量子比特系统时，需要做张量积：

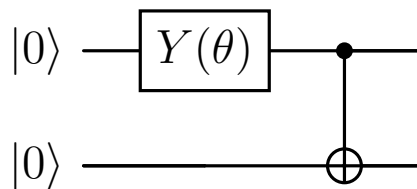
$$\psi = \psi_1 \otimes \psi_2 = \begin{pmatrix} \alpha_1\alpha_2 \\ \alpha_1\beta_2 \\ \beta_1\alpha_2 \\ \beta_1\beta_2 \end{pmatrix}$$

代表四种经典组合 (00, 01, 10, 11) 的概率幅。

CNOT 门可以写成 4×4 置换矩阵：

$$U_{\text{CNOT}} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

生成纠缠态的标准流程为：**制造叠加态 + 施加 CNOT 门。**



首先利用 $R_y(\theta)$ 旋转门制造叠加：

$$|0\rangle \otimes |0\rangle \xrightarrow{R_y(\theta)} \left(\cos \frac{\theta}{2} |0\rangle + \sin \frac{\theta}{2} |1\rangle \right) \otimes |0\rangle$$

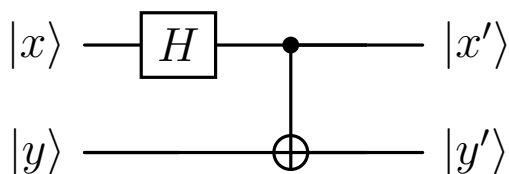
此时两个比特还没有纠缠，第一个比特处于叠加态，第二个比特是 0。

再施加 CNOT 门：

$$\psi = \cos \frac{\theta}{2} |00\rangle + \sin \frac{\theta}{2} |11\rangle$$

得到了纠缠态。

贝尔态：



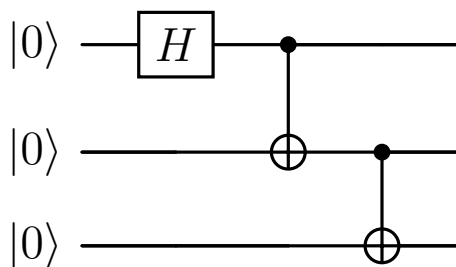
输入为 $|x\rangle = 0$ 且 $|y\rangle = 0$ ，经过 H 门：

$$\psi_1 = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes |0\rangle = \frac{|00\rangle + |10\rangle}{\sqrt{2}}$$

再经过 CNOT 门：

$$\psi_2 = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$

GHZ 态：



初始状态为：

$$\psi_0 = |000\rangle$$

经过 H 门:

$$\psi_1 = \frac{|000\rangle + |100\rangle}{\sqrt{2}}$$

经过第一个 CNOT 门:

$$\psi_2 = \frac{|000\rangle + |110\rangle}{\sqrt{2}}$$

经过第二个 CNOT 门:

$$\psi_3 = \frac{|000\rangle + |111\rangle}{\sqrt{2}}$$